

Header Linked List – Introduction

A Header Linked List is a special type of linked list in which an additional node called the header node is placed at the beginning of the list. Unlike ordinary nodes, the header node generally does not store actual user data. Instead, it stores information about the list such as:

- *Pointer to the first node*
- *Number of nodes in the list*
- *Special metadata or control information*

The header node simplifies many linked list operations such as insertion, deletion, and traversal because the list always starts with a fixed node.

In a conventional singly linked list, the head pointer directly points to the first data node:

```
typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *head = NULL;
```

But in a Header Linked List, we use a dedicated header node:

```
typedef struct HeaderNode
{
    int count;           // stores total number of nodes
    struct HeaderNode *next;
} HeaderNode;
```

Here:

- *count keeps track of the number of elements in the list.*
- *next points to the first actual data node.*

The structure of a Header Linked List is:

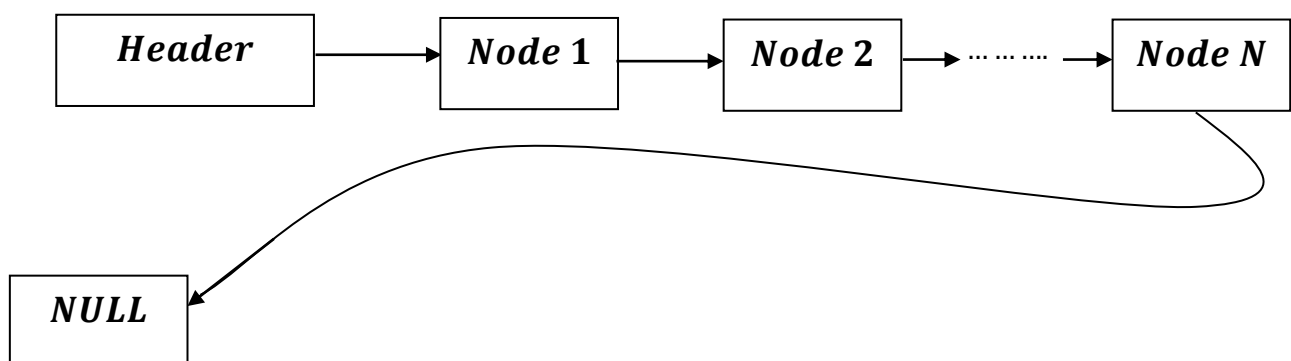
Though count is not mandatory for Header Node ,it can comprise of any information of the Linked List .

Such as:

One may optionally store additional information like:

- *Number of nodes (count)*
- *Last node pointer*
- *Maximum size*
- *Flags or status information*

The structure of a Header Linked List is:



The header node itself may contain:

- *Size of the list*
- *Starting address of the first node*
- *Special flags or control values*

There are mainly two types of Header Linked Lists:

1. *Grounded Header Linked List*
 - *The last node points to NULL.*
2. *Circular Header Linked List*
 - *The last node points back to the header node.*

Advantages of Header Linked List

- *Simplifies insertion and deletion operations*
- *Eliminates many special cases*
- *Easier traversal implementation*
- *Can store useful metadata like node count*
- *Improves list management efficiency*

Disadvantages of Header Linked List

- *Requires extra memory for the header node*
- *Slightly more complex implementation than a simple linked list*

Applications

- *Polynomial manipulation*
- *Sparse matrix representation*
- *Dynamic memory management*
- *Circular queue implementation*
- *Complex data structure management*

A Header Linked List provides a cleaner and more organized implementation of linked lists by introducing a dedicated control node at the beginning of the list.

.

2. Header Linked List

–Creation of Header List .

Program:

```
typedef struct Node
{
    int data;
    struct Node *next;
} Node;

void createHeaderList()
{
    // Allocate memory for the Header Node
    head = (Node *)malloc(sizeof(Node));

    if (head == nullptr) {
        cout << "Error: Memory allocation for Header
failed." << endl;
        exit(1);
    }

    // internal counter for position validation (metadata)
    head->data = 0;
    head->next = nullptr;
}

int main()
{
    createHeaderList();
}
```

Program Analysis

1. header->data (The Value)

- **Can it be any number? Yes.** Since `data` is defined as an `int`, you can set it to `-1`, `0`, `9999`, or any other integer.
- **Can it be a string or decimal? No.** It must match the type defined in our `struct`. If our `struct` says `int data;` we cannot store `"Hello"` or `3.14` inside it.
- **Does the value matter?**
 - **In a standard Header List:** The value is usually ignored (garbage). Developers often use `-1` or `0` just to indicate "this node is a dummy."
 - **In the Optimized version (my previous reply):** We deliberately used it to store the **Count** of items. In that specific logic, you *must* initialize it to `0`.

2. header->next (The Pointer) — CRITICAL

- **Can it be "anything"? Absolutely NOT.** This is the most dangerous part of pointers.
- **What MUST it be?** When we create the header node (and the list is empty), `header->next` **MUST be set to `NULL`** (or `nullptr` in modern C++).
- **Why?**
 - `NULL` acts as the "Stop Sign."
 - When we write functions like `display()` or `insertAtEnd()`, your `while` loop looks for `NULL` to know when to stop:

``while (temp → next != NULL) { ... }``

- If we leave `header->next` uninitialized or set it to "anything" (a random memory address), it becomes a **Dangling Pointer**. Hence our program will try to read memory it doesn't own, resulting in a **Segmentation Fault (Crash)**.

Hence , we have:

```
head = (Node *)malloc(sizeof(Node));
```

→ *The size of Header Node will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

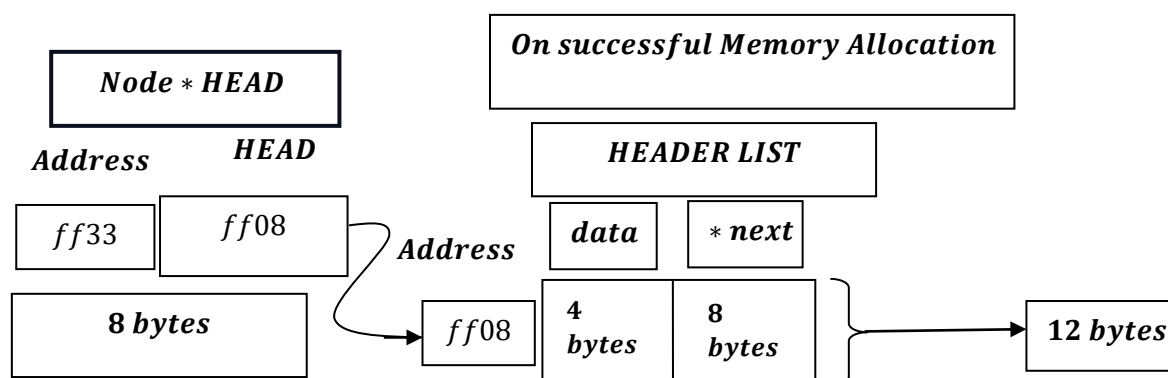
→ *The size of a pointer Header Node is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for next) .*



```

void createHeaderList()
{
    // Allocate memory for the Header Node
    head = (Node *)malloc(sizeof(Node));

    if (head == nullptr) {
        cout << "Error: Memory allocation for Header
failed." << endl;
        exit(1);
    }

    // internal counter for position validation (metadata)
    head->data = 0;
    head->next = nullptr;
}

```

Now, head = ff08, hence → Memory Allocation is Successful !

if head = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

And then exit(1) means abnormal termination of the program.

OR ,

Instead of exit(1) → we can use a macro : EXIT_FAILURE .

```

if (head == nullptr) {
    cout << "Error: Memory allocation for Header
failed." << endl;
    //exit(1);
    EXIT_FAILURE;
}

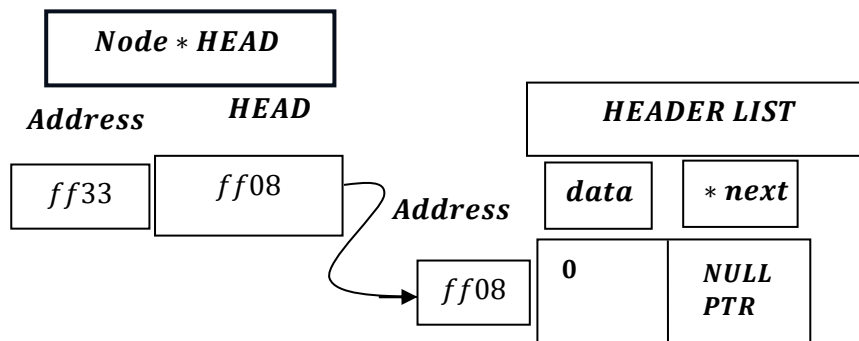
```

Also,

Instead of `exit(0)` → we can use a macro : `EXIT_SUCCESS`.

```
head->data = 0;  
head->next = nullptr;
```

Hence ,on successful memory allocation,we get:



3. Header Linked List

–Insert At Beginning.

Program:

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    // Link new node to the current first node
    newNode->next = head->next;

    // Link header to the new node
    head->next = newNode;

    // Increment internal count
    head->data++;

    cout << data << " inserted at the beginning." << endl;
}

...
case 1:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtBeginning(data);
    break;
```

Program Analysis

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a Node structure and *newNode* is declared as a pointer to Node structure.

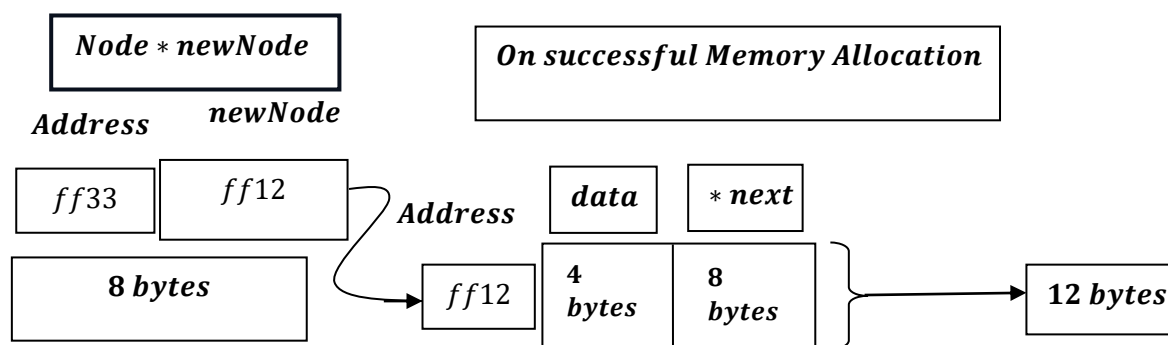
→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ *data*(an integer) is typically 4 bytes.

→ *next*(a pointer to another Node) is typically 8 bytes.

→ Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for *data* + 8 bytes for *next*) .



```

if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}

```

Now newNode = ff12, hence → Memory Allocation is Successful !

if newNode = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

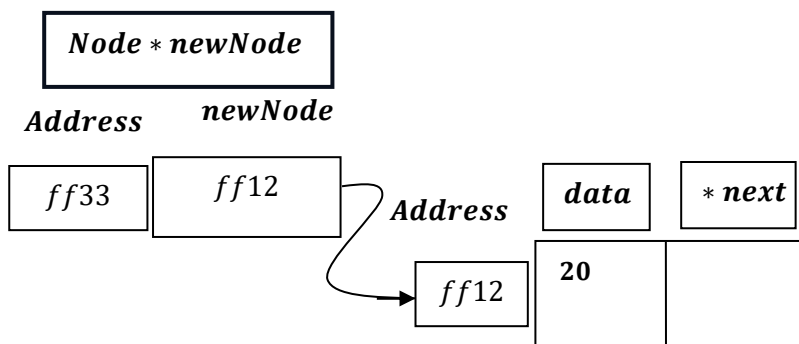
And then reurn will return void and exit from the module (function).

```

newNode->data = data;

```

Lets say , data = 20.

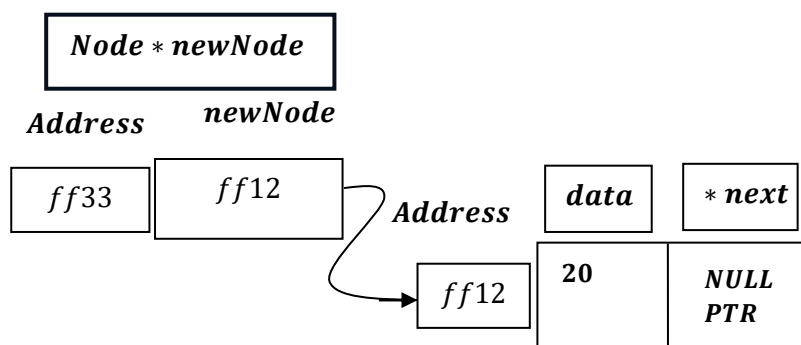
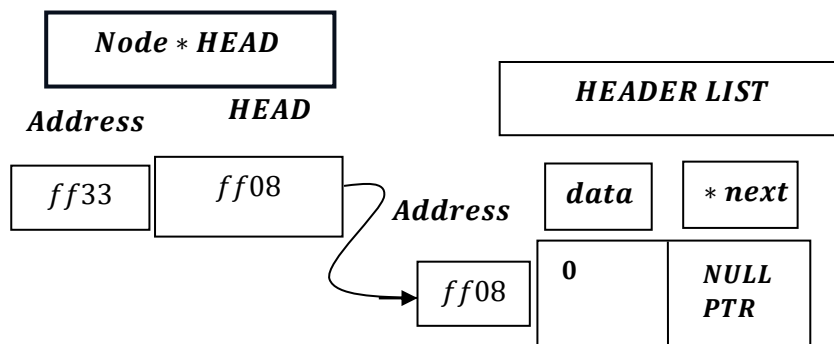


```

newNode->next = head->next;

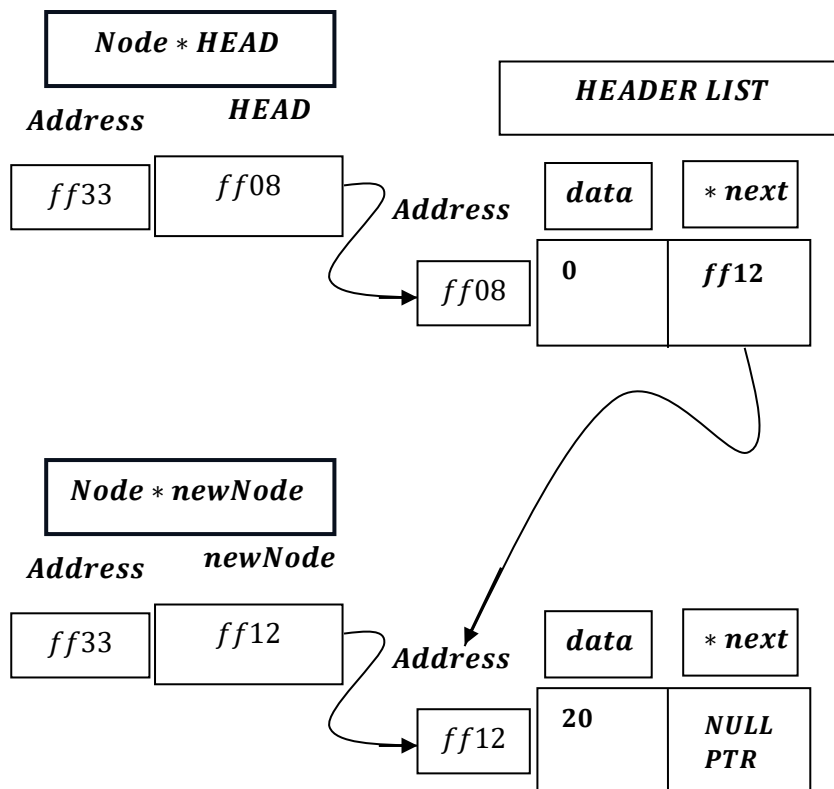
```

newNode → next = head → next = NULLPTR, as we get from:



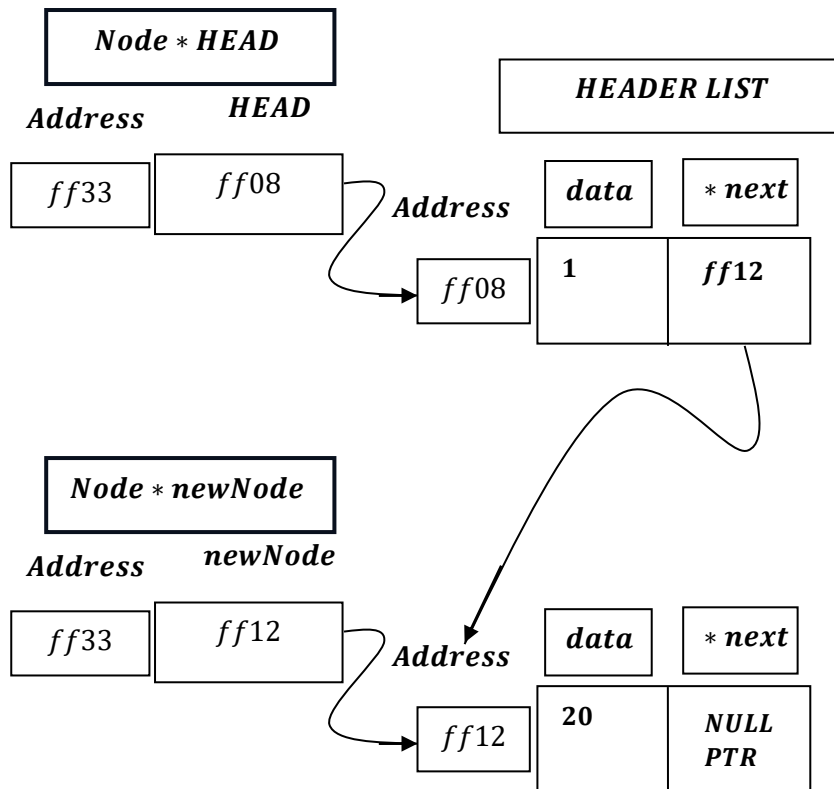
```
head->next = newNode;
```

head $\rightarrow$ ***next*** = ***newNode*** = ***ff12***.

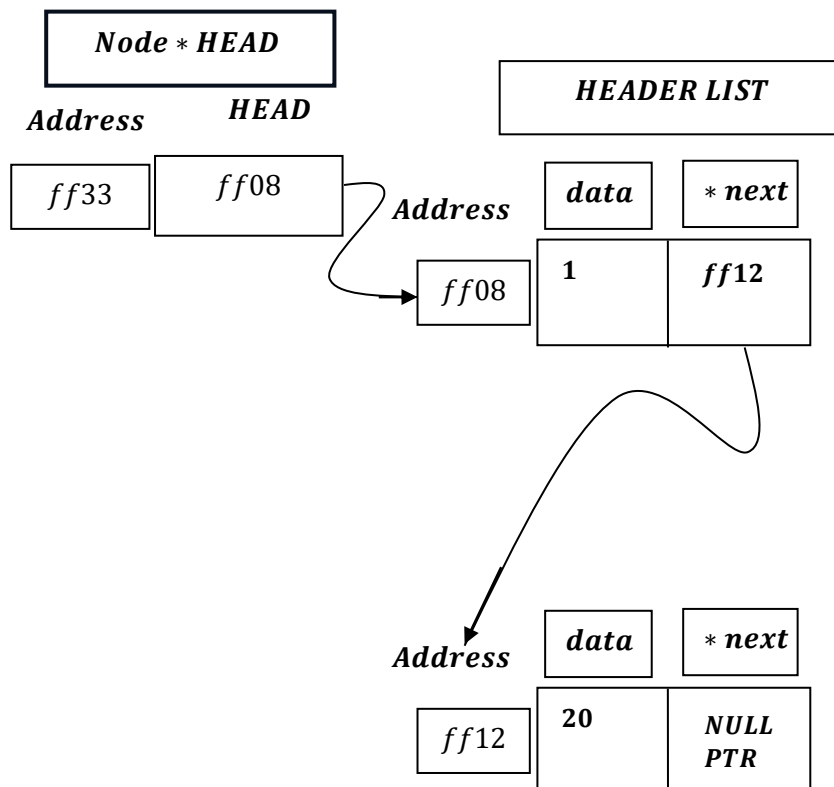


```
head->data++;
```

$$Head \rightarrow data ++ \Rightarrow Head \rightarrow data \Rightarrow Head \rightarrow data + 1 \Rightarrow 0 + 1 = 1.$$

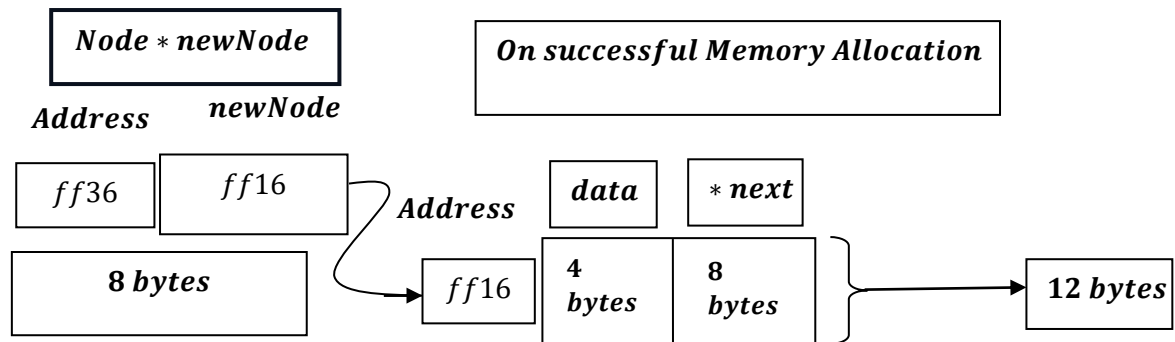


And we got our first list:



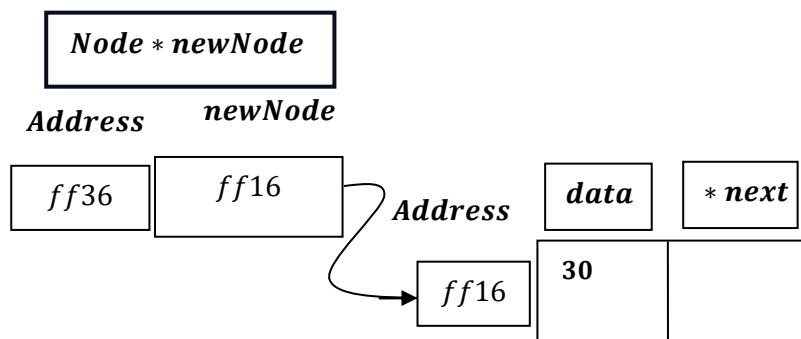
Similarly, for 2nd node we get:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

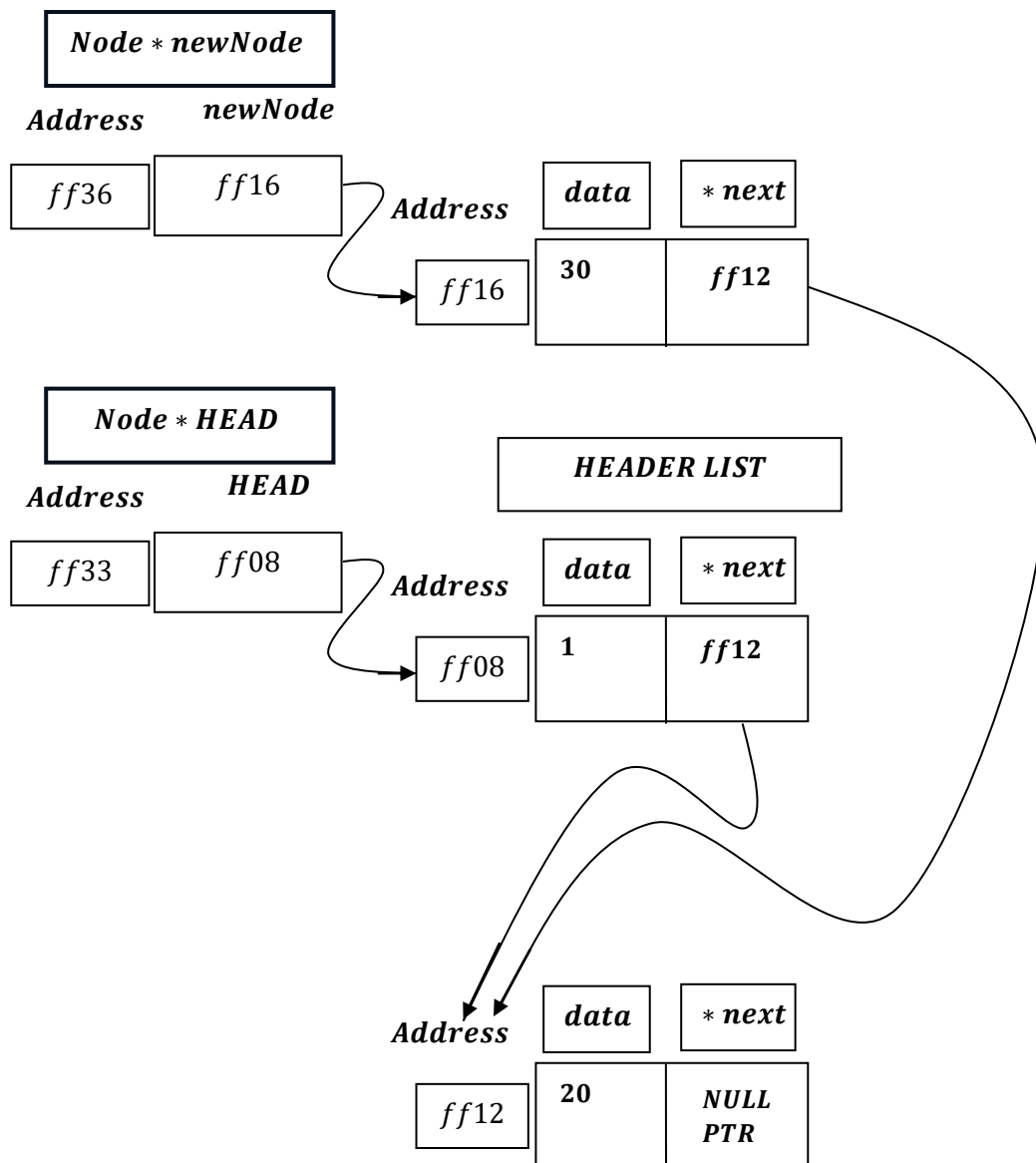


```
newNode->data = data;
```

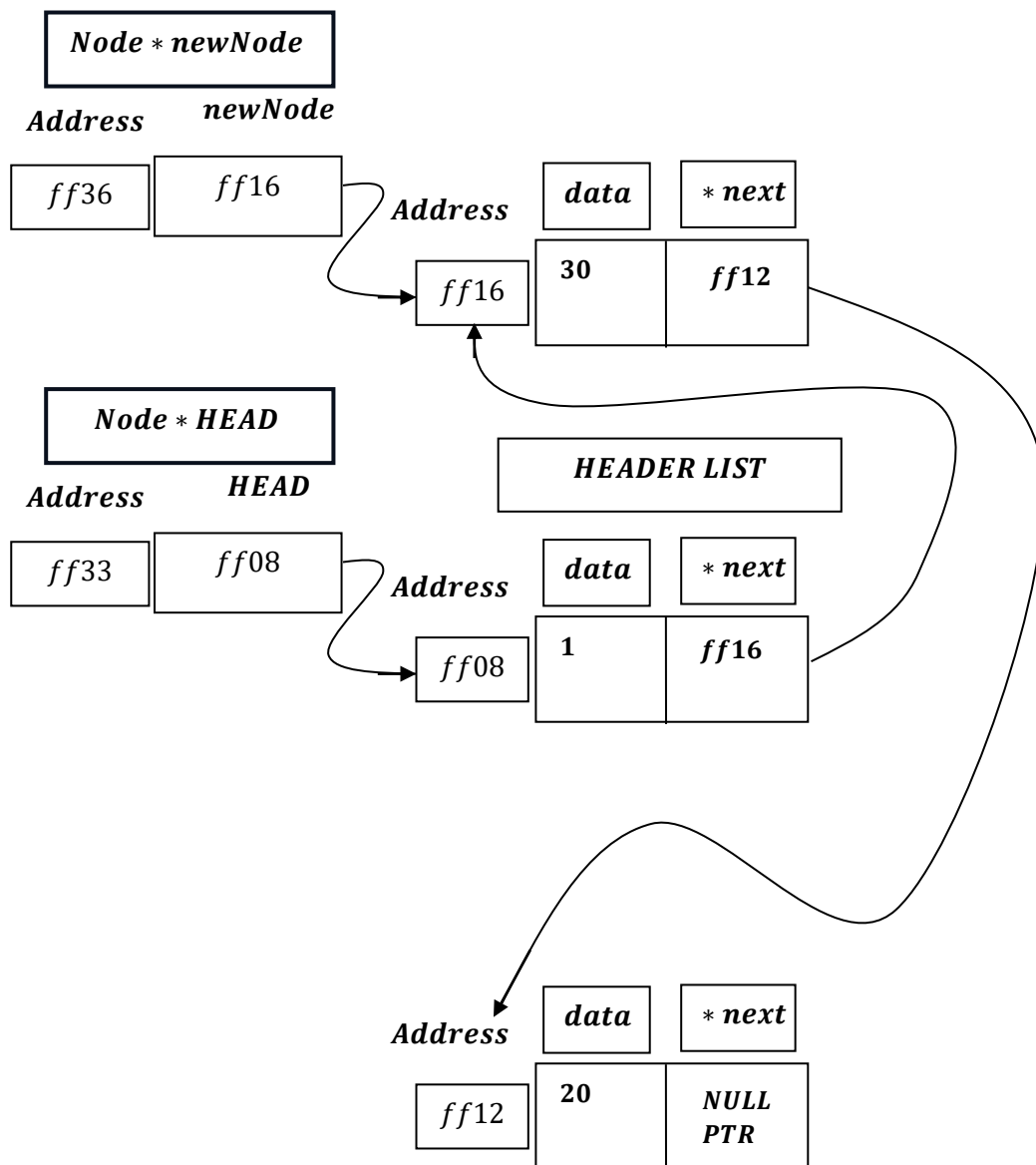
Lets say , $data = 30$.



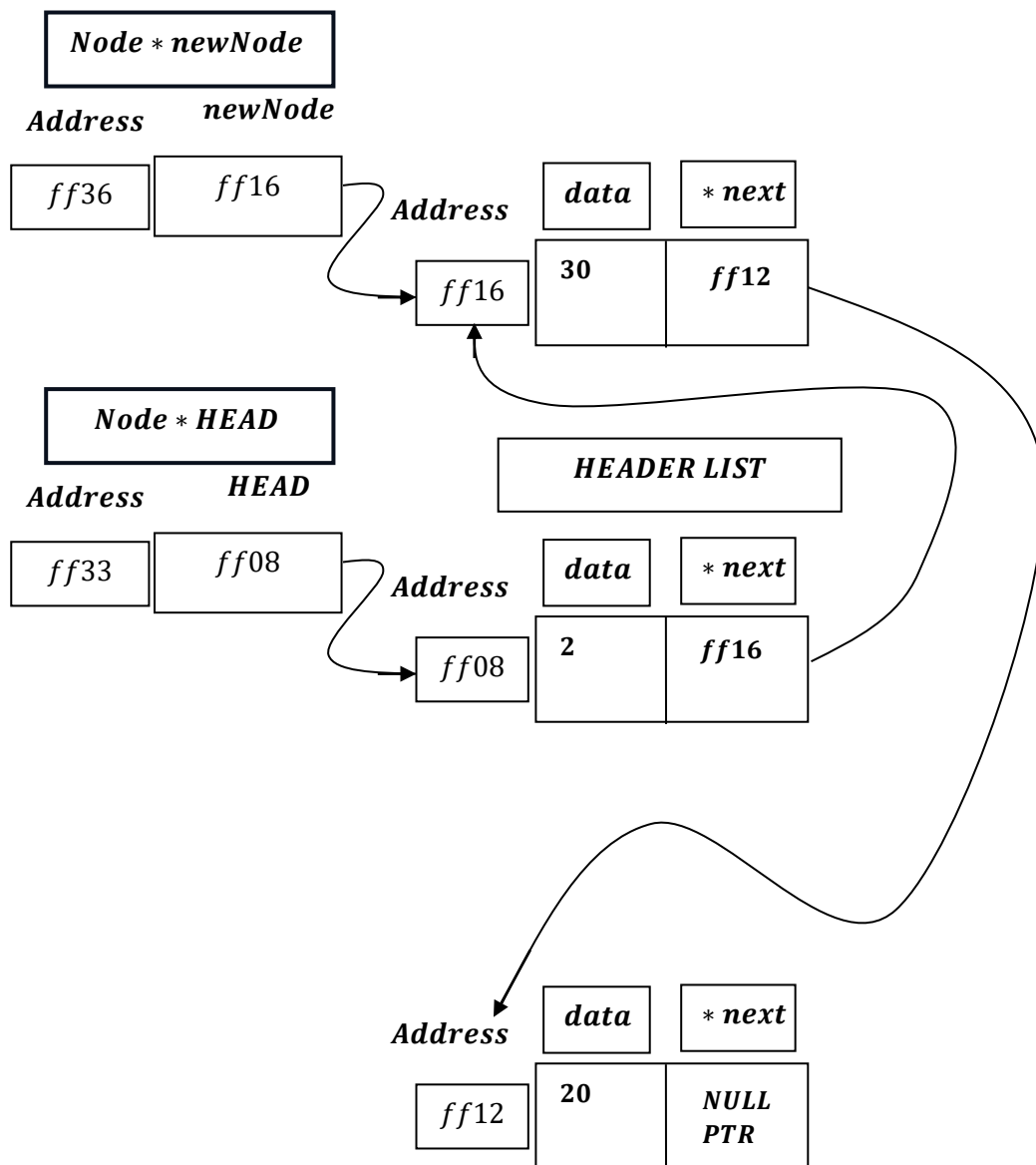
```
newNode->next = head->next;
```



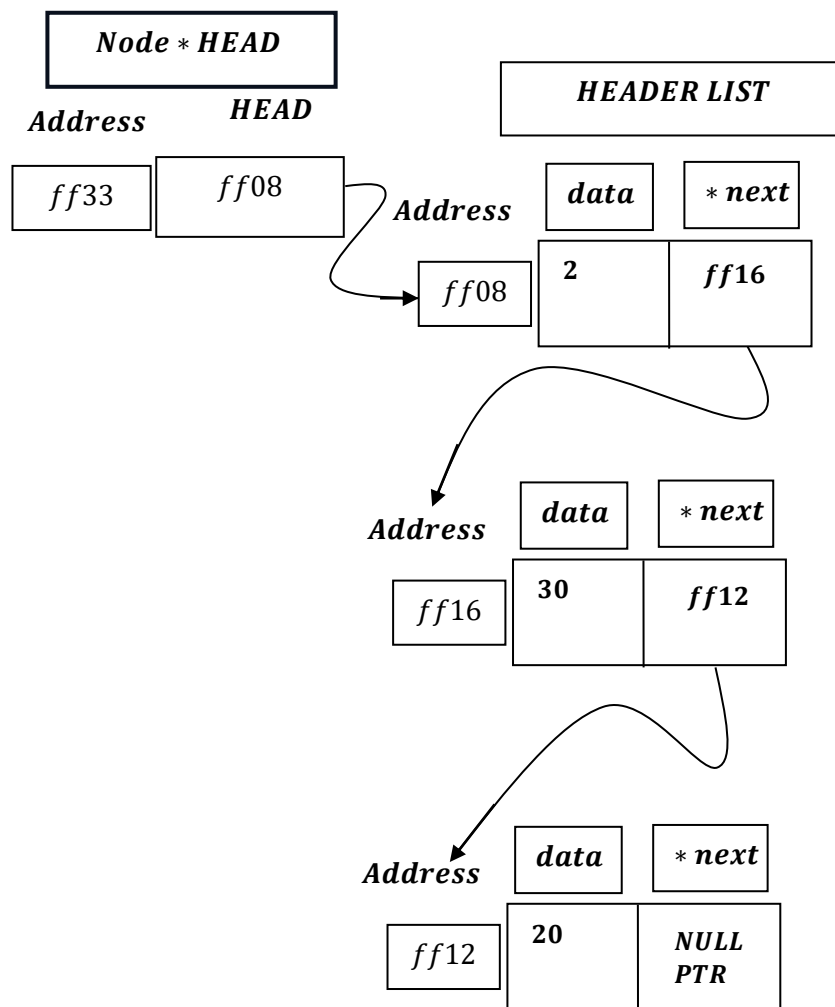
```
head->next = newNode;
```



```
head->data++;
```



And we got our second list:



4. Header Linked List

–Insert At END.

Program:

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr) {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;
    newNode->next = nullptr;

    // Start traversal from head
    Node *temp = head;

    // Move to the last node
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }

    // Link the new node
    temp->next = newNode;

    // Increment internal count
    head->data++;

    cout << data << " inserted at the end." << endl;
}

...case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

Program Analysis

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a *Node* structure and *newNode* is declared as a pointer to *Node* structure.

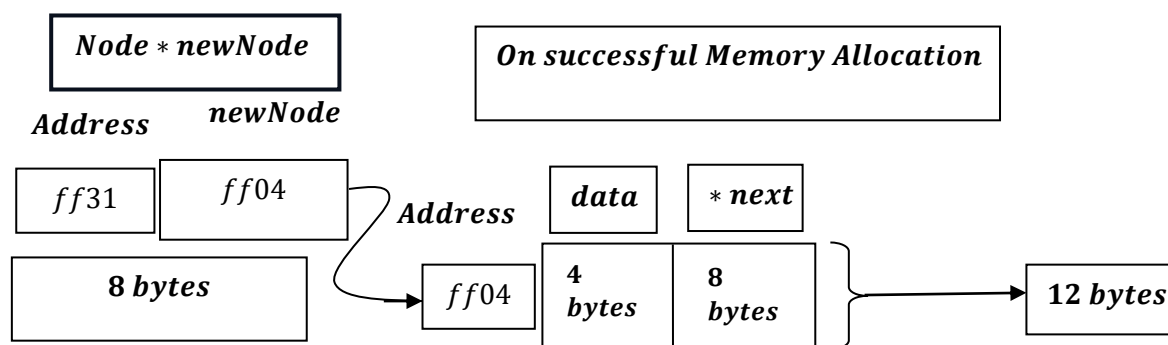
→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the *Node* structure itself is determined by the sum of the sizes of its members:

→ *data*(an integer) is typically 4 bytes.

→ *next*(a pointer to another *Node*) is typically 8 bytes.

→ Therefore ,the size of the *Node* structure is typically 12 bytes(4 bytes for *data* + 8 bytes for *next*) .



```

if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}

```

Now newNode = ff04, hence → Memory Allocation is Successful !

if newNode = NULL ,then :

Message → "Error: Memory Allocation for Header Failed."

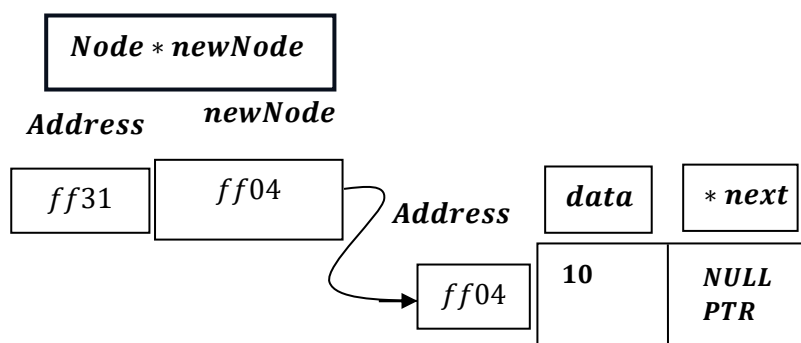
And then reurn will return void and exit from the module (function).

```

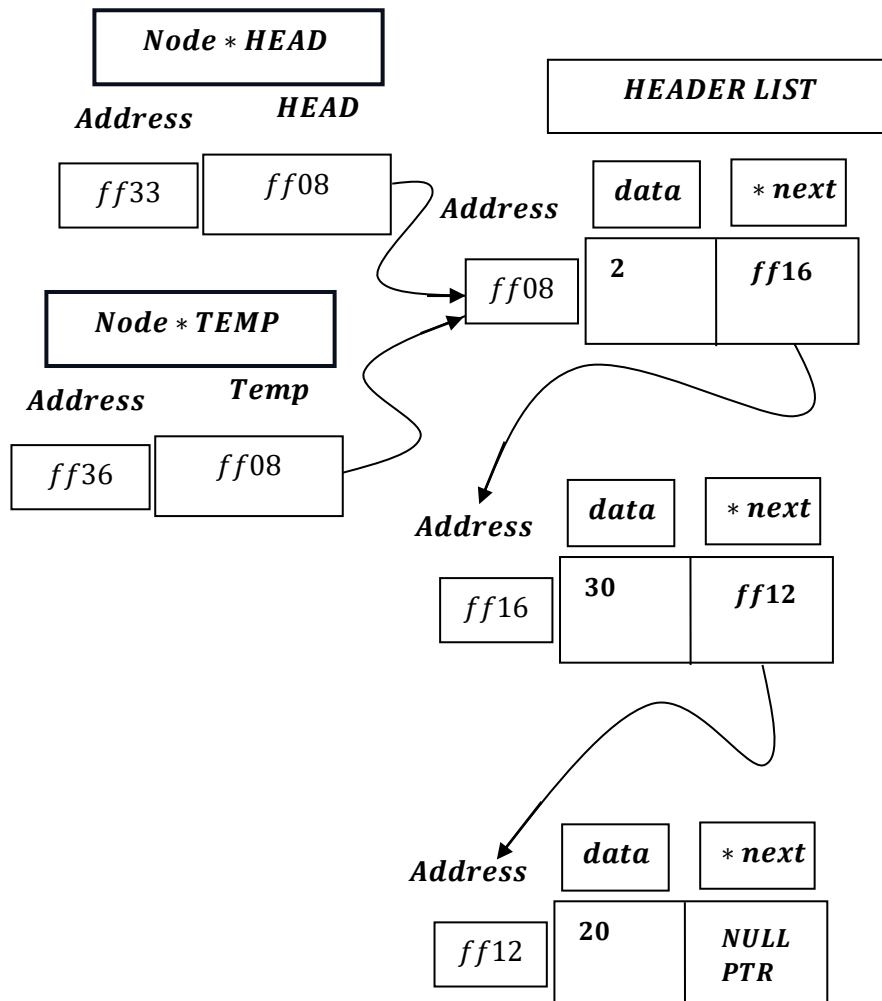
newNode->data = data;
newNode->next = nullptr;

```

Let data = 10, next = nullptr, then:



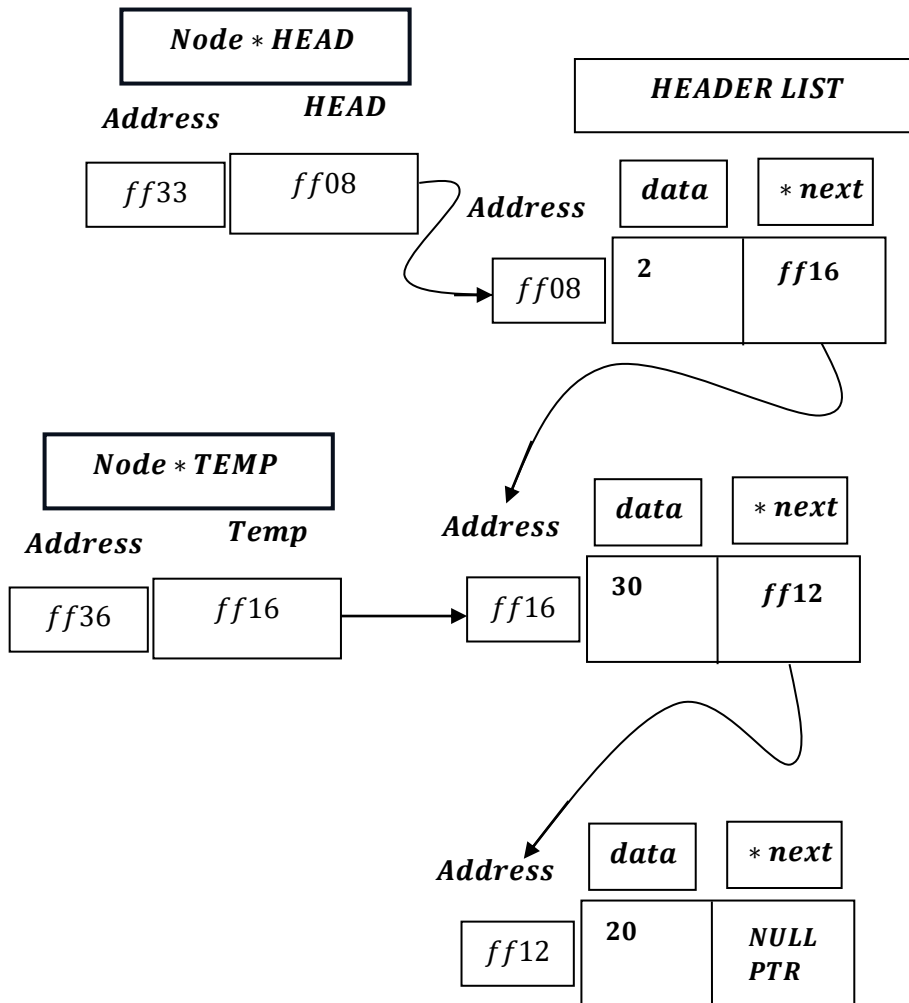
```
Node *temp = head;
```



```
while (temp->next != nullptr)
{
    temp = temp->next;
}
```

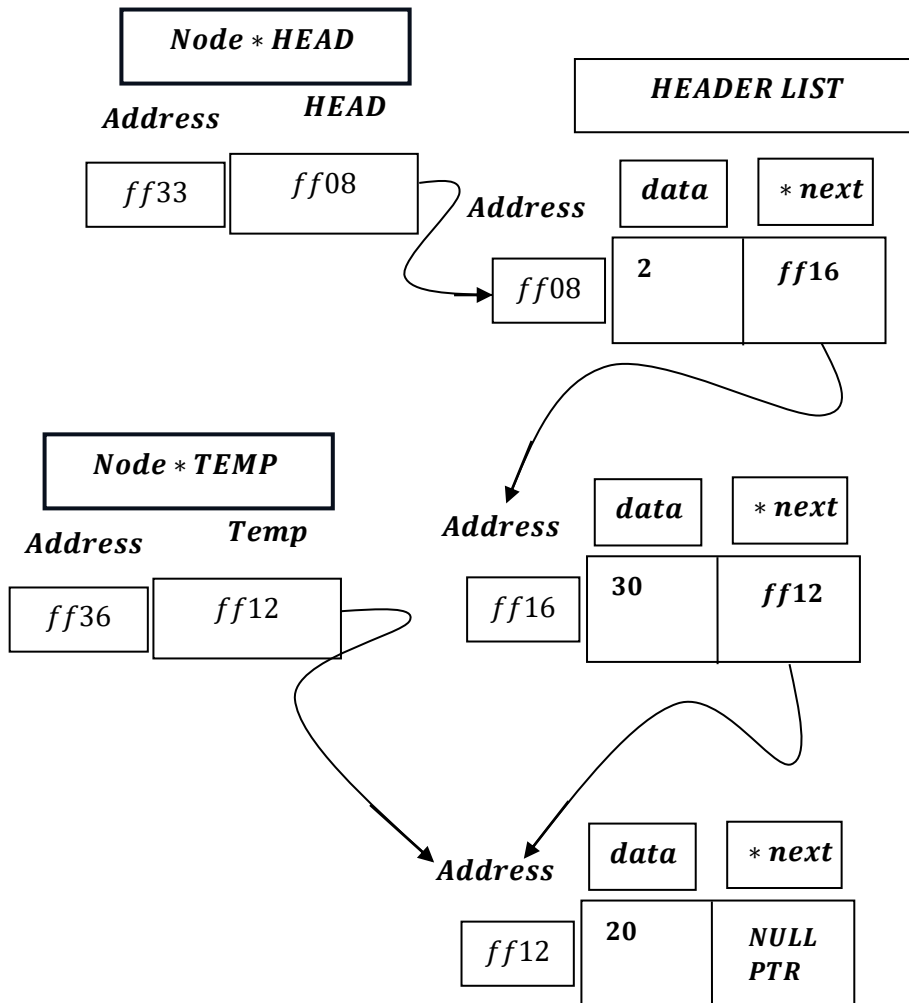
$temp \rightarrow next = ff16 \neq NULLPTR :$

$TEMP = TEMP \rightarrow next = ff16$



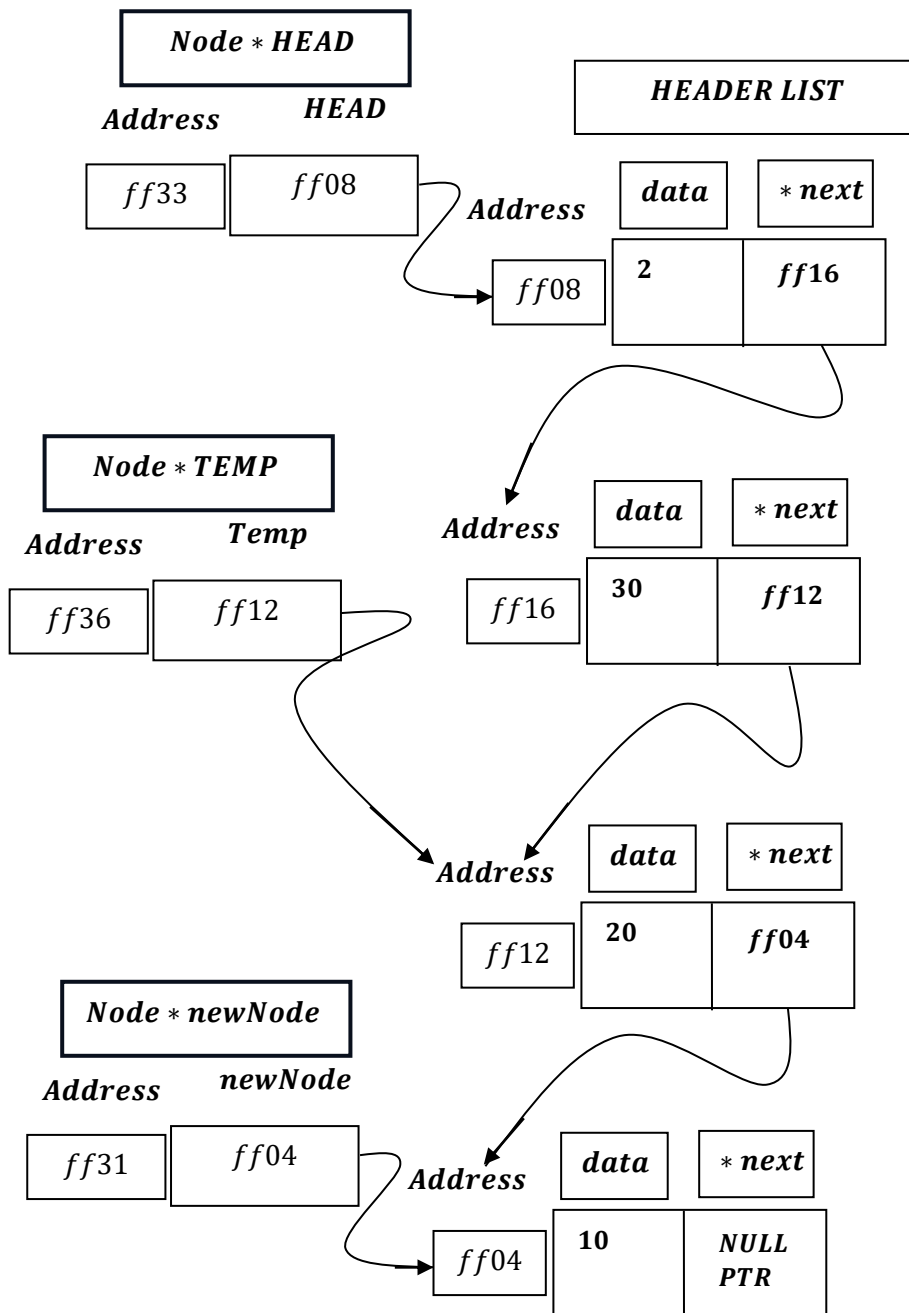
$temp \rightarrow next = ff12 \neq NULLPTR :$

$TEMP = TEMP \rightarrow next = ff12$

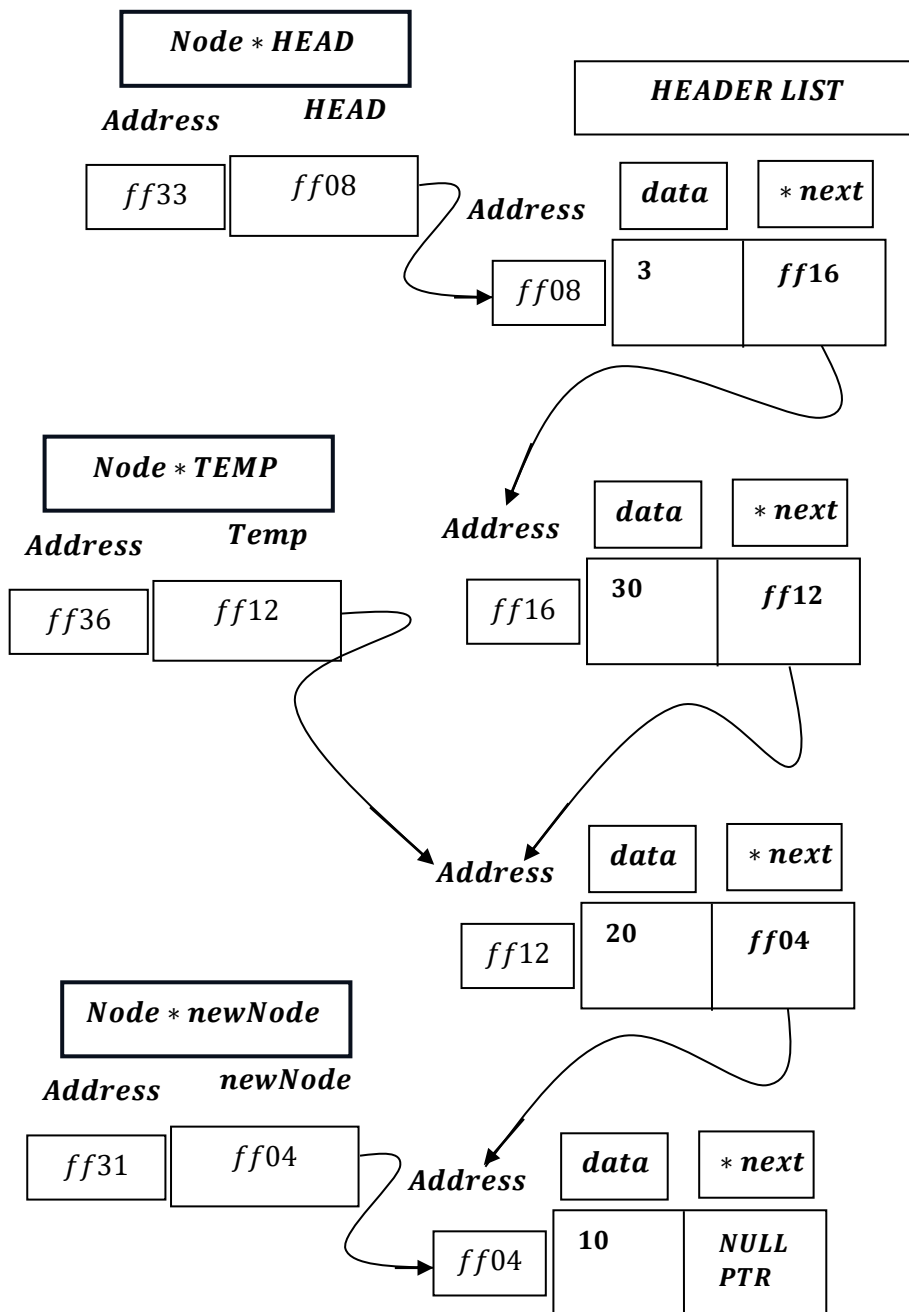


$temp \rightarrow next = NULLPTR \neq NULLPTR$ is false, hence exit from loop.

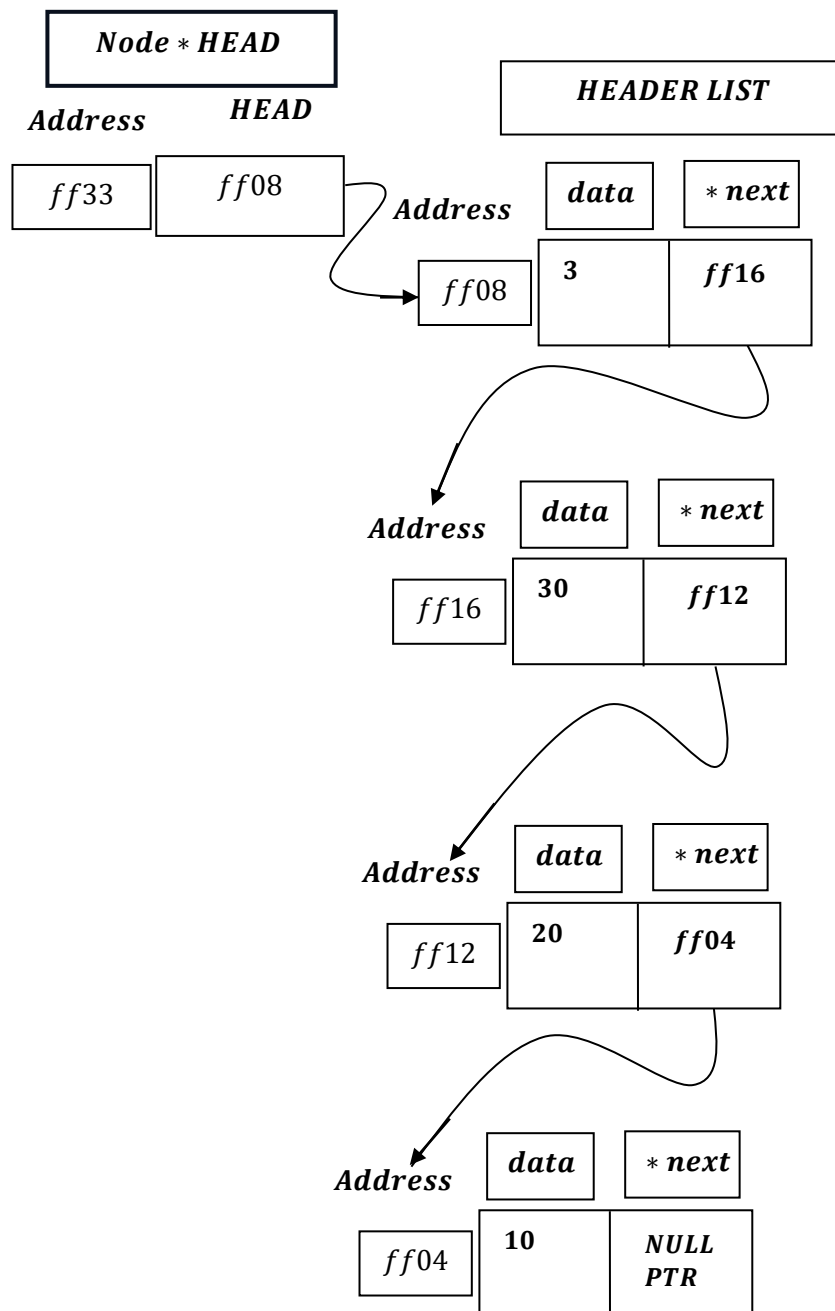
```
temp->next = newNode;
```



```
head->data++;
```



And, we get our final list:



5. Header Linked List

–Insert At Position

Program:

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1 || pos > head->data + 1)
    {
        cout << "Invalid position! Current list size: " <<
head->data << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr) {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    Node *temp = head;
    for (int i = 1; i < pos; i++)
    {
        temp = temp->next;
    }

    // Insert the node
    newNode->next = temp->next;
    temp->next = newNode;
    head->data++;

    cout << data << " inserted at position " << pos << "."
<< endl;
}

...

case 3:
    cout << "Enter data to insert: ";
    cin >> data;
```

```
cout << "Enter position: ";  
cin >> pos;  
insertAtPosition(data, pos);  
break;
```

Program Analysis

Now, if list has 4 nodes, Header Node's Data is also 4.

Now we can insert header node's data at position: 1, 2, 3, 4, 5. Greater than that of five (5) will result: Invalid Position.

Therefore, when `INVALID POSITION` : When $POS < 1$ or $Pos > Header\ Node's\ Data + 1$.

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

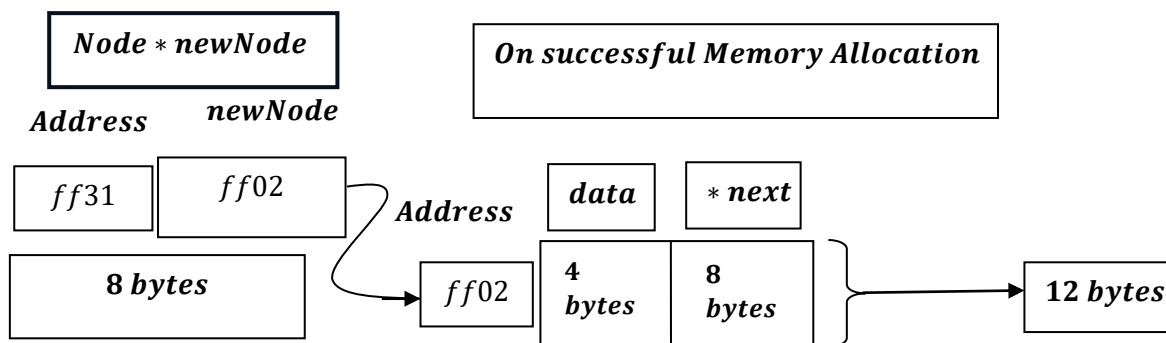
→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *Therefore ,the size of the Node structure is typically 12 bytes (4 bytes for data + 8 bytes for next) .*



```
if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}
```

Now newNode = ff04, hence → Memory Allocation is Successful !

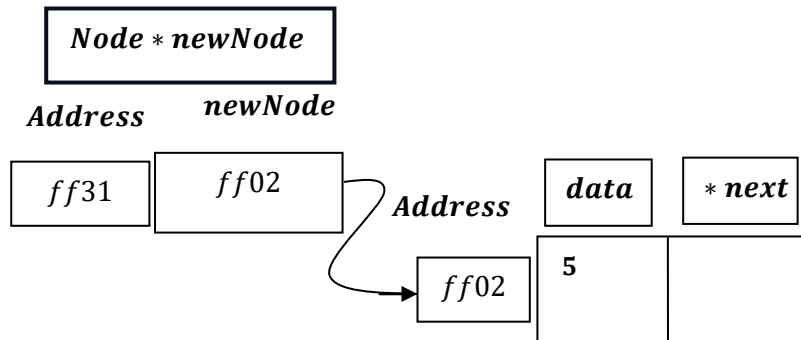
if newNode = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

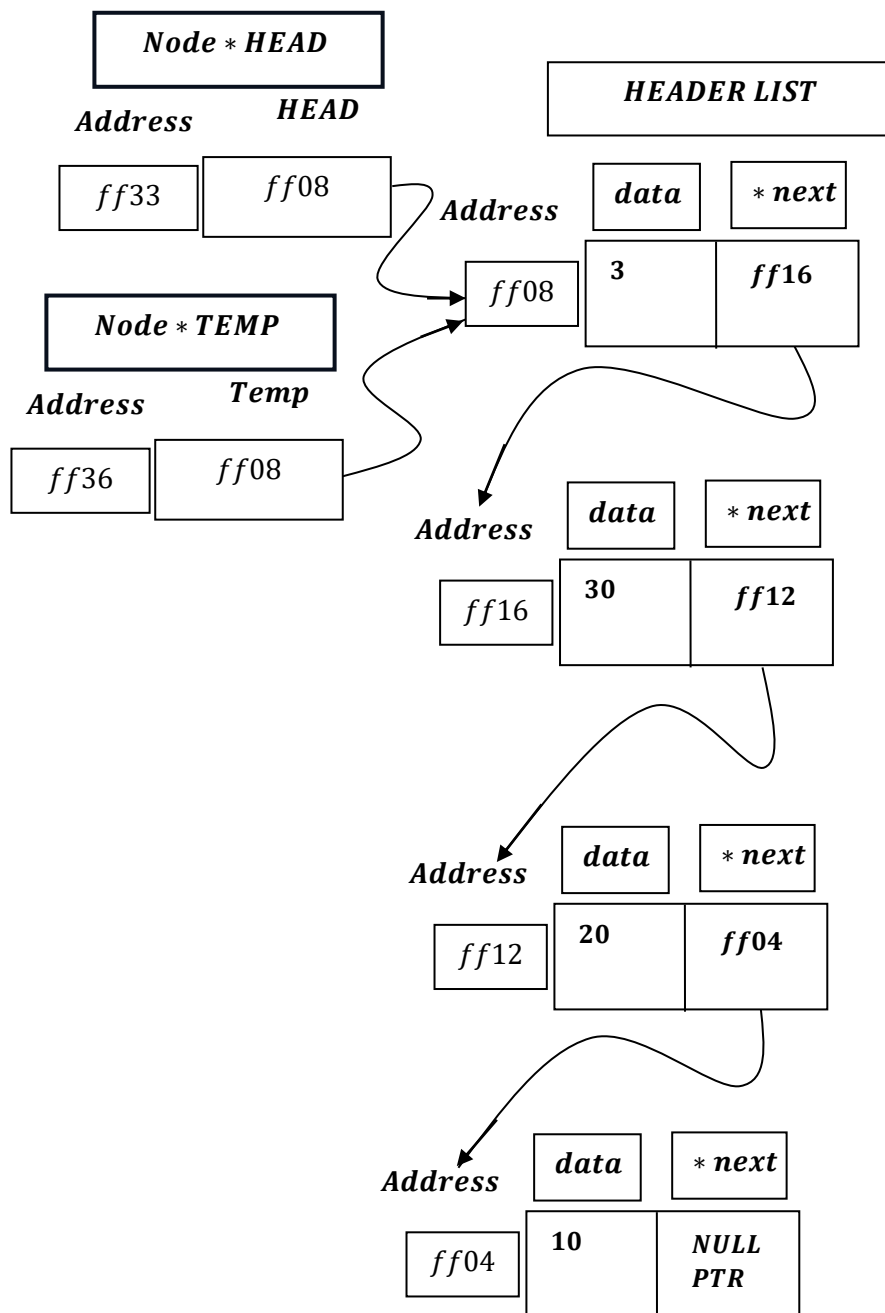
And then reurn will return void and exit from the module (function).

```
newNode->data = data;  
Node *temp = head;
```

Let data = 5.



*And , Node * temp = head;*



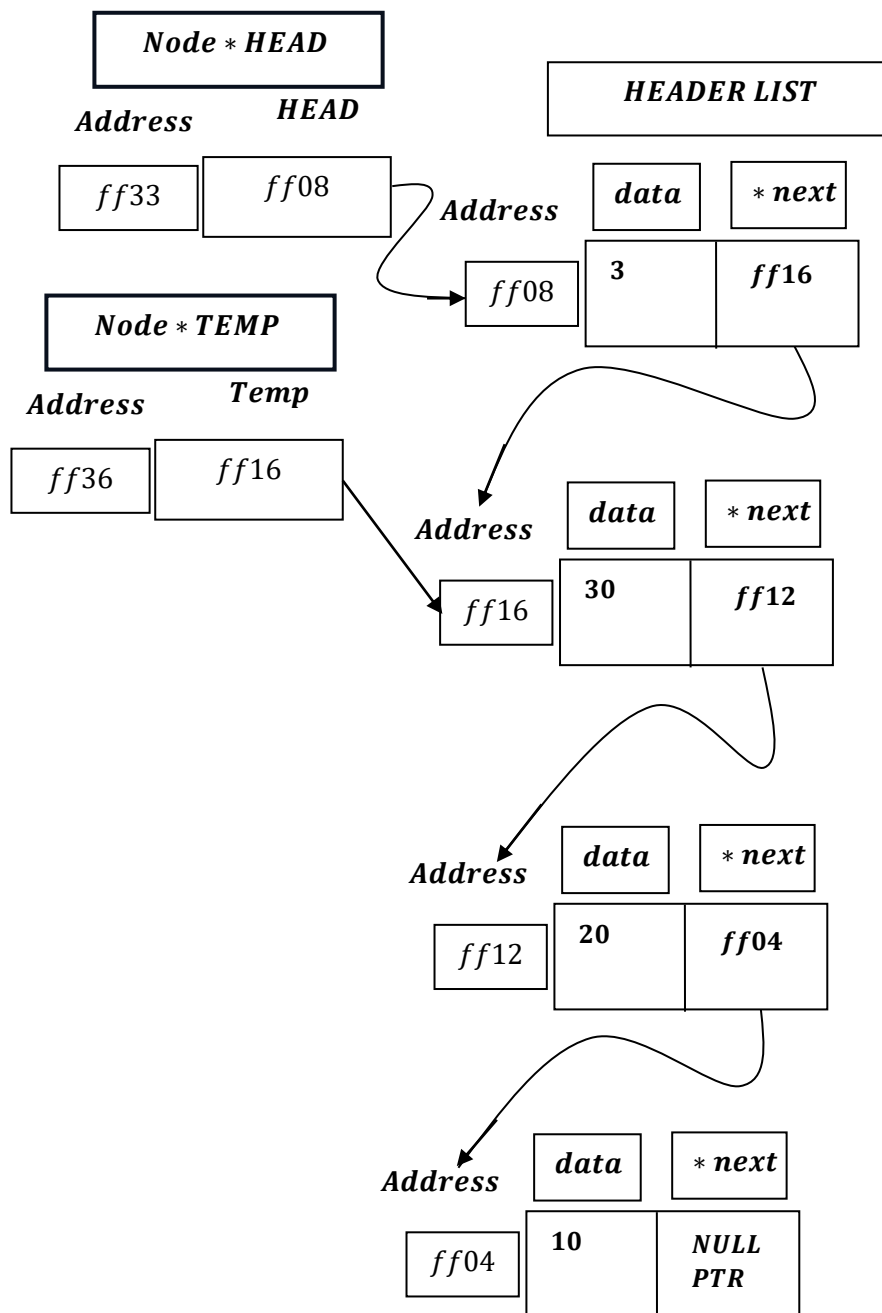
```
for (int i = 1; i < pos; i++)  
{  
    temp = temp->next;  
}
```

say, Position `pos`, pos: 3.

Hence,

$i = 1$ and $i < pos = 3$:

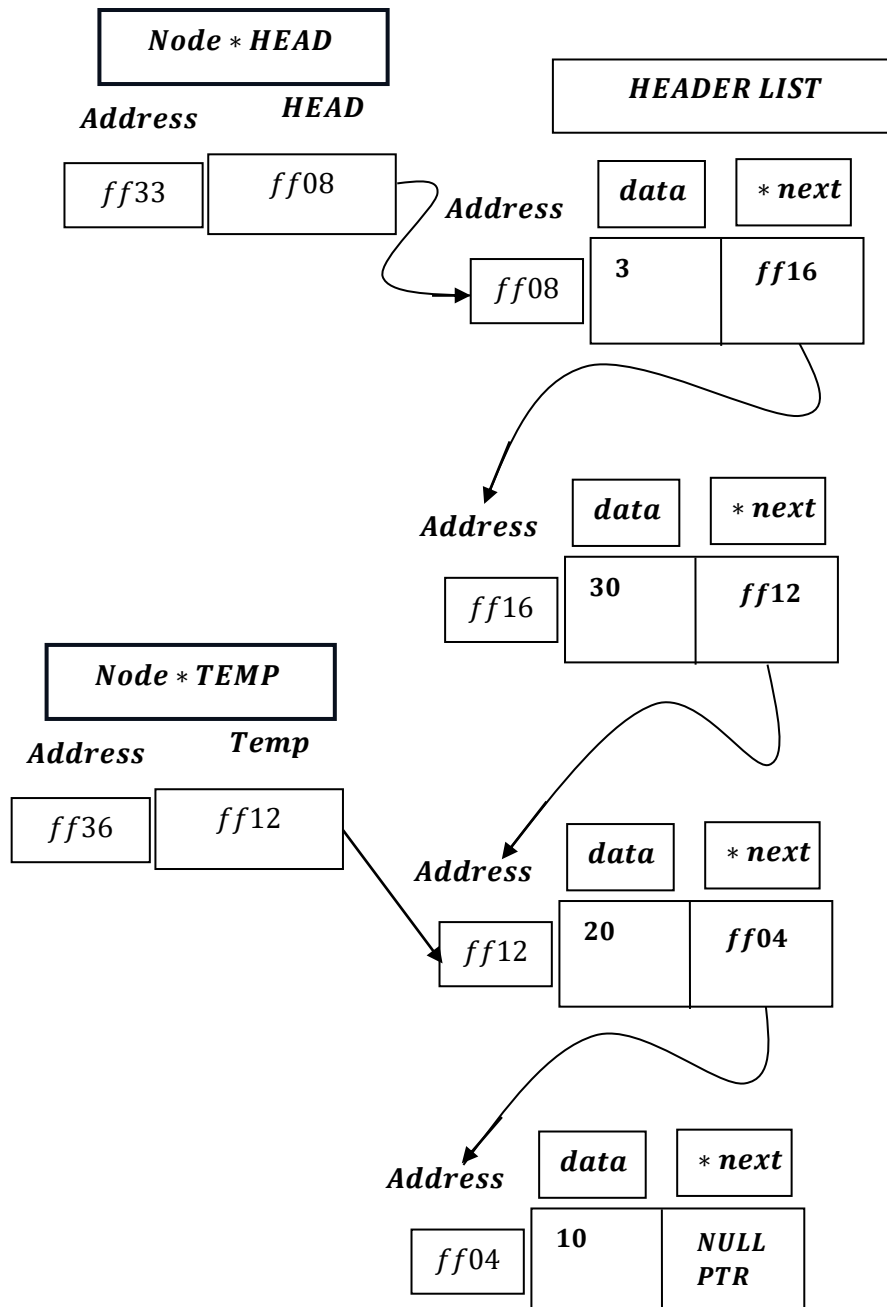
$temp = temp \rightarrow next = ff16$



$$i = i + 1 = 1 + 1 = 2.$$

$i = 2$ and $i < pos = 3$:

$temp = temp \rightarrow next = ff12$

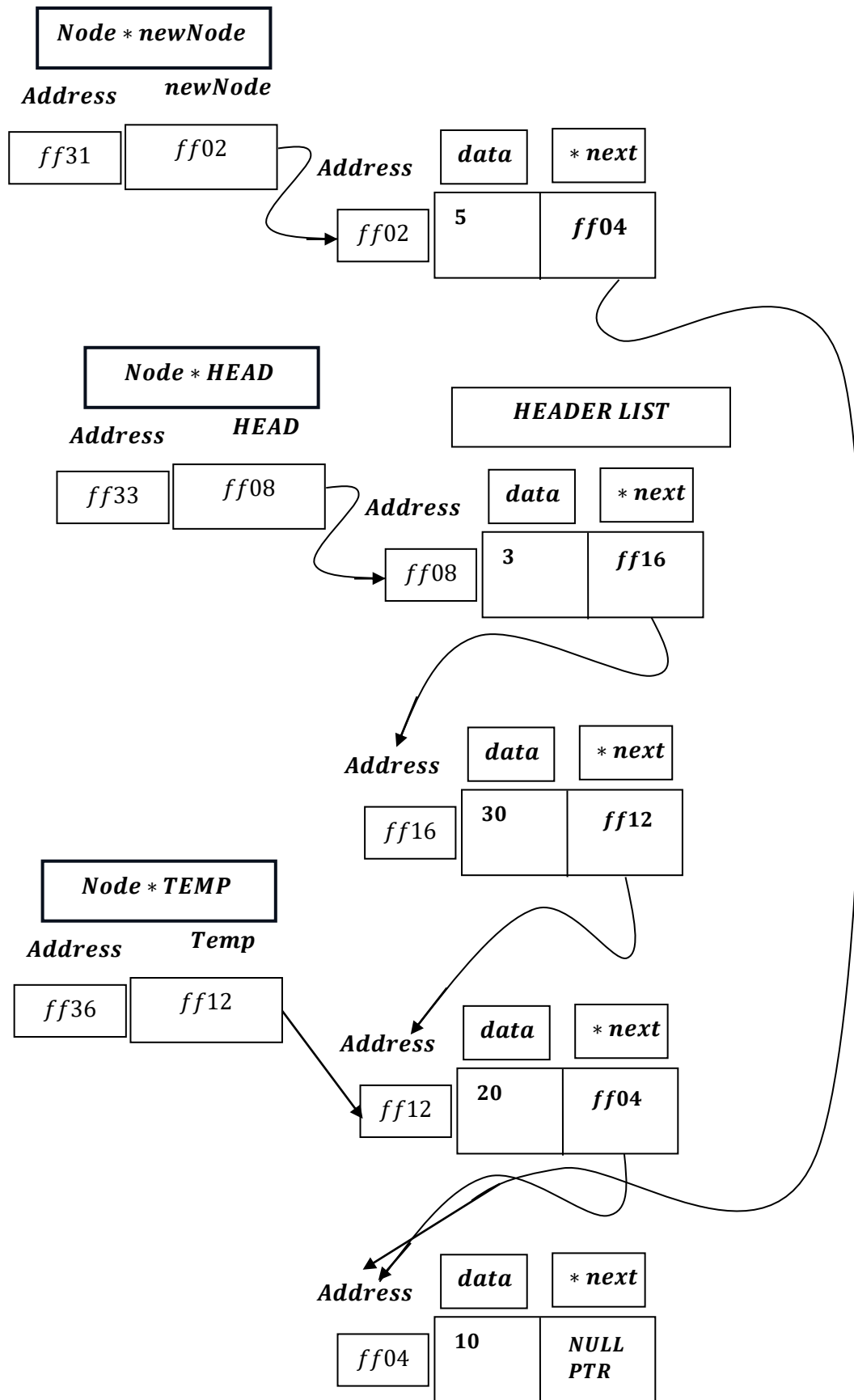


$i = i + 1 = 2 + 1 = 3.$

$i = 3$ and $i = 3 < pos = 3$ is false.

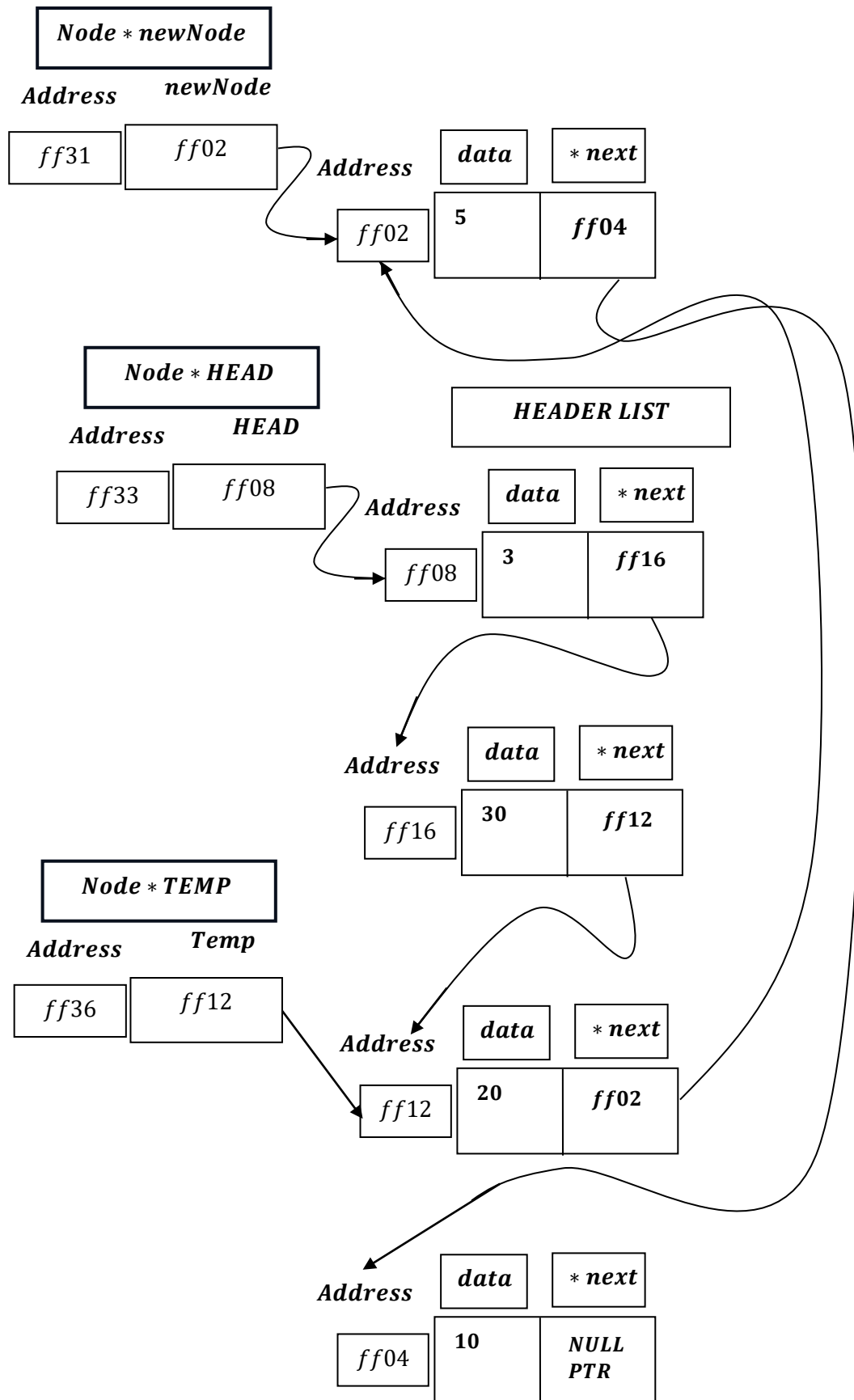
```
newNode->next = temp->next;
```

$newNode \rightarrow next = temp \rightarrow next = ff04$.

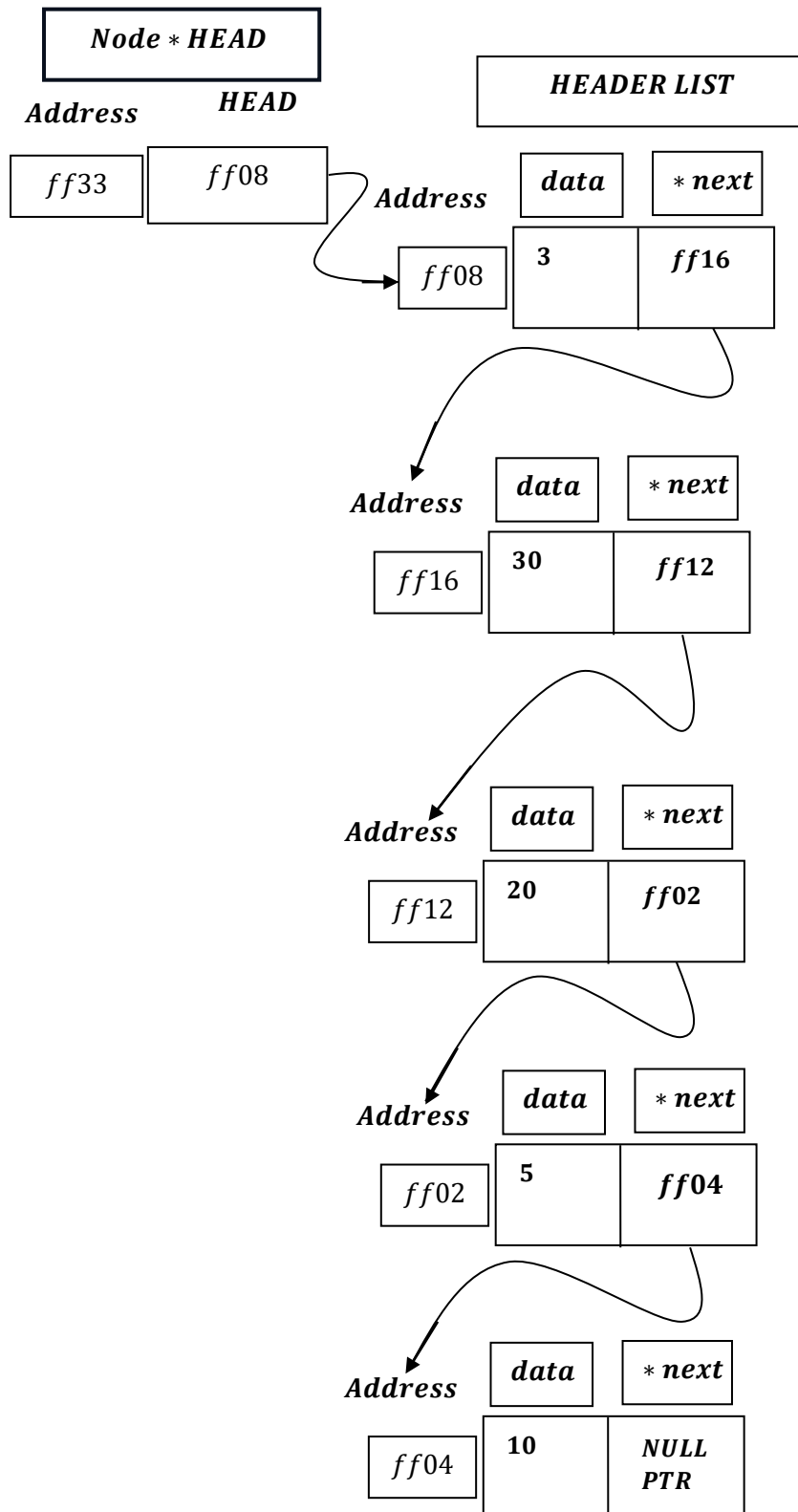


```
temp->next = newNode;
```

temp → *next* = *newNode* = *ff02*.

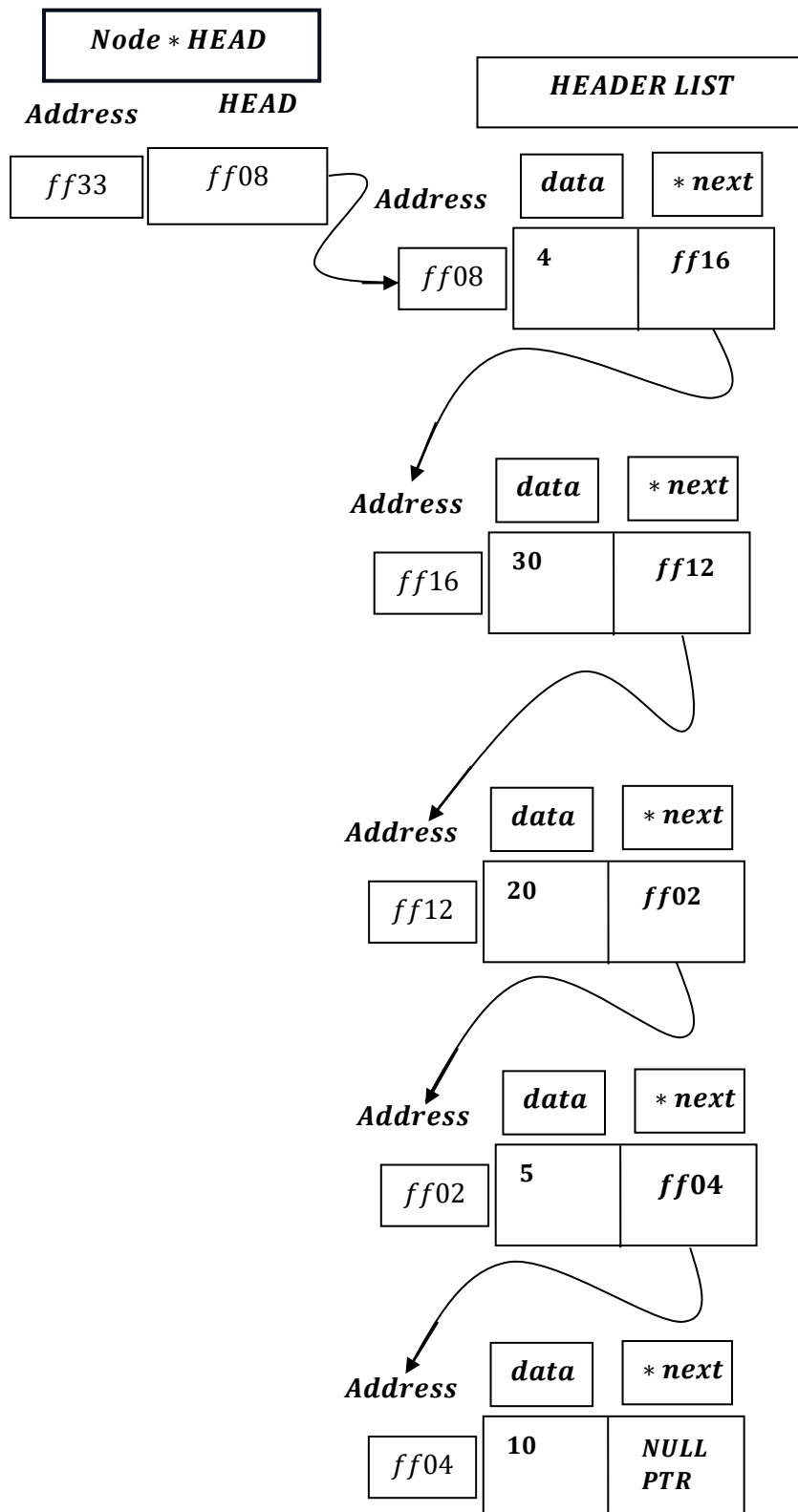


Hence we get our list:



`head->data++;`

$head \rightarrow data = head \rightarrow data + 1 = 3 + 1 = 4.$



6. Header Linked List
–Insert After Element

Program:

```
void insertAfterElement(int data, int element)
{
    Node *temp = head->next;
    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr) {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;

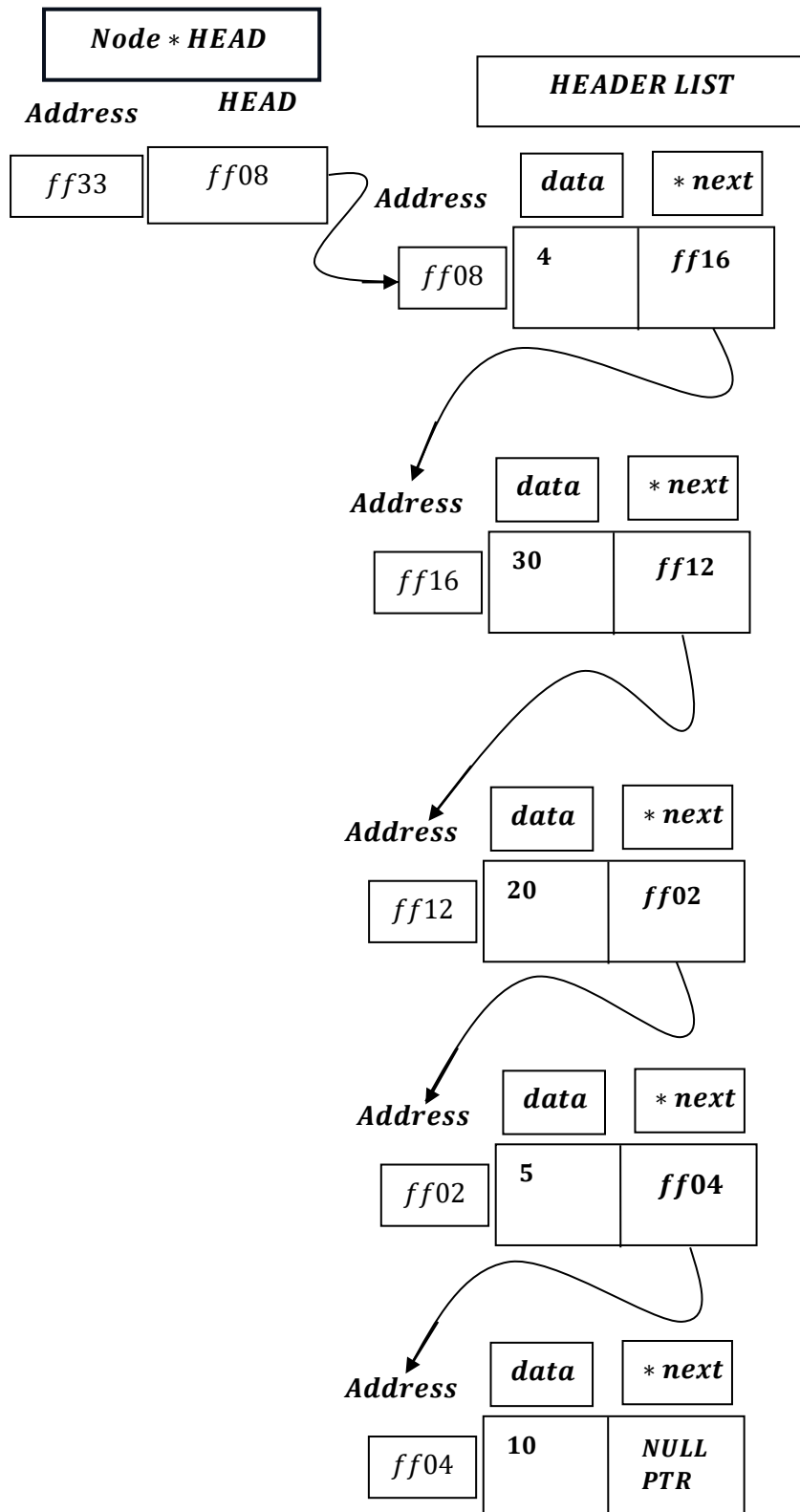
    // Link new node
    newNode->next = temp->next;
    temp->next = newNode;

    // Increment internal count
    head->data++;

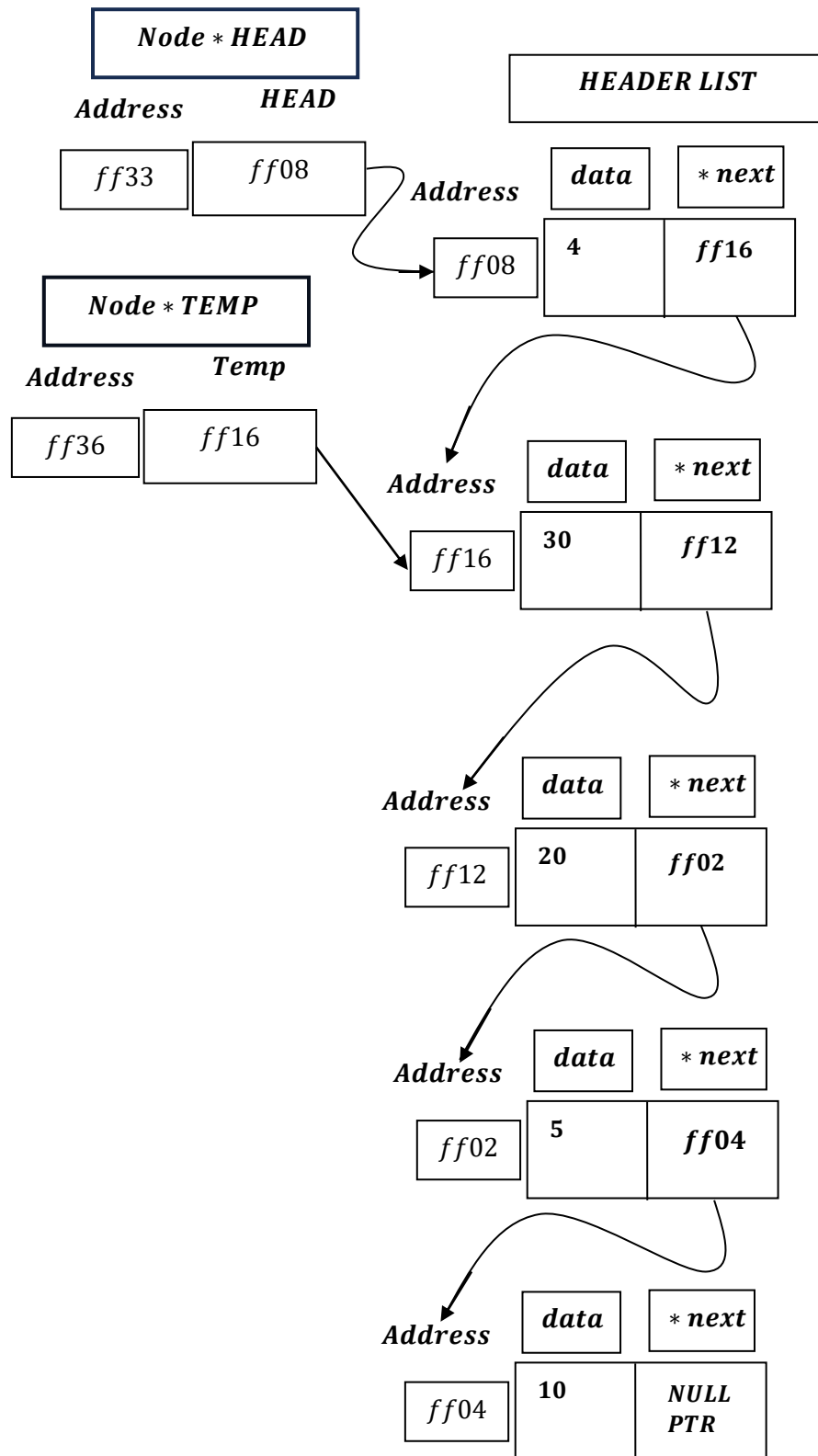
    cout << data << " inserted after " << element << "." <<
endl;
}
...•
```

```
case 4:  
    cout << "Enter data to insert: ";  
    cin >> data;  
    cout << "Enter the element to insert after: ";  
    cin >> element;  
    insertAfterElement(data, element);  
    break;
```

Program Analysis



`Node *temp = head->next;`
TEMP = head → next = ff16.

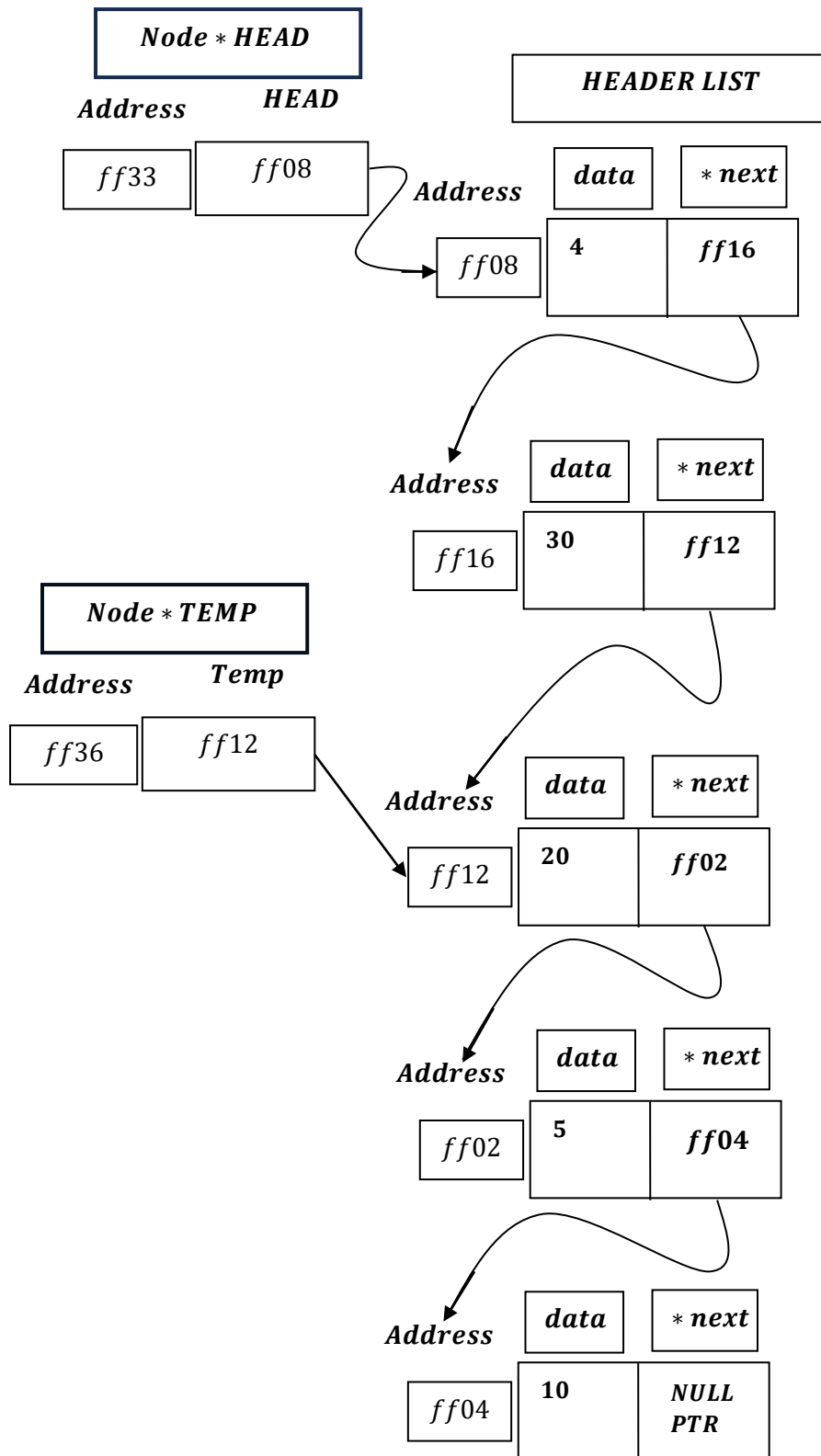


```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

Say after element = 10 and

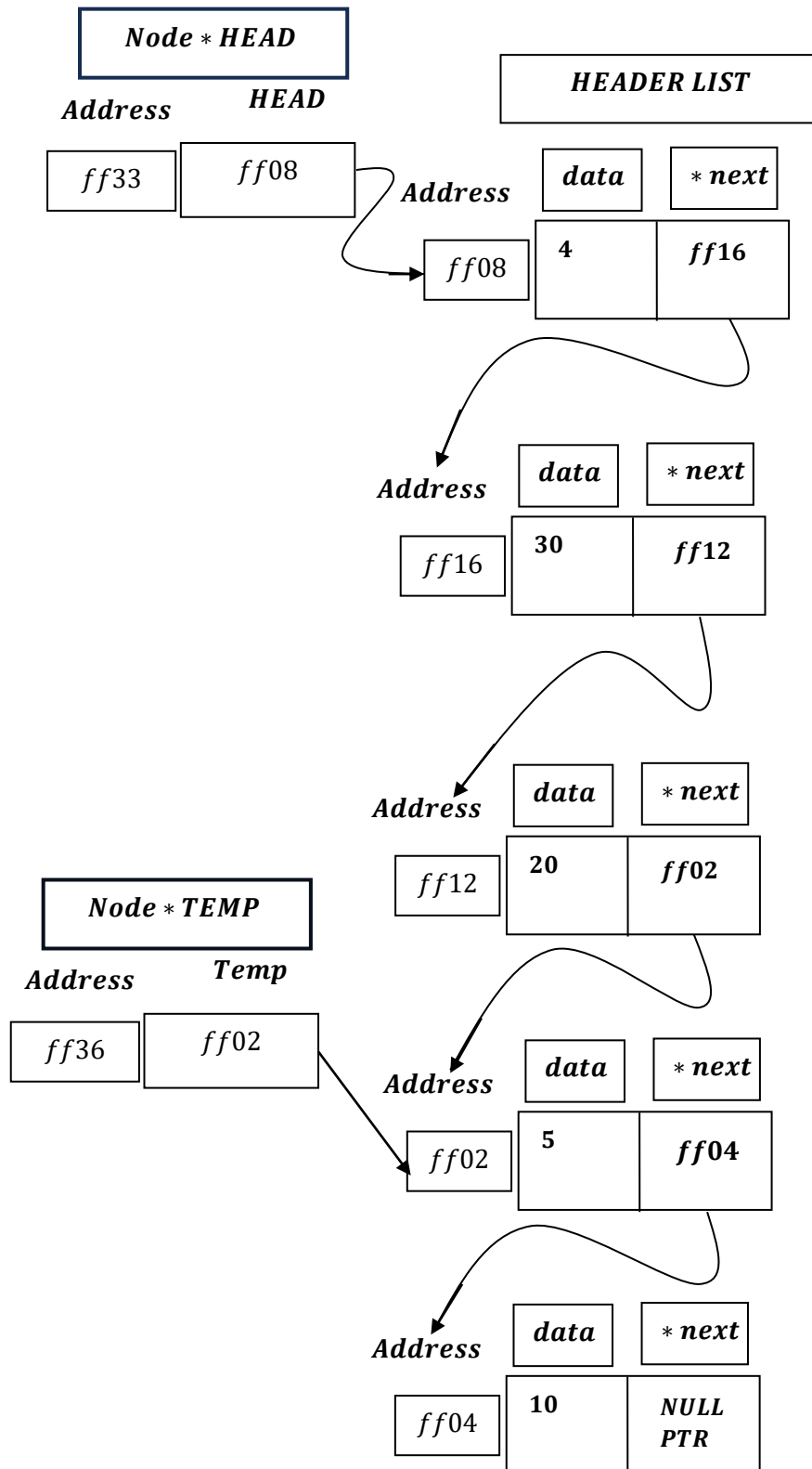
TEMP = ff16 ≠ NULLPTR AND TEMP → DATA = 30 ≠ ELEMENT : 10:

TEMP = TEMP → NEXT = ff12.



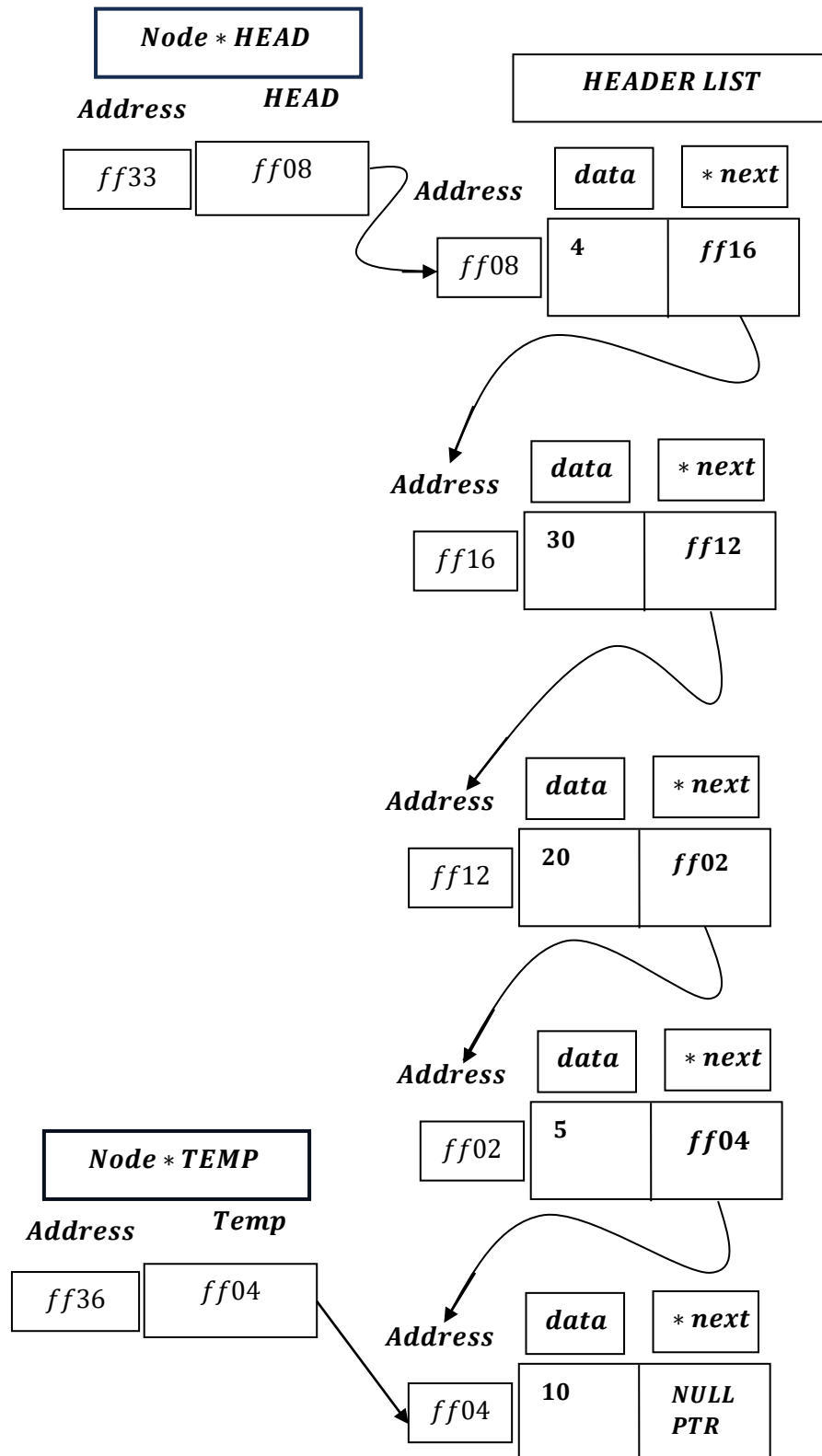
TEMP = ff12 ≠ NULLPTR AND TEMP → DATA = 20 ≠ ELEMENT : 10:

TEMP = TEMP → NEXT = ff02.



TEMP = ff02 ≠ NULLPTR AND TEMP → DATA = 5 ≠ ELEMENT : 10:

TEMP = TEMP → NEXT = ff04.



TEMP = ff04 ≠ NULLPTR AND TEMP → DATA = 10 ≠ ELEMENT : 10:

Condition is False and hence , exit from loop.

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

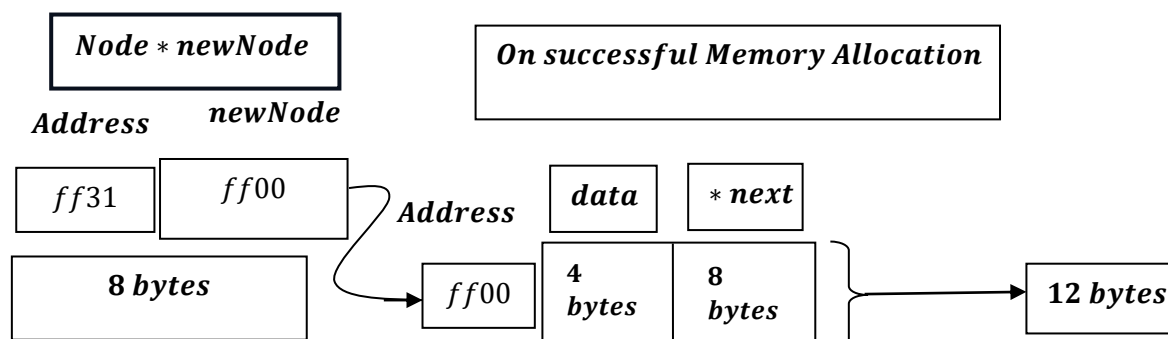
→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *Therefore ,the size of the Node structure is typically 12 bytes (4 bytes for data + 8 bytes for next) .*



```

if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}

```

Now newNode = ff00, hence → Memory Allocation is Successful !

if newNode = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

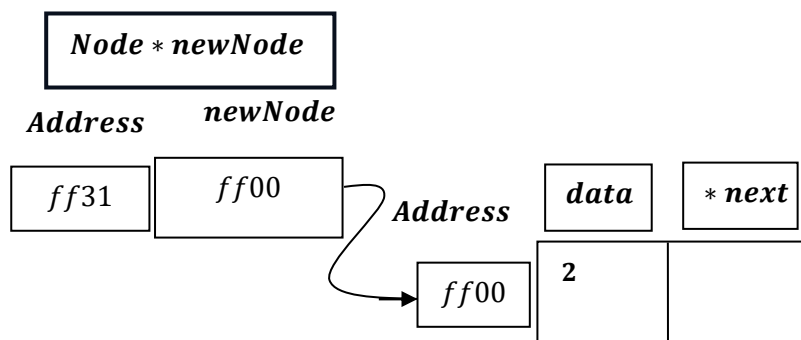
And then reurn will return void and exit from the module (function).

```

newNode->data = data;

```

Say, data = 2 , hence : newNode → data = data = 2.

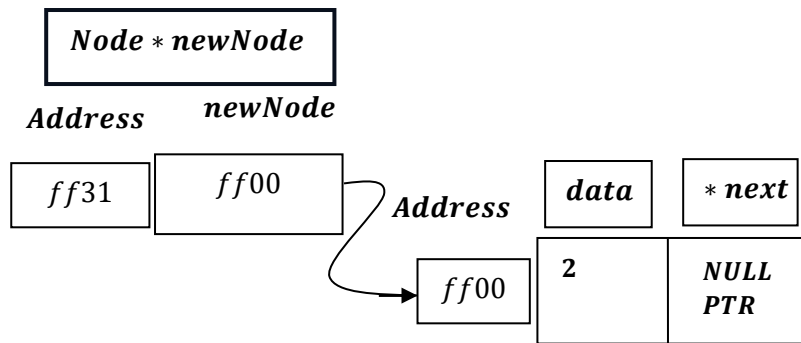


```

newNode->next = temp->next;

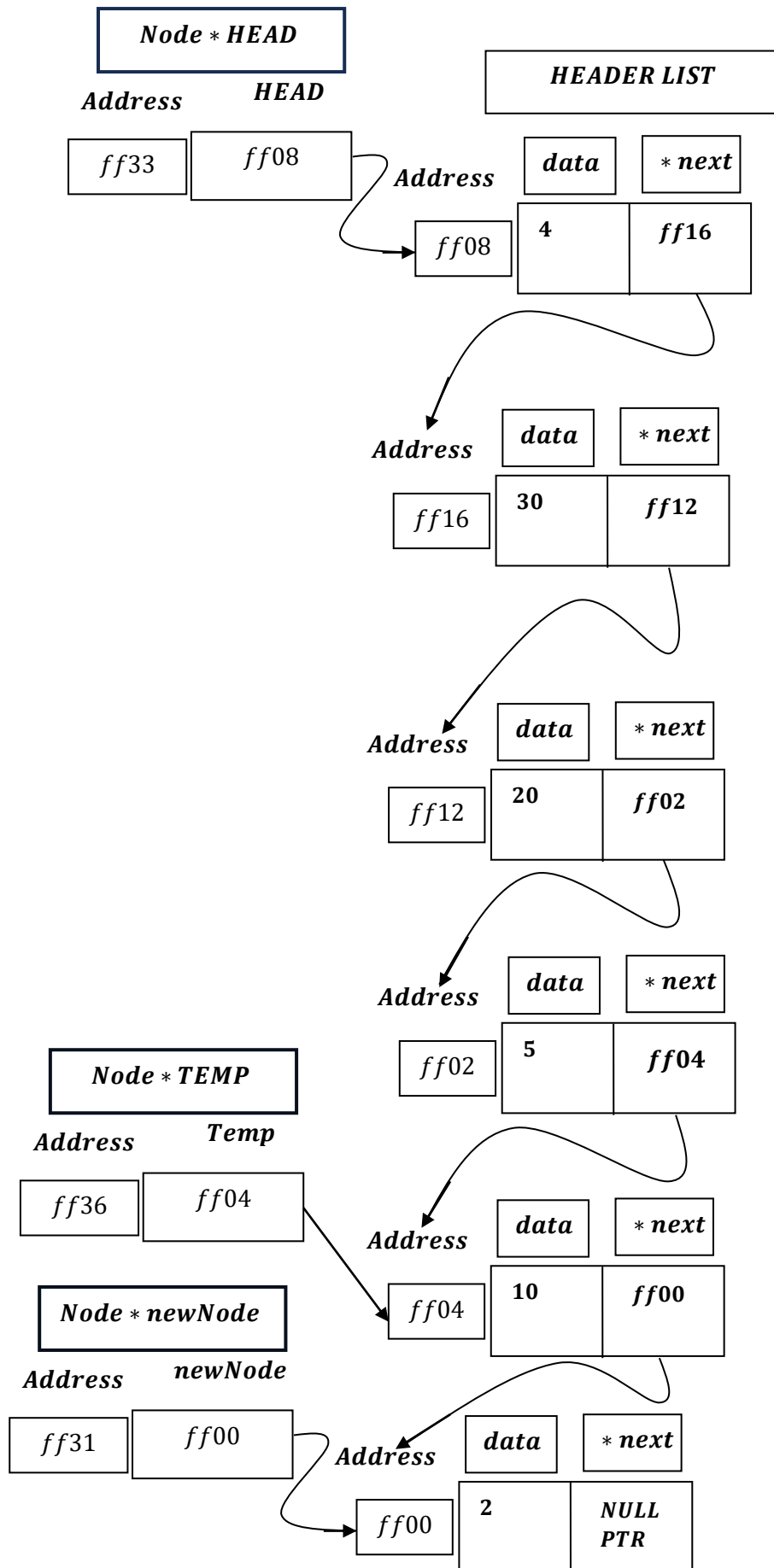
```

newNode → next = temp → next = NULLPTR



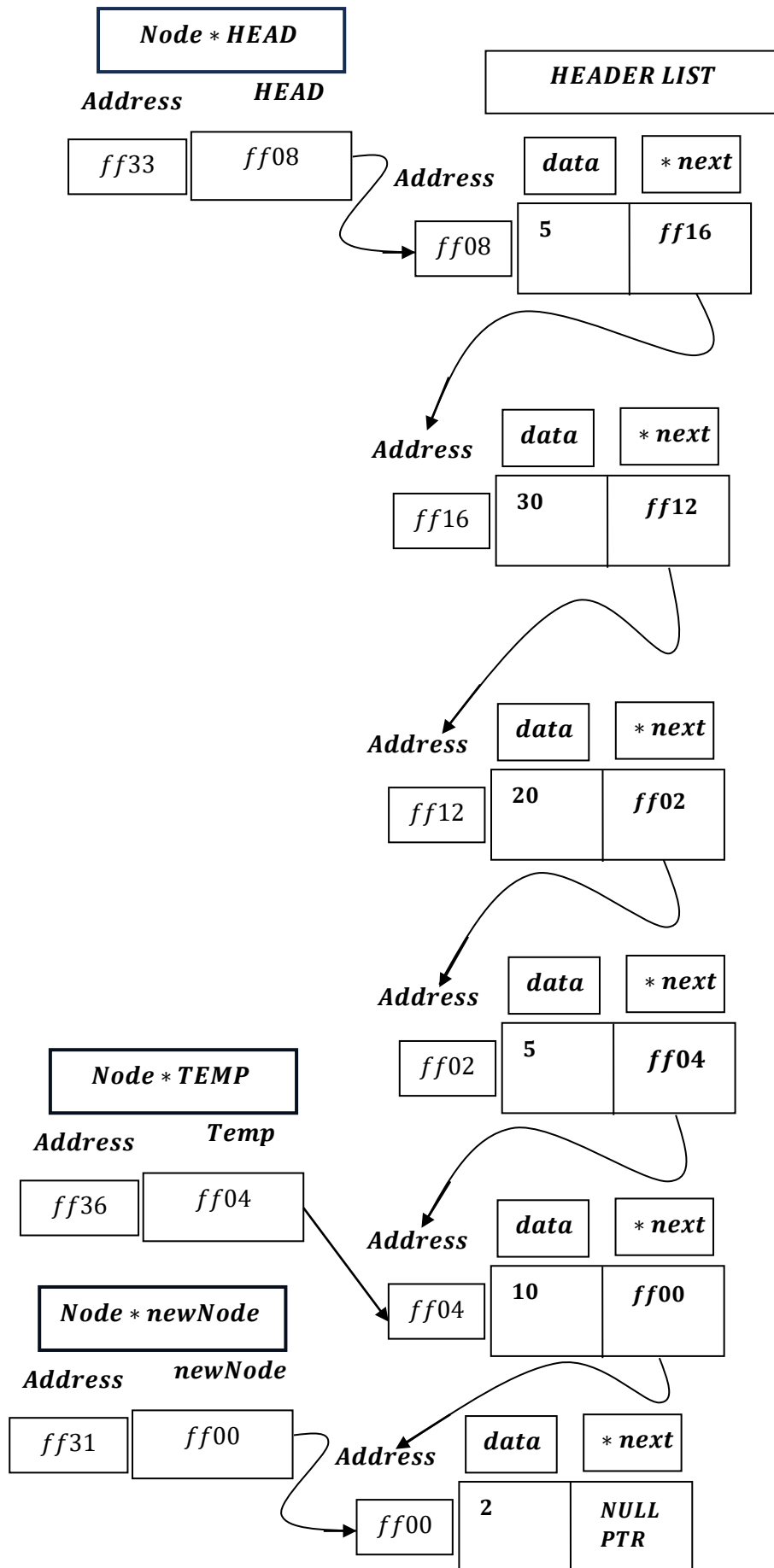
```
temp->next = newNode;
```

TEMP → NEXT = NEWNODE = ff00.

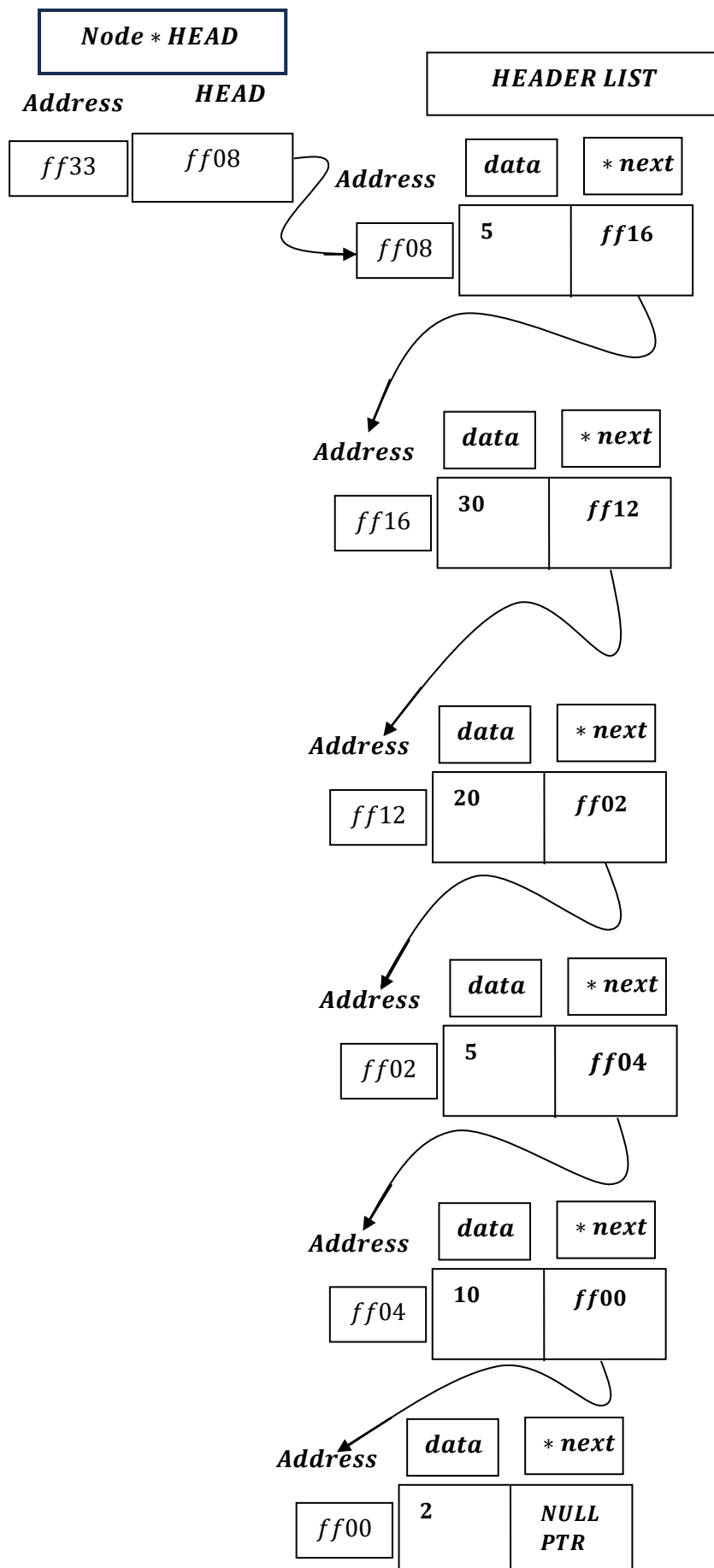


```
head->data++;
```

***Head* → *Data* = *Head* → *Data* + 1 = 4 + 1 = 5.**



And, we get our final list : –



When `Element is not found` ,in:

```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

Then Temp gets assigned to NULLPTR:

```
if (temp == nullptr)
{
    cout << "Element " << element << " not found in the
list." << endl;
    return;
}
```

Therefore, it outputs :

Element [User Input Element] not found in the list.

7. Header Linked List
–Delete From Beginning

Program:

```
void deleteFromBeginning()
{
    if (head->next == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head->next; // The first actual node

    // Update header to point to the second node
    head->next = temp->next;

    cout << "Deleted " << temp->data << " from the
beginning." << endl;

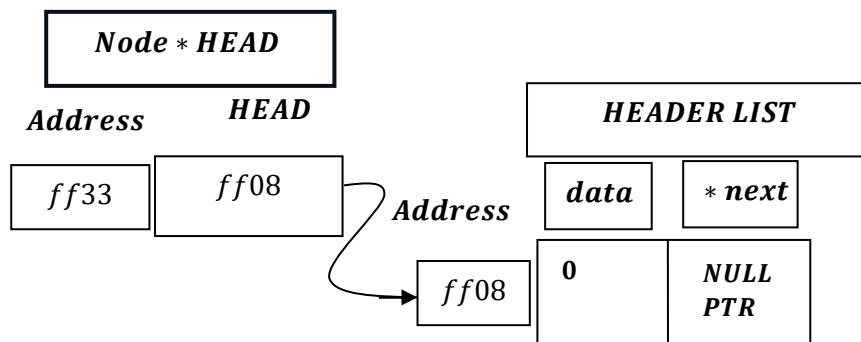
    free(temp);
    temp = nullptr;

    // Decrement internal count
    head->data--;
}
...
    case 5:
        deleteFromBeginning();
        break;
```

Program Analysis

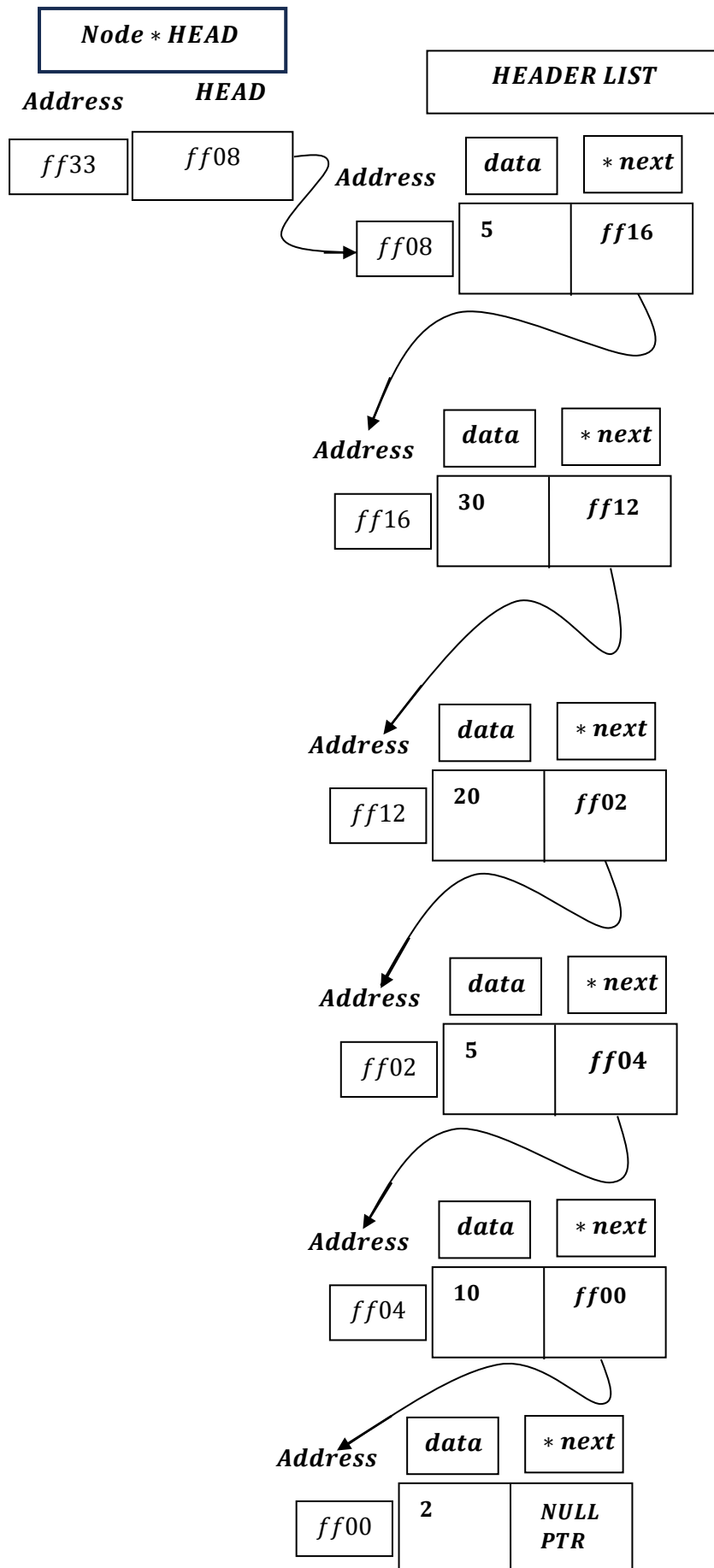
```
if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

i.e.



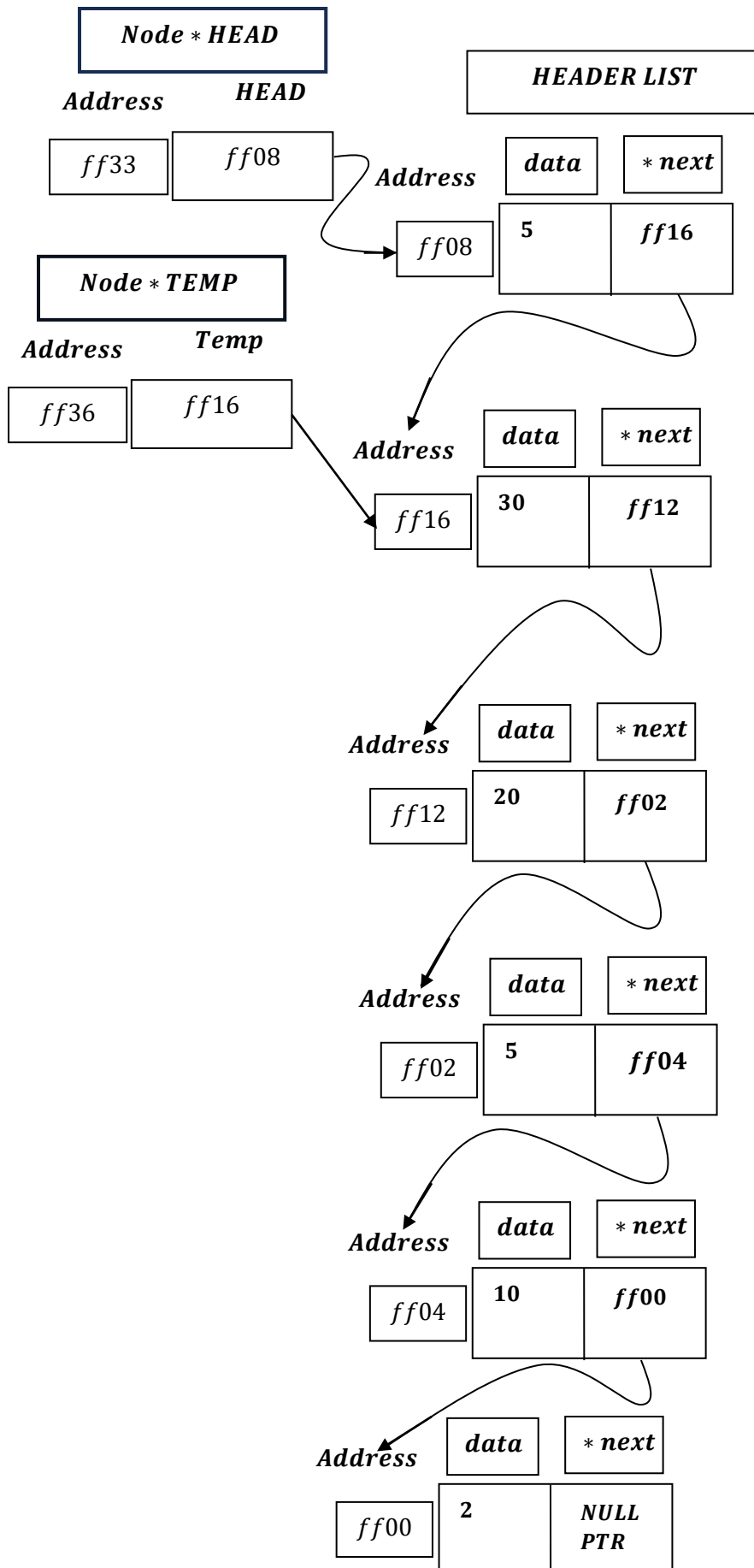
head → *next* = *NULLPTR* , *head* → *Data* = **0** , then list is empty .

If List is not Empty

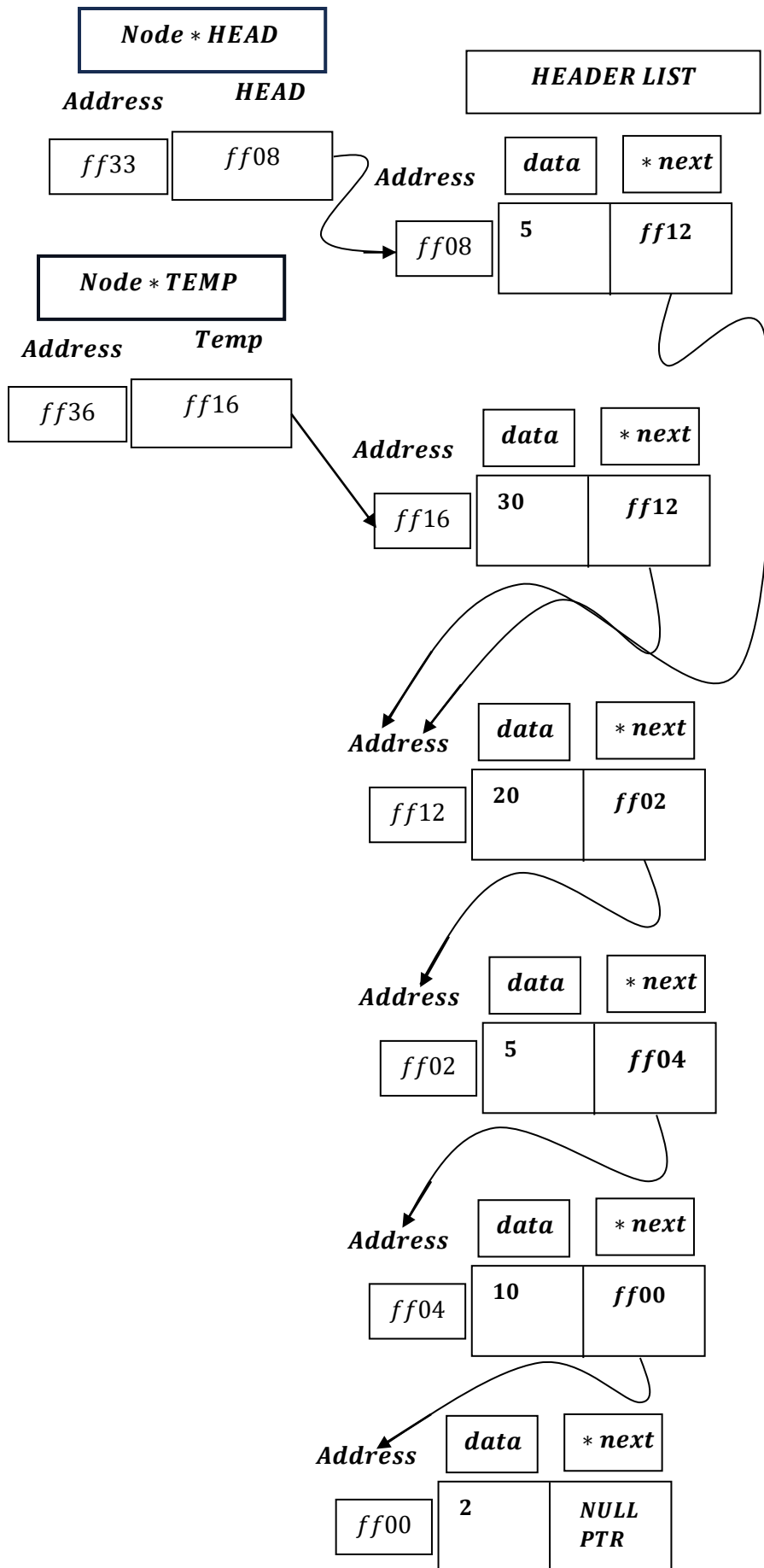


```
Node *temp = head->next; // The first actual node
```

Temp = Head → Next = ff16.



`head->next = temp->next;`
Head → *Next* = *Temp* → *Next* = *ff12*.

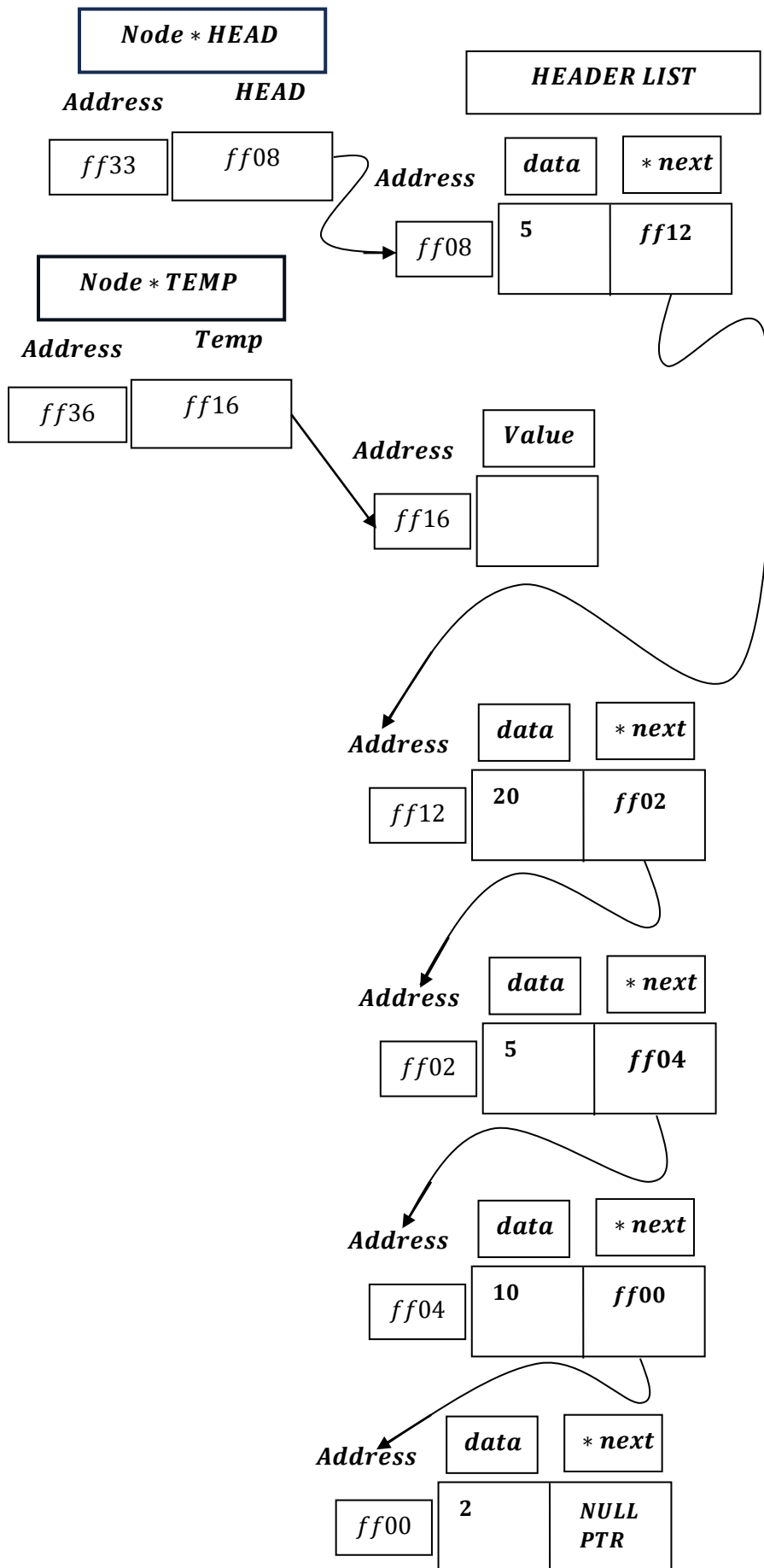


```
cout << "Deleted " << temp->data << " from the  
beginning." << endl;
```

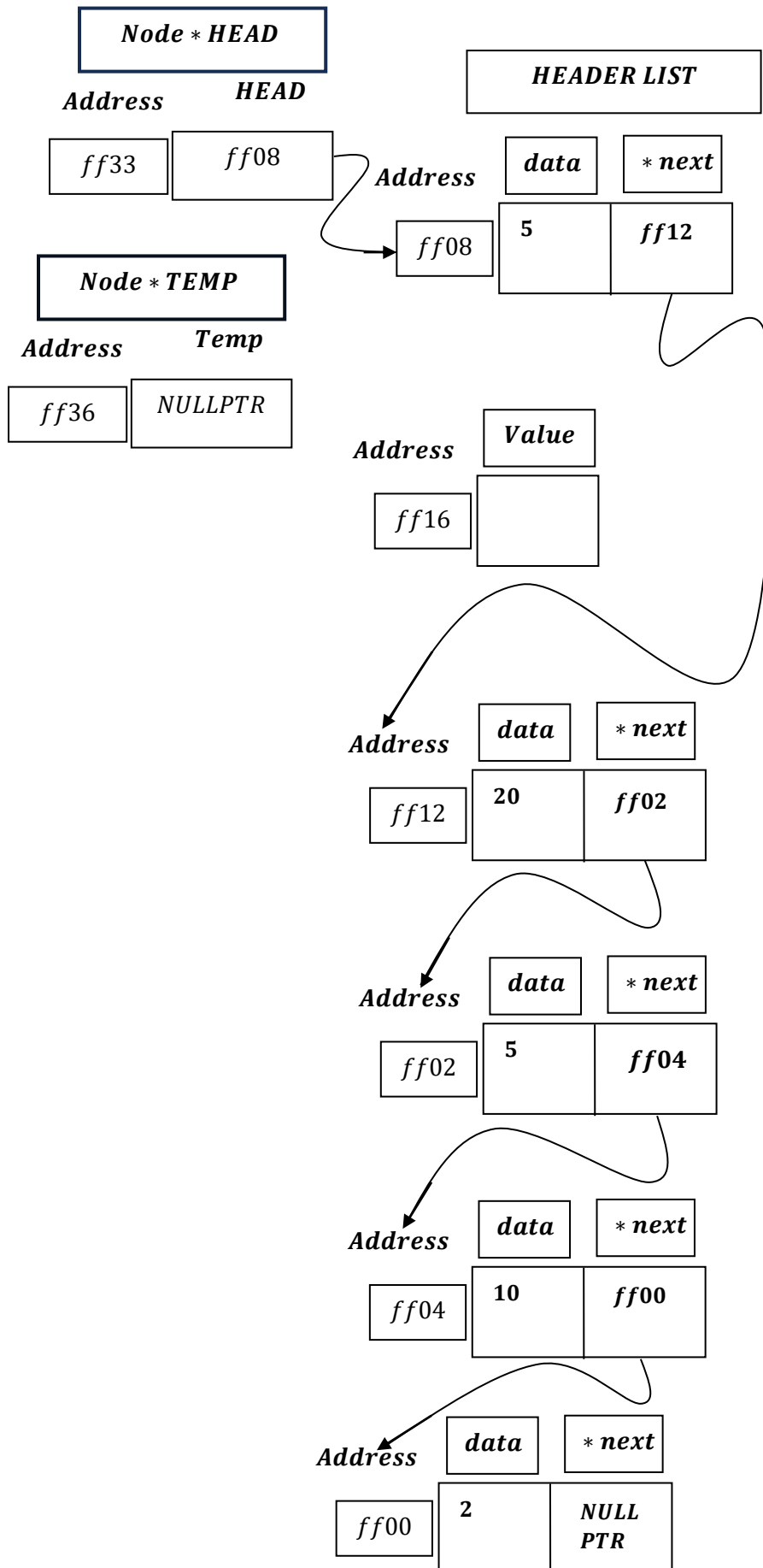
Prints: Deleted ``Temp → Data: 30`` from the beginning.

```
free(temp);
```

It will free the memory that TEMP pointer that it point to .

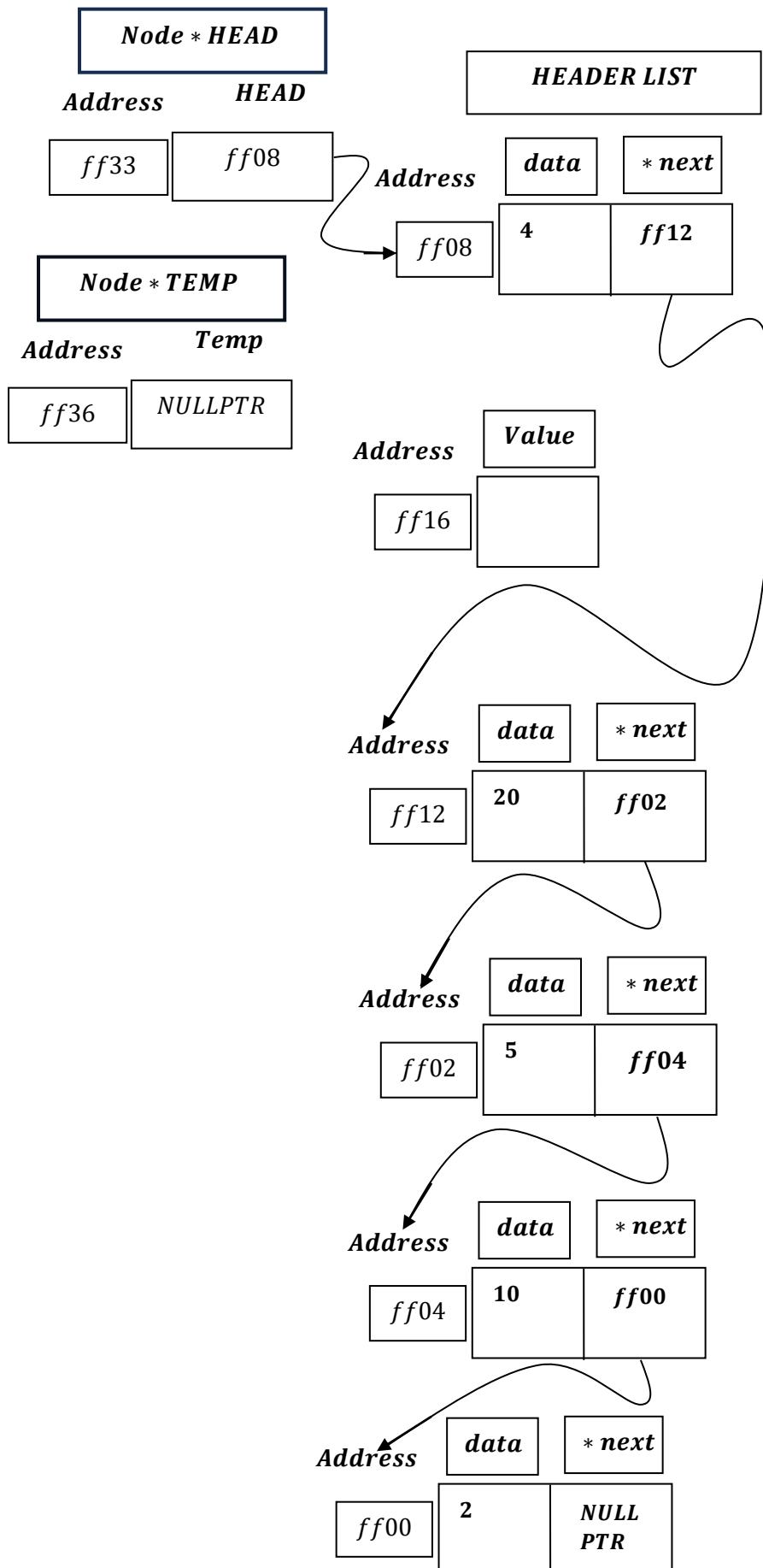


```
temp = nullptr;
```

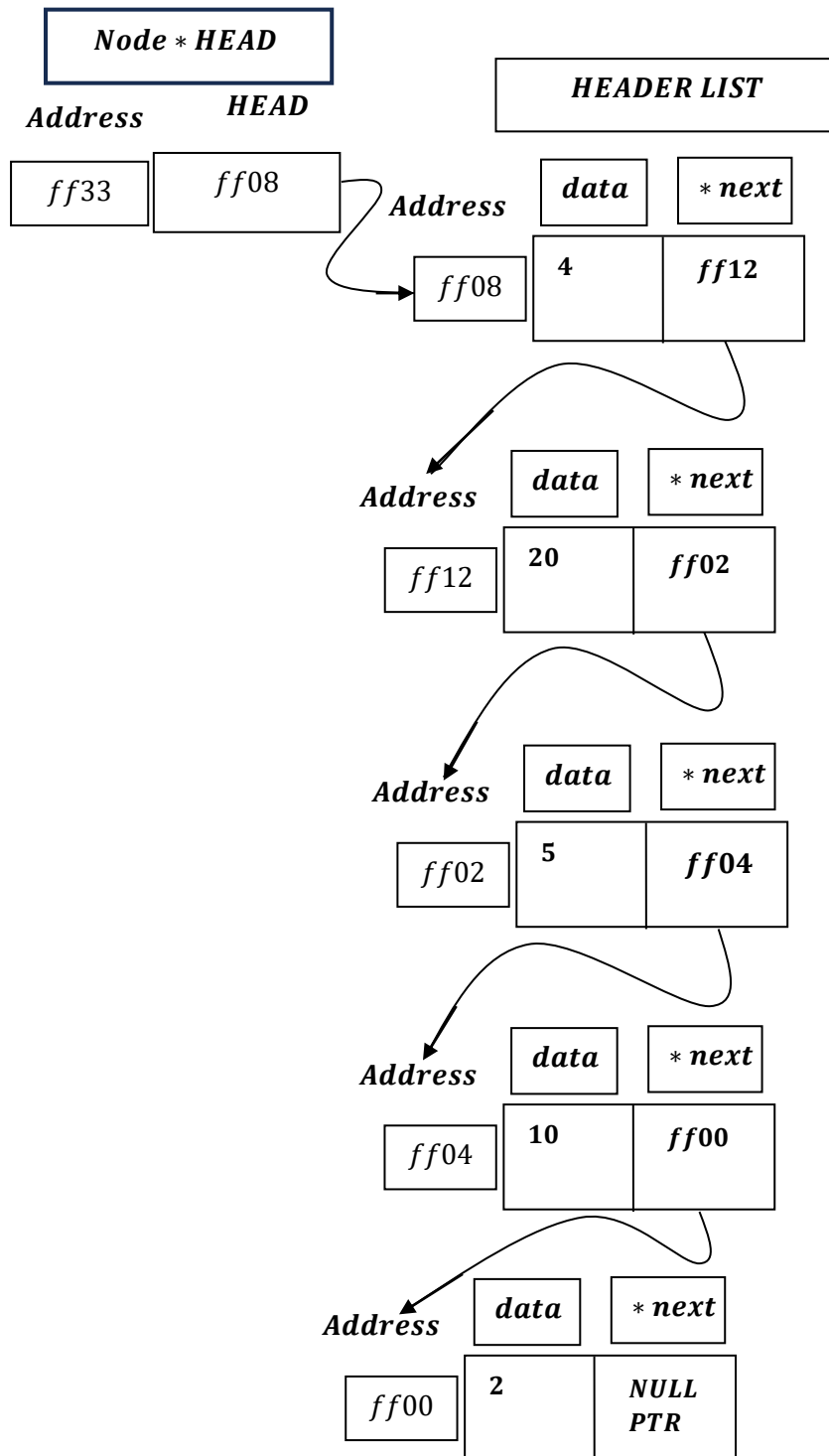


`head->data--;`

$Head \rightarrow Data -- \Rightarrow Head \rightarrow Data = Head \rightarrow Data - 1 = 5 - 1 = 4.$



And we get our list:



8. Header Linked List

–Delete At End

Program:

```
void deleteFromEnd()
{
    if (head->next == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head;

    // Traverse to the node BEFORE the last node
    while (temp->next->next != nullptr)
    {
        temp = temp->next;
    }

    // temp->next is the node to be deleted
    Node *nodeToDelete = temp->next;

    // Unlink
    temp->next = nullptr;

    cout << "Deleted " << nodeToDelete->data << " from the
end." << endl;

    free(nodeToDelete);
    nodeToDelete = nullptr;

    // Decrement internal count
    head->data--;
}
...•
```

```

case 6:
    deleteFromEnd();
    break;

```

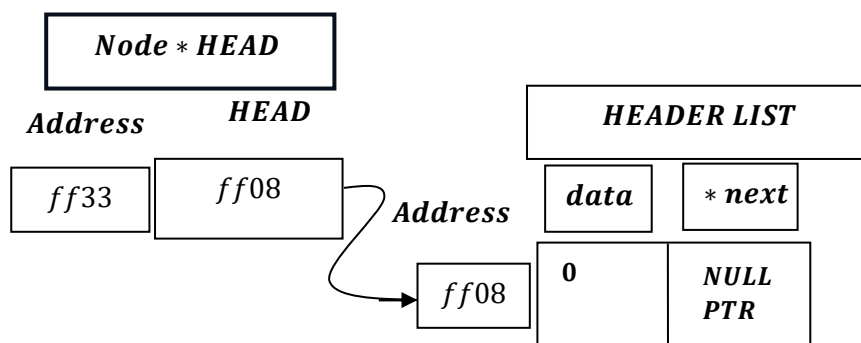
Program Analysis

```

if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}

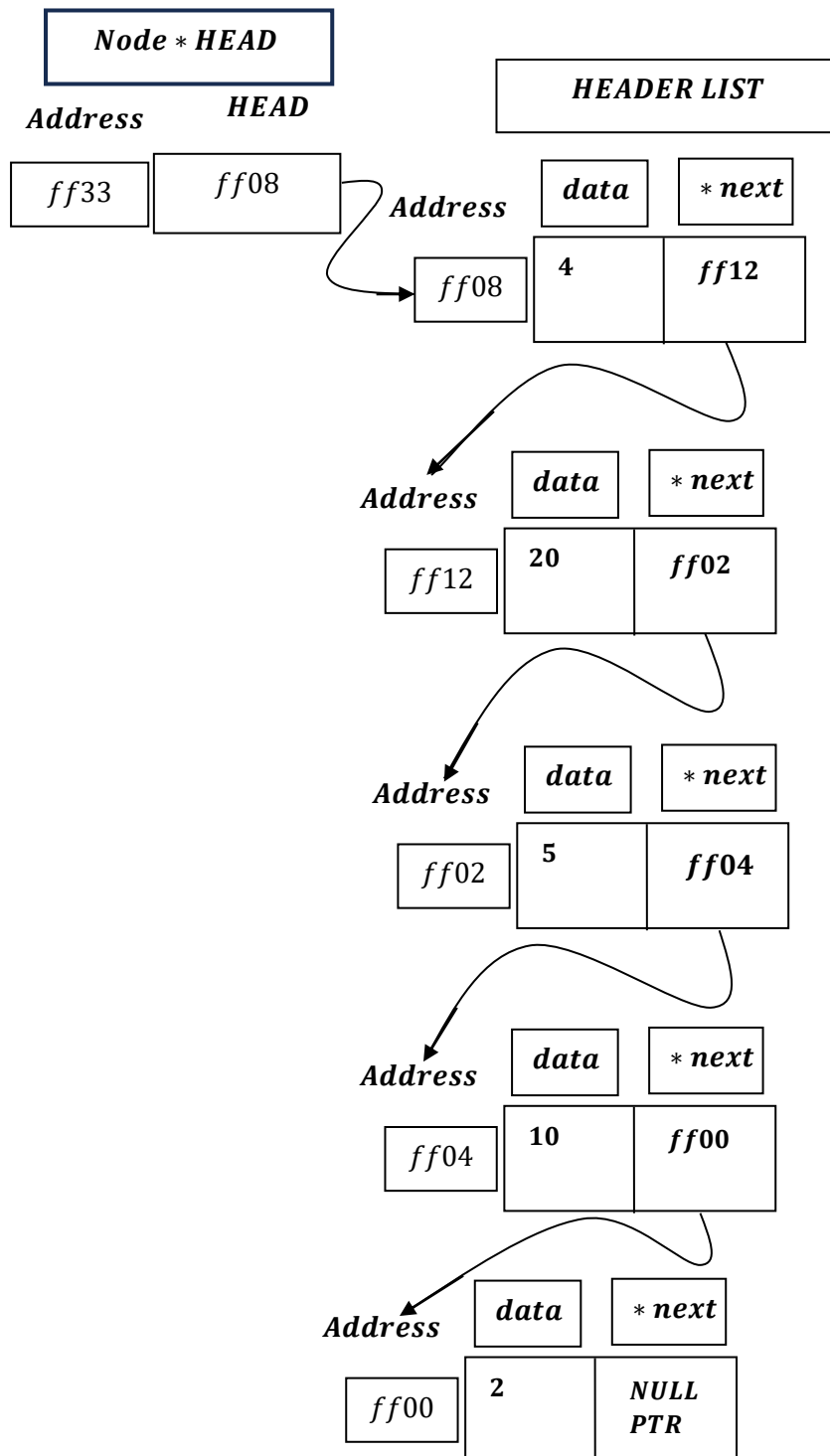
```

i.e.

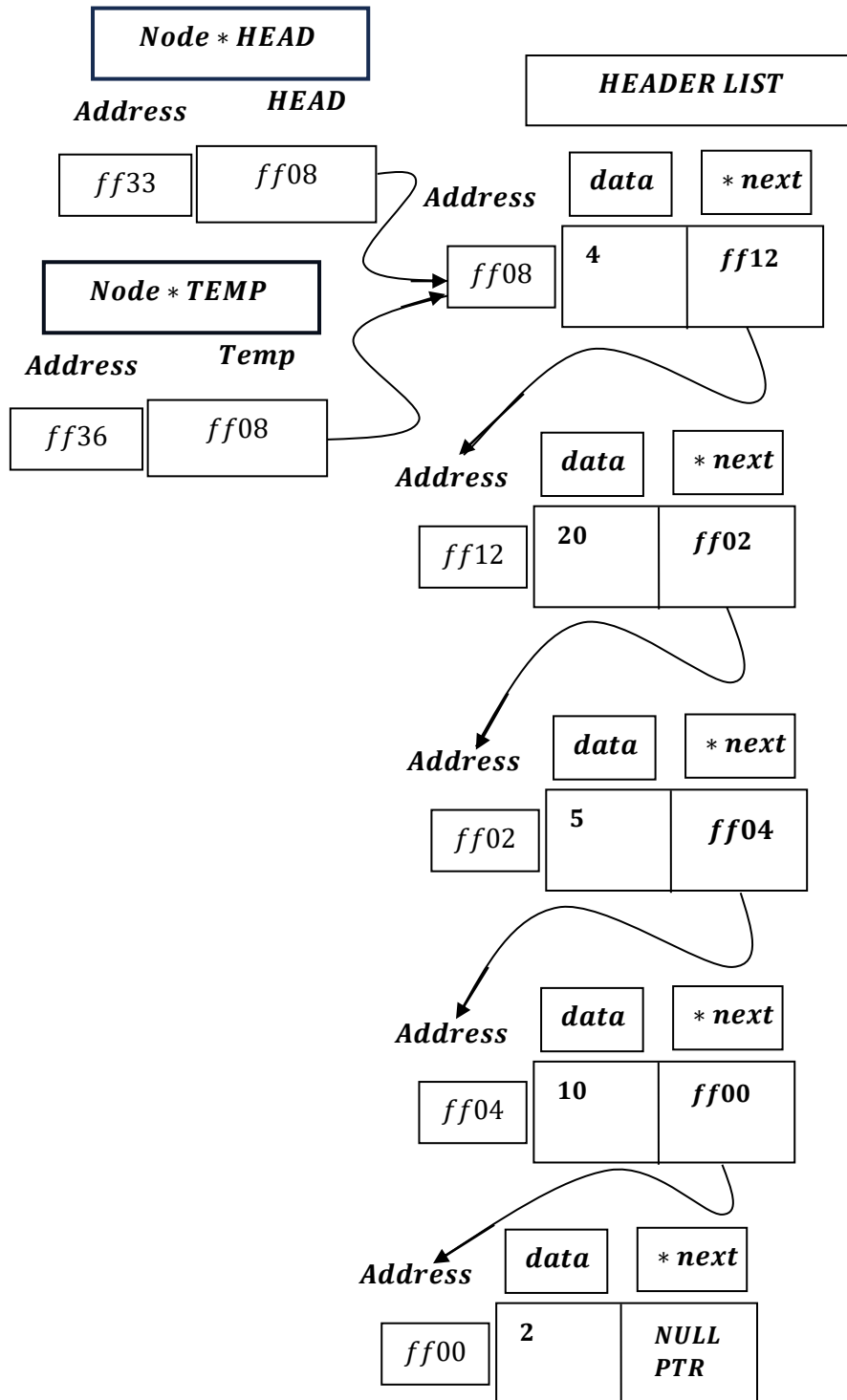


head → next = NULLPTR , head → Data = 0 , then list is empty .

If List is not Empty



```
Node *temp = head;  
Temp = Head = ff08.
```



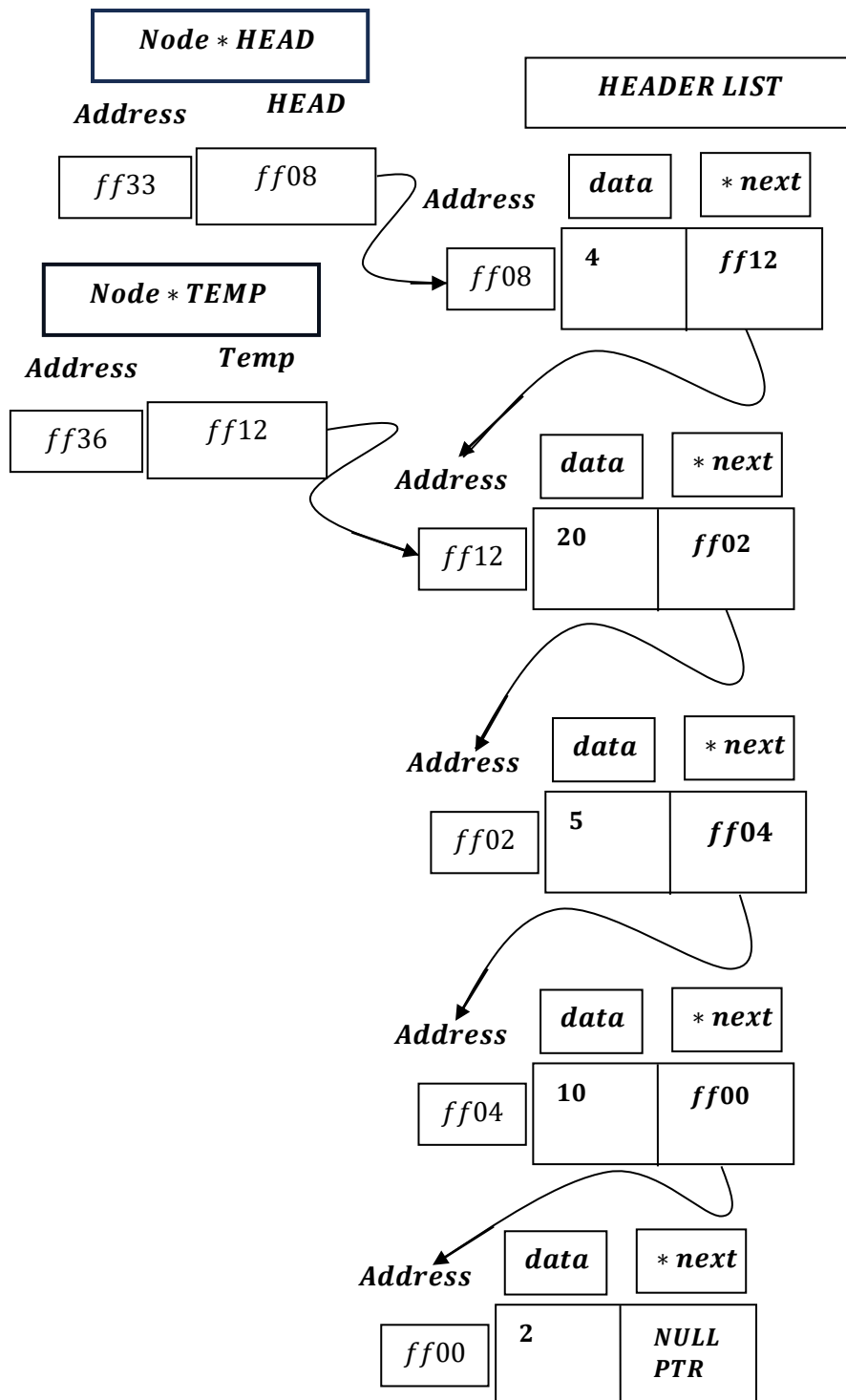
```

while (temp->next->next != nullptr)
{
    temp = temp->next;
}

```

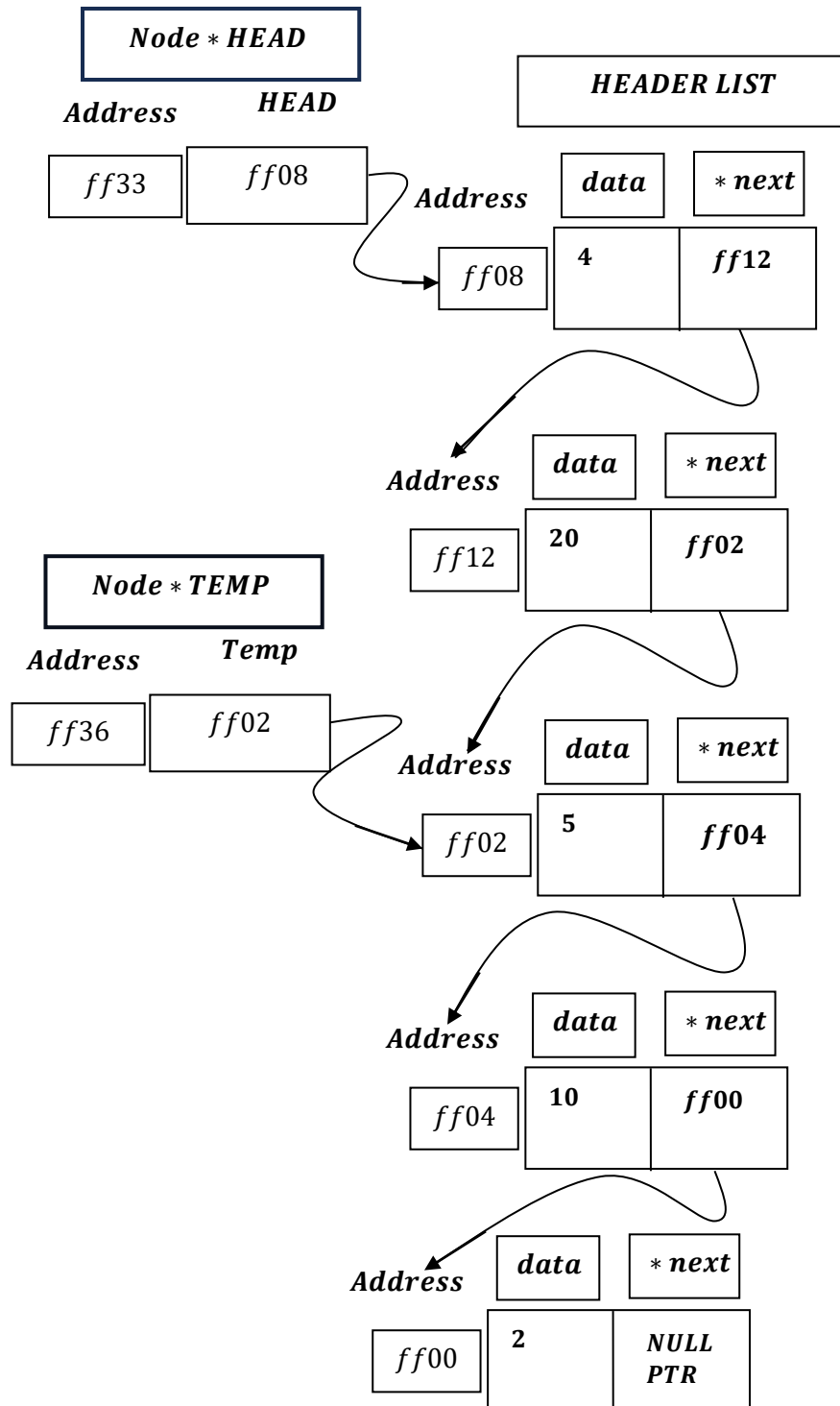
TEMP → NEXT → NEXT = ff02 ≠ NULLPTR is TRUE:

TEMP = TEMP → NEXT = ff12



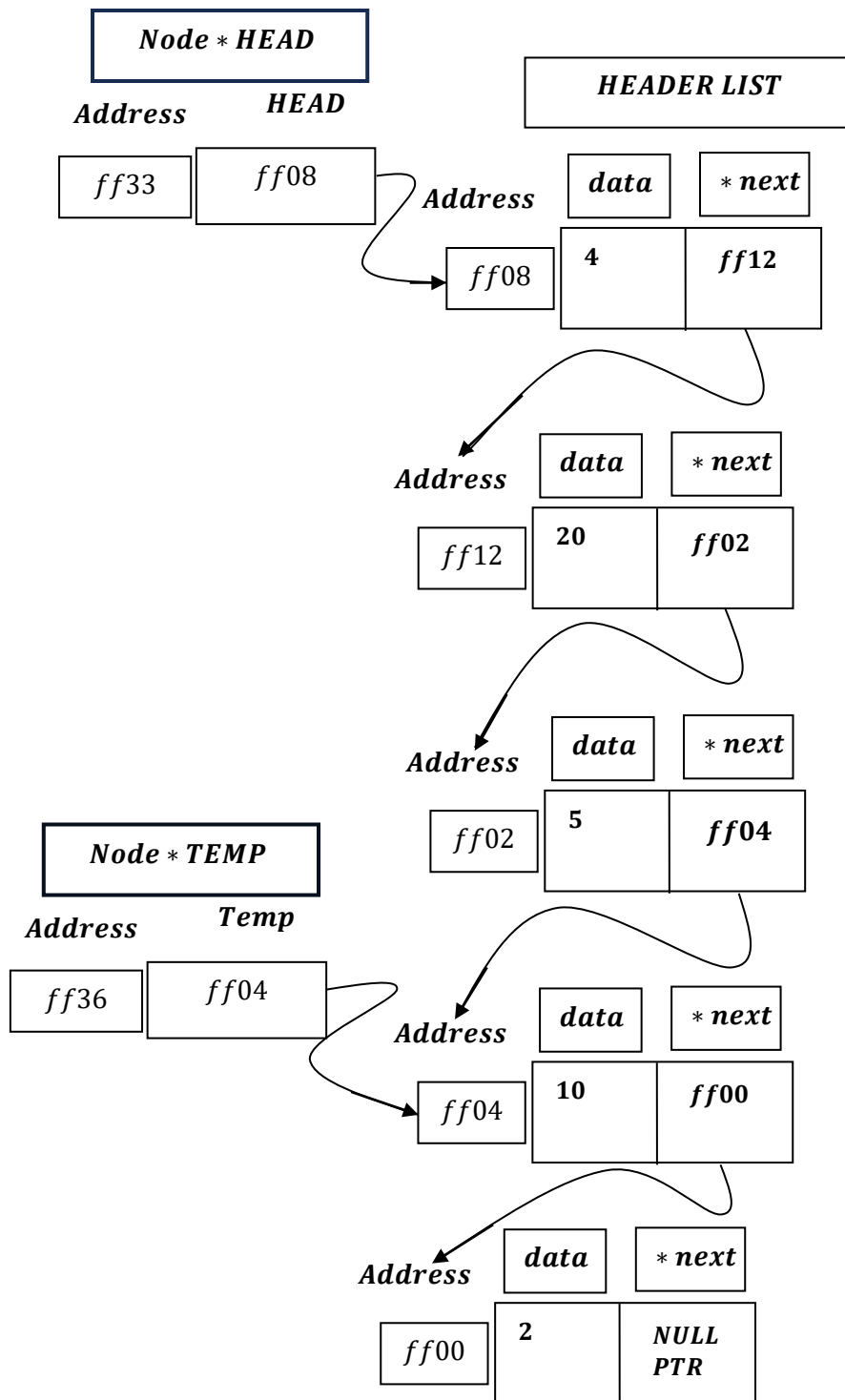
$TEMP \rightarrow NEXT \rightarrow NEXT = ff04 \neq NULLPTR$ is TRUE:

$TEMP = TEMP \rightarrow NEXT = ff02$



$TEMP \rightarrow NEXT \rightarrow NEXT = ff00 \neq NULLPTR$ is TRUE:

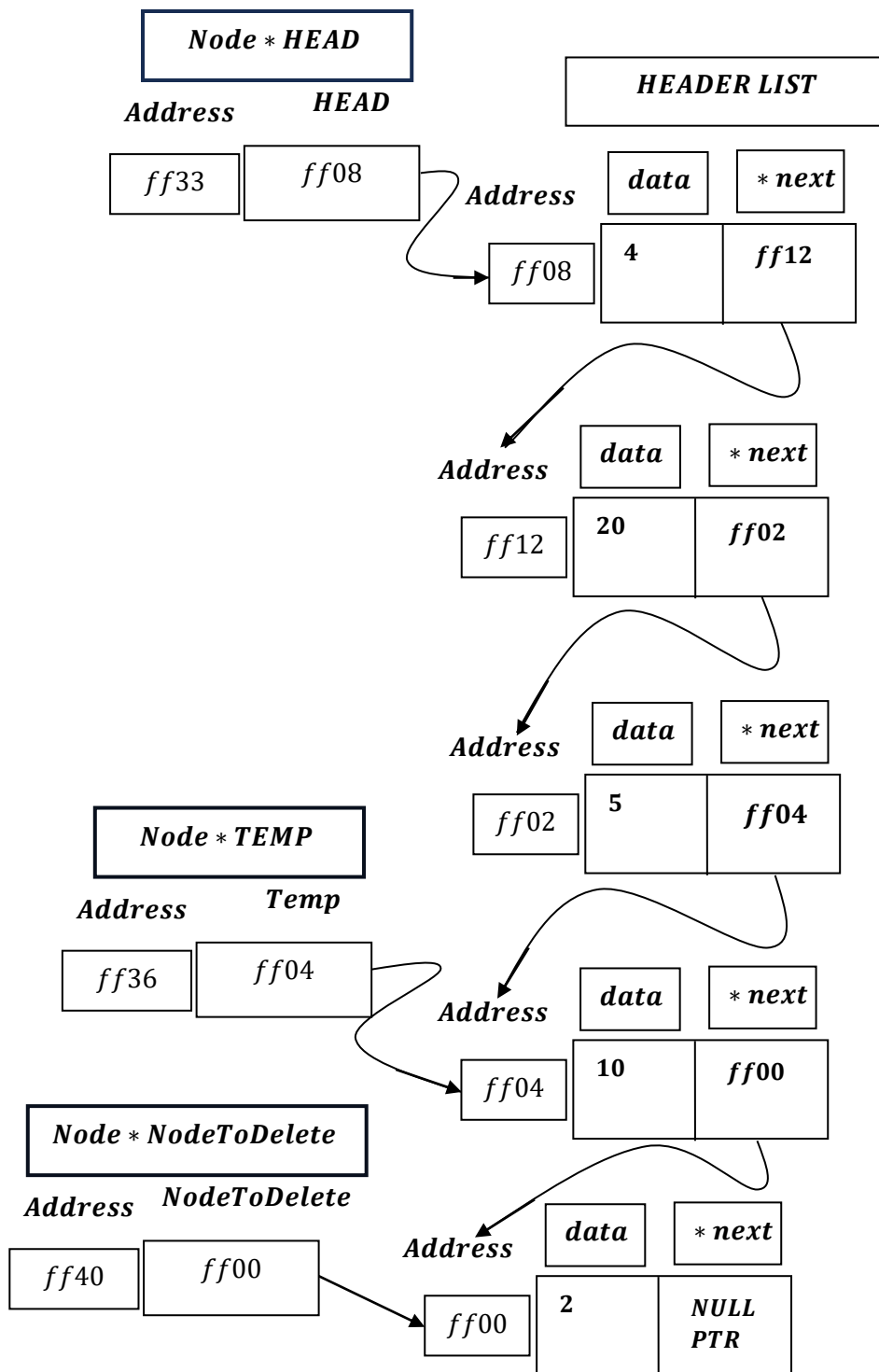
$TEMP = TEMP \rightarrow NEXT = ff04$



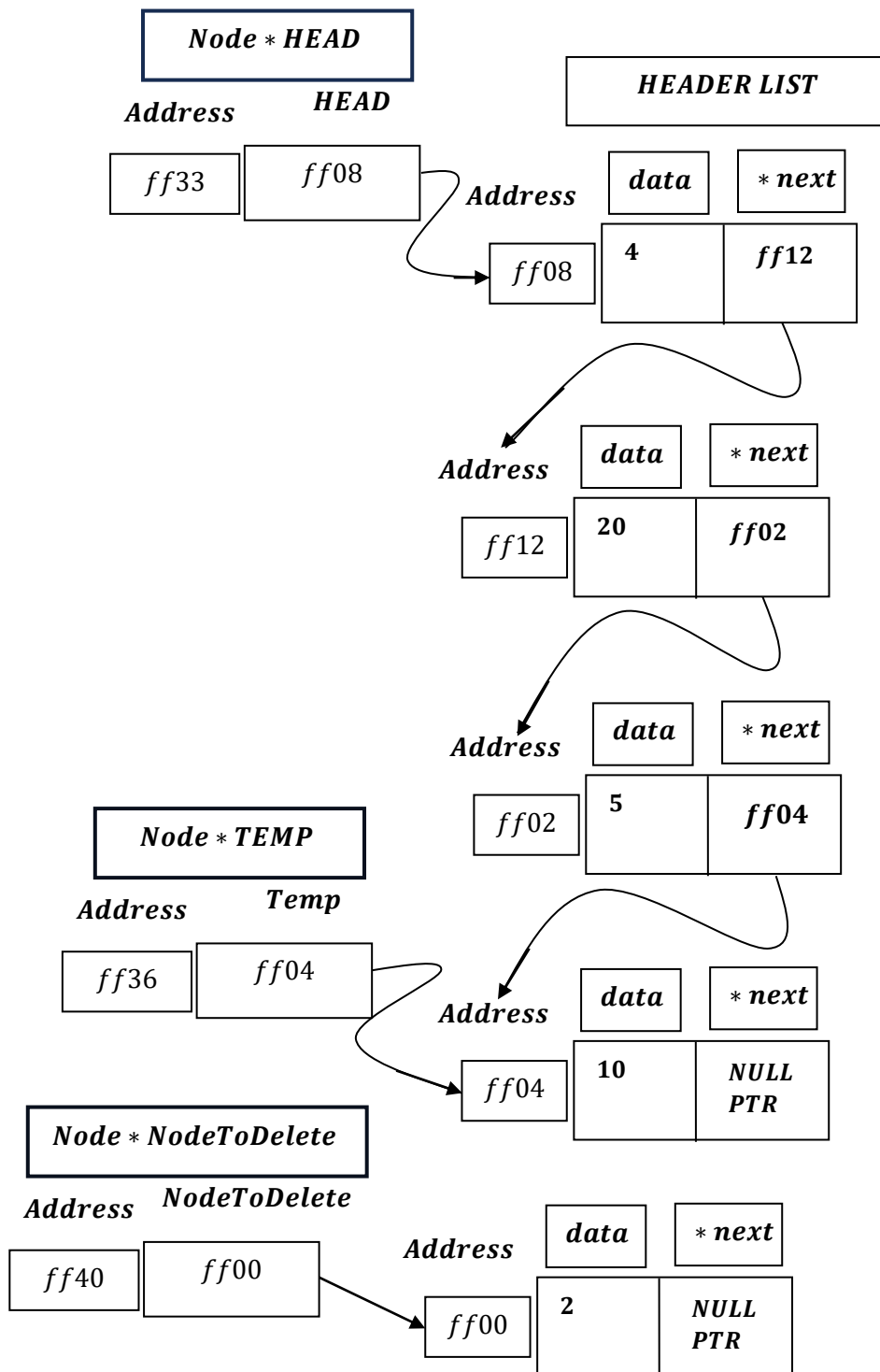
$TEMP \rightarrow NEXT \rightarrow NEXT = NULLPTR \neq NULLPTR$ is FALSE, hence, exit from loop.

```
Node *nodeToDelete = temp->next;
```

$NodeToDelete = TEMP \rightarrow NEXT = ff00$.



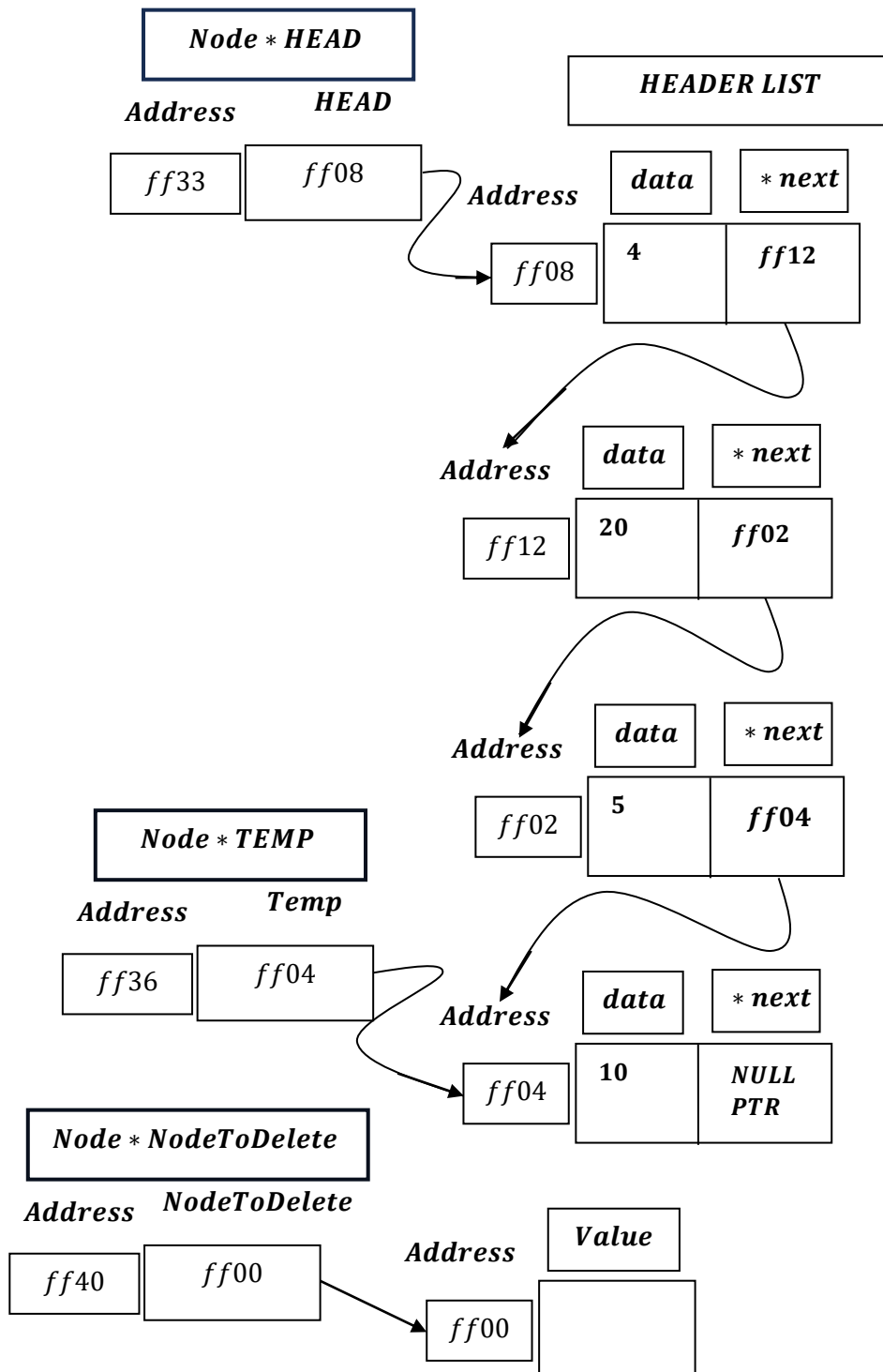
```
temp->next = nullptr;  
TEMP → NEXT = NULLPTR.
```



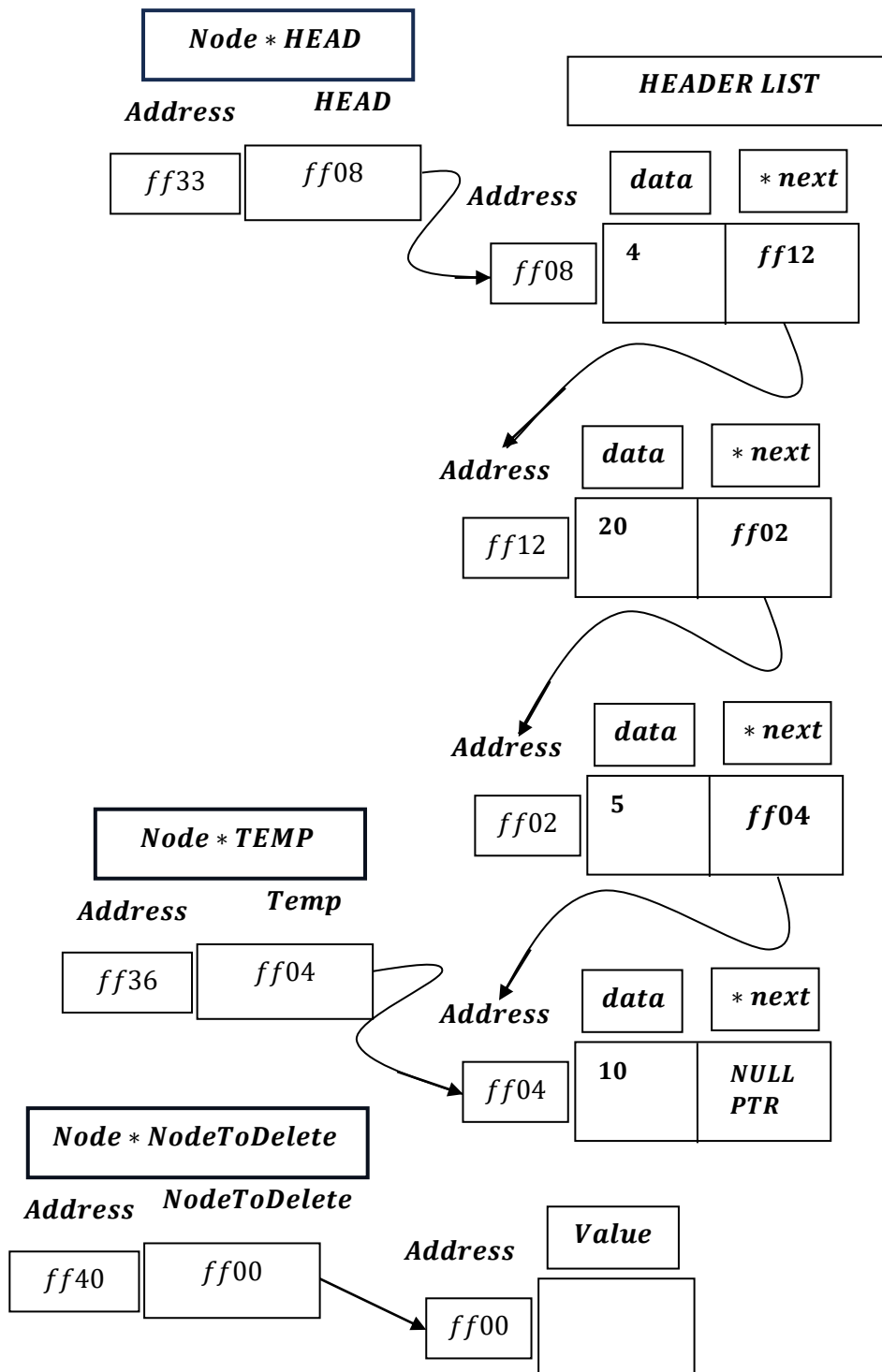
```
cout << "Deleted " << nodeToDelete->data << " from the end." << endl;
```

Output $\Rightarrow$ Print: Deleted `2` from the end.

```
free(nodeToDelete);
```

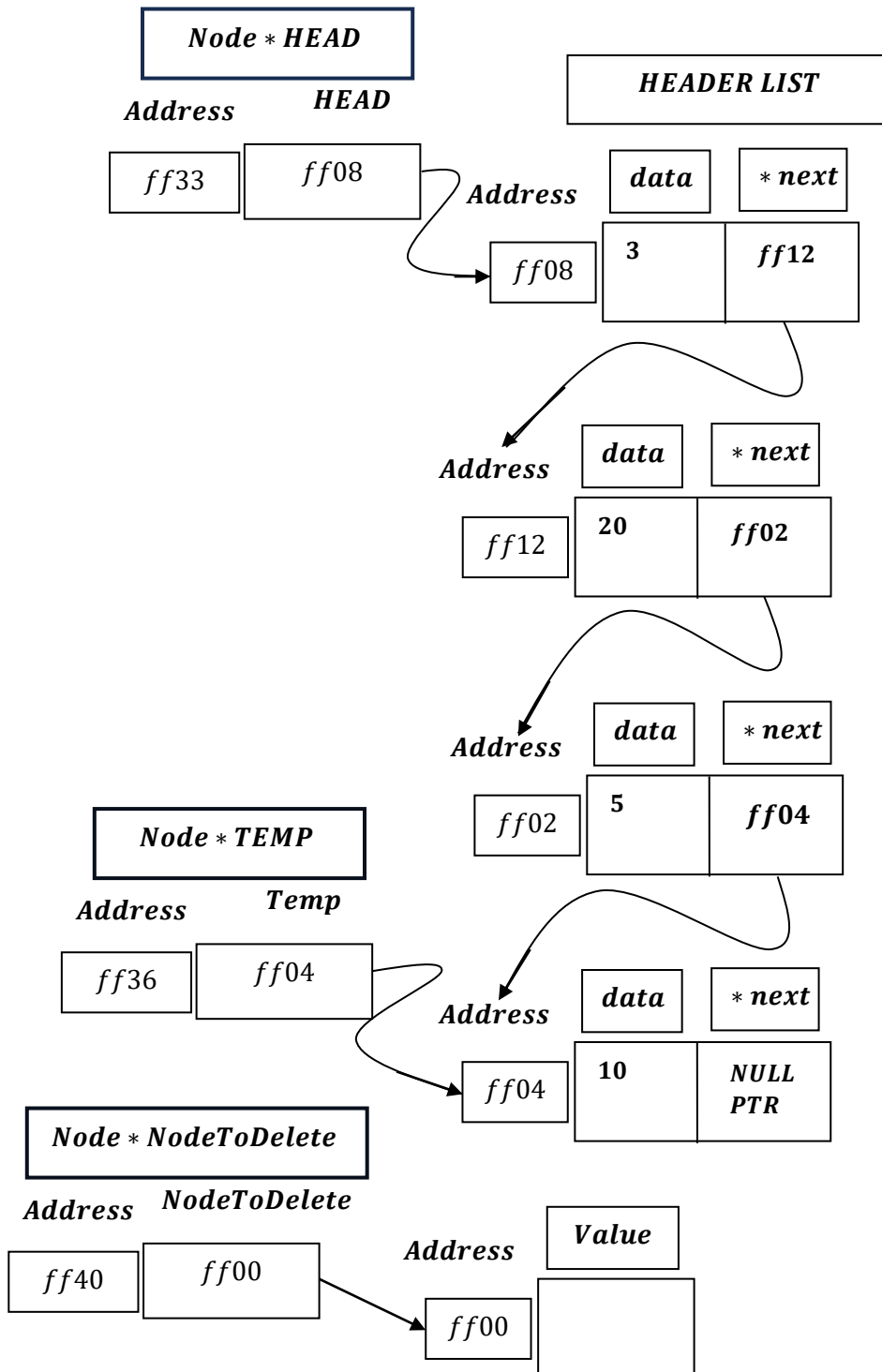


```
nodeToDelete = nullptr;
```

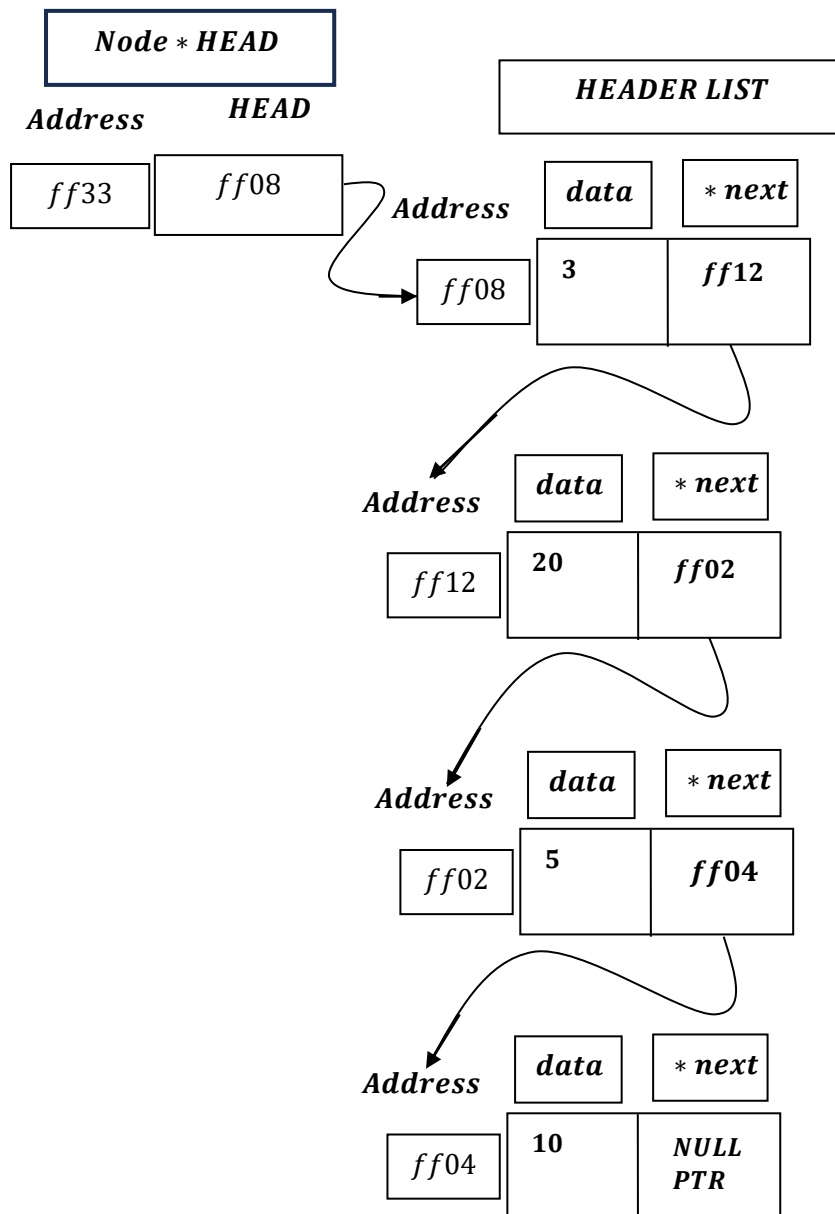


`head->data--;`

$Head \rightarrow Data -- \Rightarrow Head \rightarrow Data = Head \rightarrow Data - 1 = 4 - 1 = 3.$



And we get our list:



9. Header Linked List

–Delete At Position

Program:

```
void deleteFromPosition(int pos)
{
    if (head->next == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    if (pos < 1 || pos > head->data)
    {
        cout << "Invalid position!" << endl;
        return;
    }

    Node *prev = head;

    // Traverse to the node BEFORE the target position
    for (int i = 1; i < pos; i++)
    {
        prev = prev->next;
    }

    Node *nodeToDelete = prev->next;

    // Unlink the node
    prev->next = nodeToDelete->next;

    cout << "Deleted " << nodeToDelete->data << " from
position " << pos << "." << endl;

    free(nodeToDelete);
    nodeToDelete = nullptr;

    // Decrement internal count
```

```

    head->data--;
}
.....
case 7:
    cout << "Enter position to delete: ";
    cin >> pos;
    deleteFromPosition(pos);
    break;

```

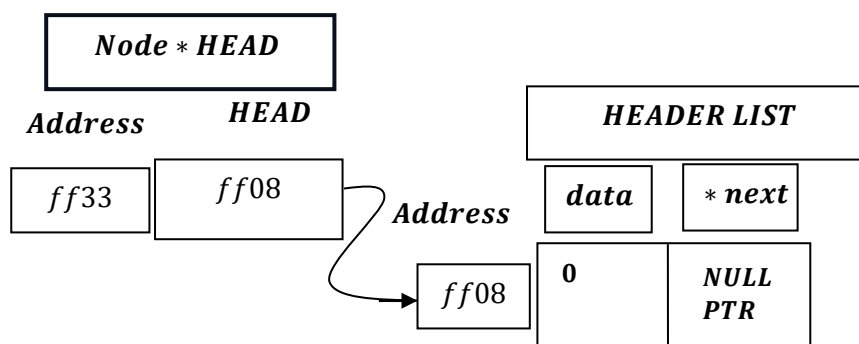
Program Analysis

```

if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}

```

i. e.



head → *next* = *NULLPTR* , *head* → *Data* = *0* , then list is empty .

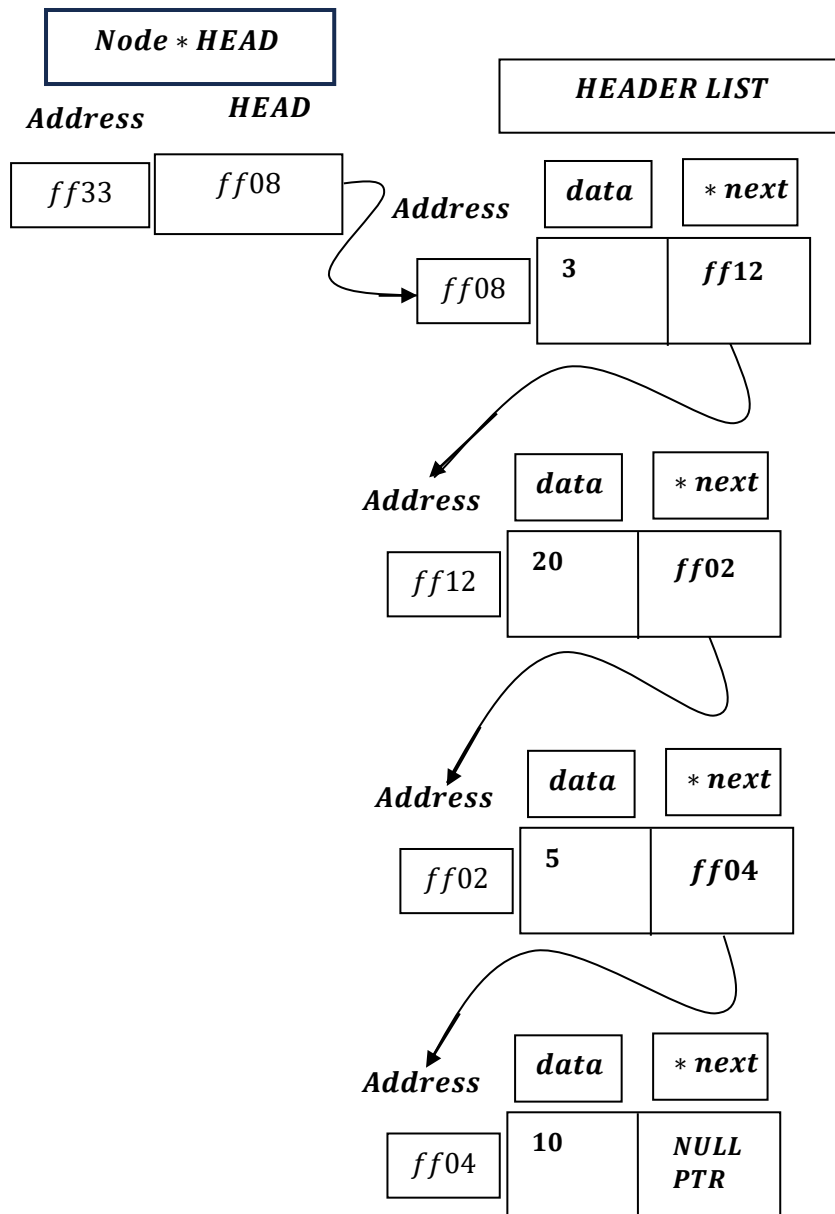
```
if (pos < 1 || pos > head->data)
{
    cout << "Invalid position!" << endl;
    return;
}
```

Now, if list has 4 nodes, Header Node's Data is also 4.

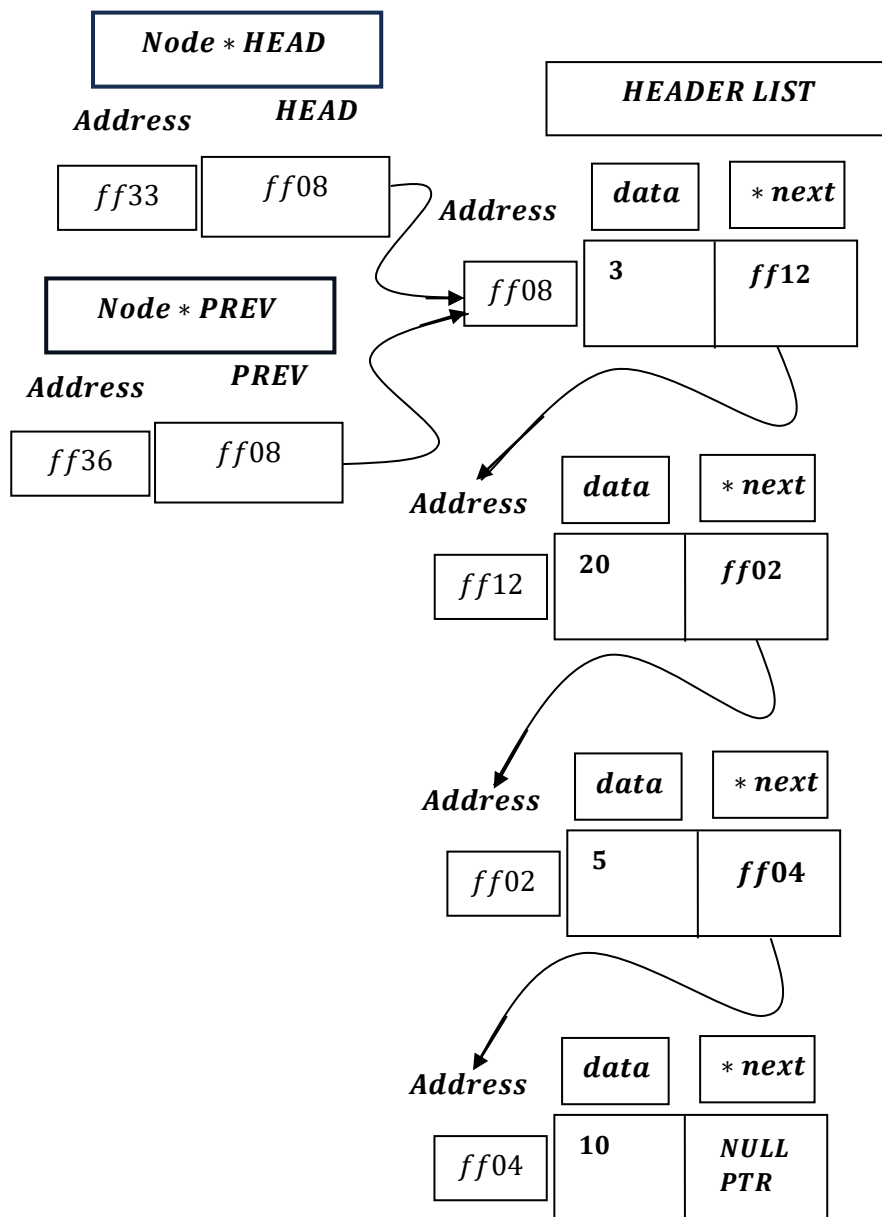
Now we can delete header node's data at position: 1, 2, 3, 4. Greater than that of four (4) will result: Invalid Position.

Therefore, when `INVALID POSITION` : When $POS < 1$ or $Pos > \text{Header Node's Data}$.

If List is not Empty



```
Node *prev = head;  
Node * Prev = head = ff08.
```

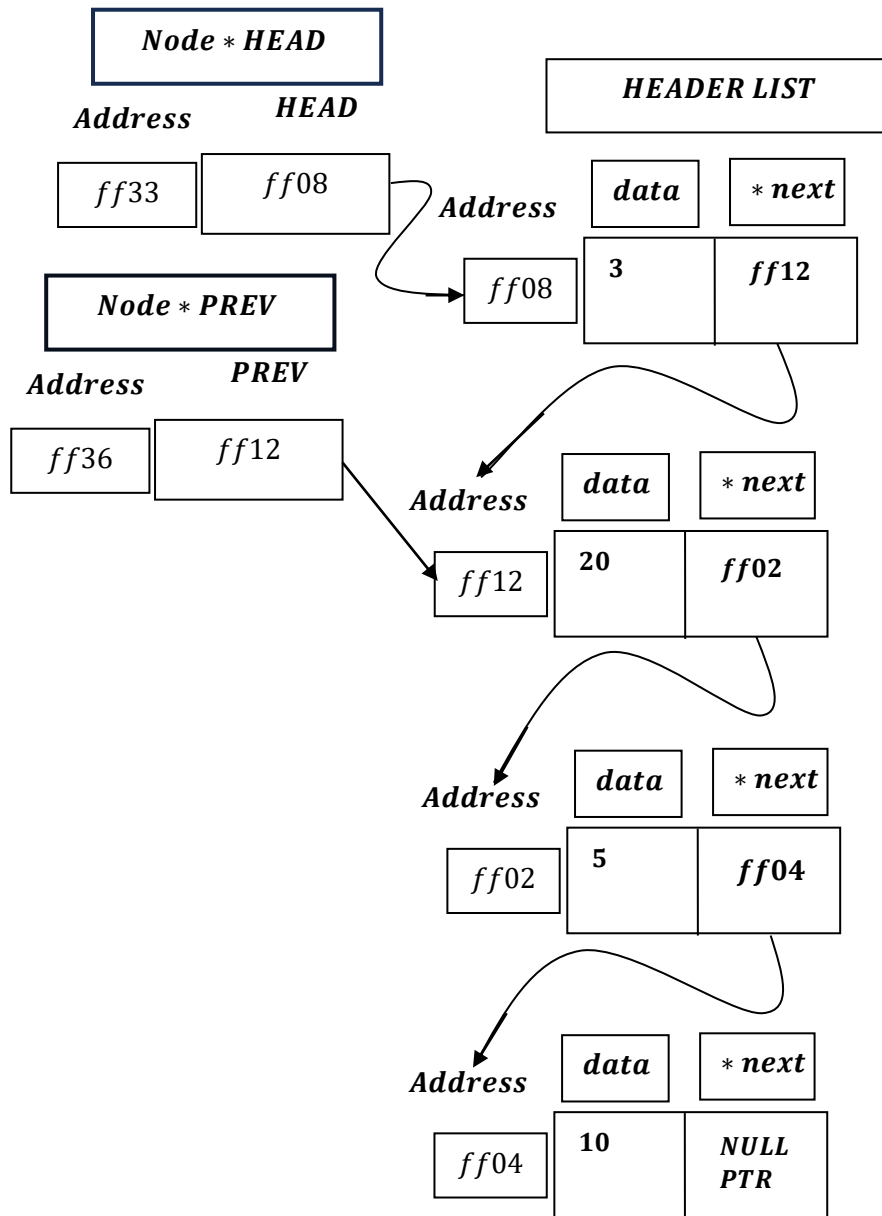


```
for (int i = 1; i < pos; i++)
{
    prev = prev->next;
}
```

Say, $Pos = 2$.

$int\ i = 1 ; i = 1 < Pos: 2\ [True]:$

$PREV = PREV \rightarrow NEXT = ff12;$

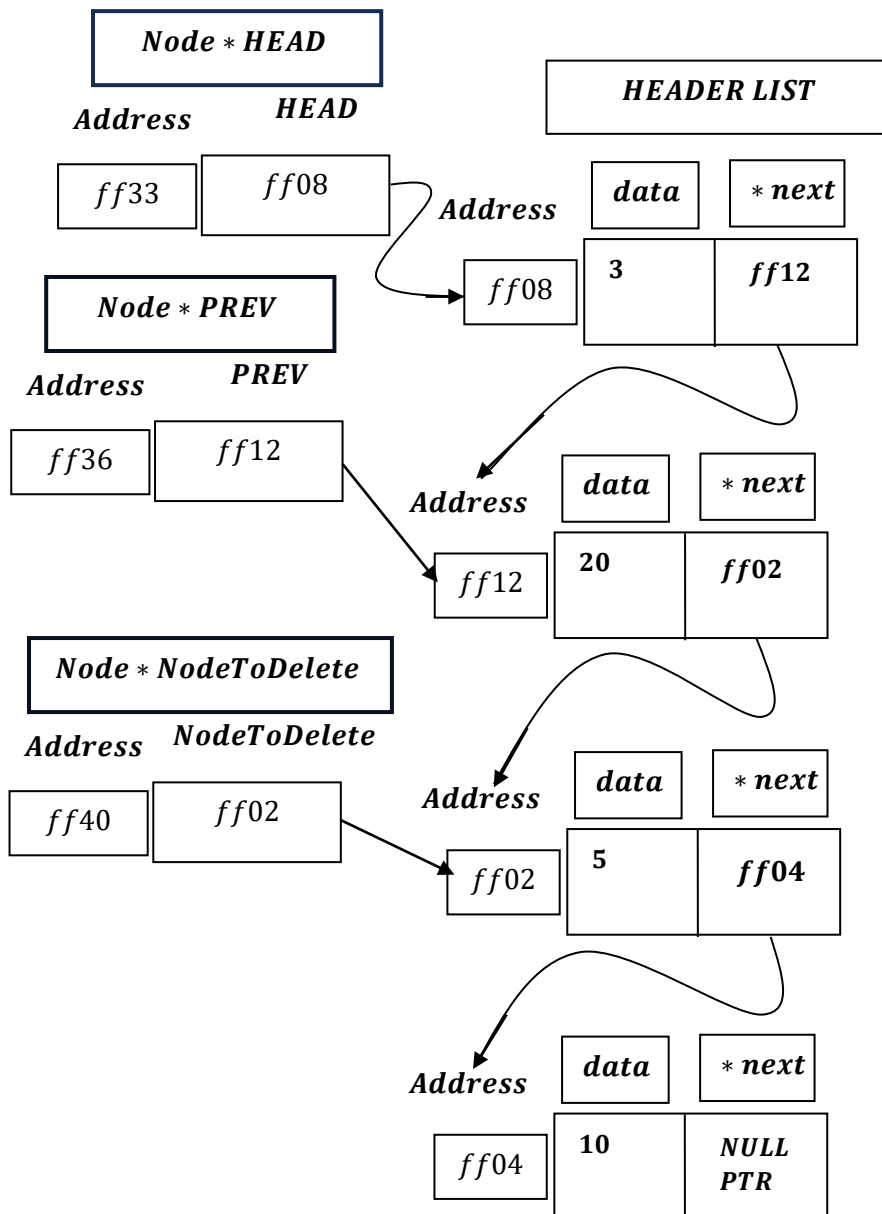


$i++ \Rightarrow i = i + 1 = 1 + 1 = 2.$

$i = 2 < \text{Pos}: 2$ [*False*], hence , exit from loop.

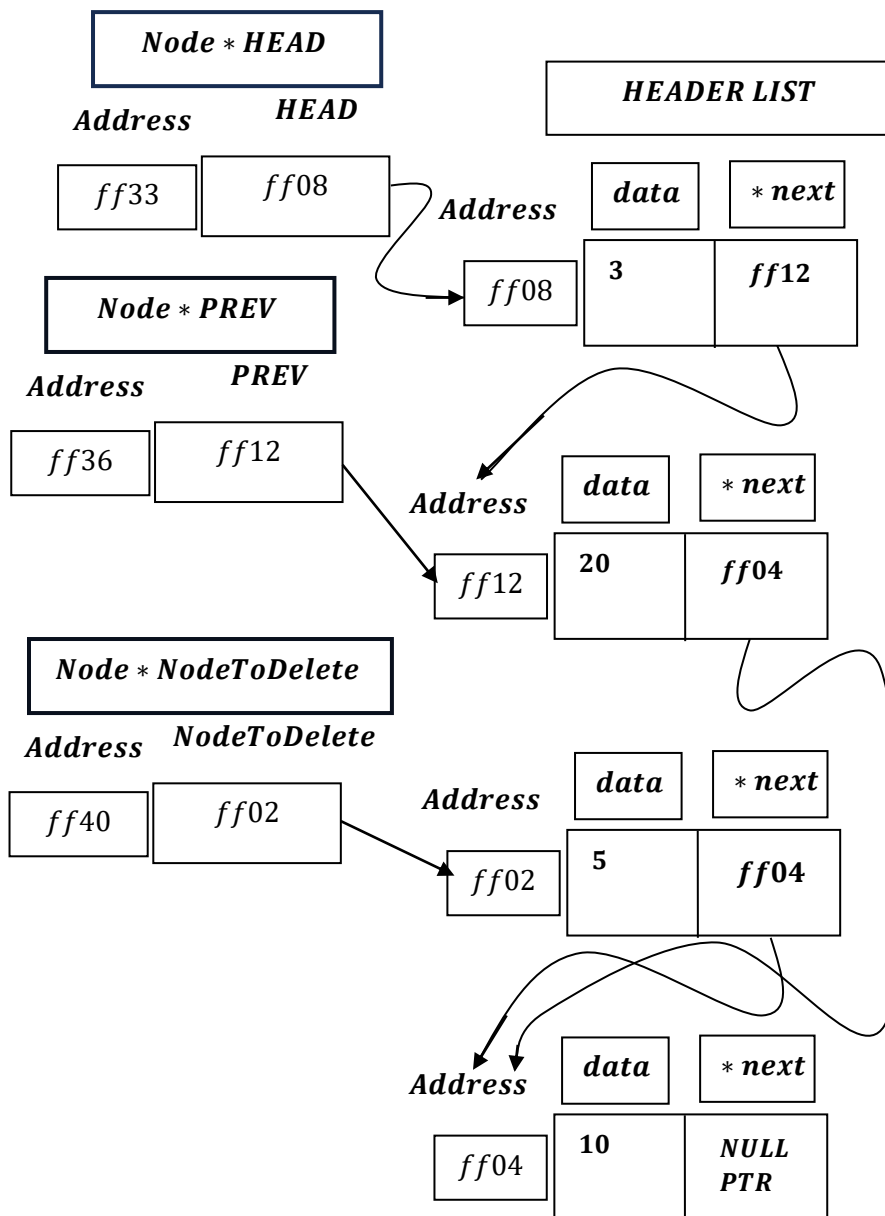
```
Node *nodeToDelete = temp->next;
```

$\text{NodeToDelete} = \text{PREV} \rightarrow \text{NEXT} = \text{ff02}$.



```
prev->next = nodeToDelete->next;
```

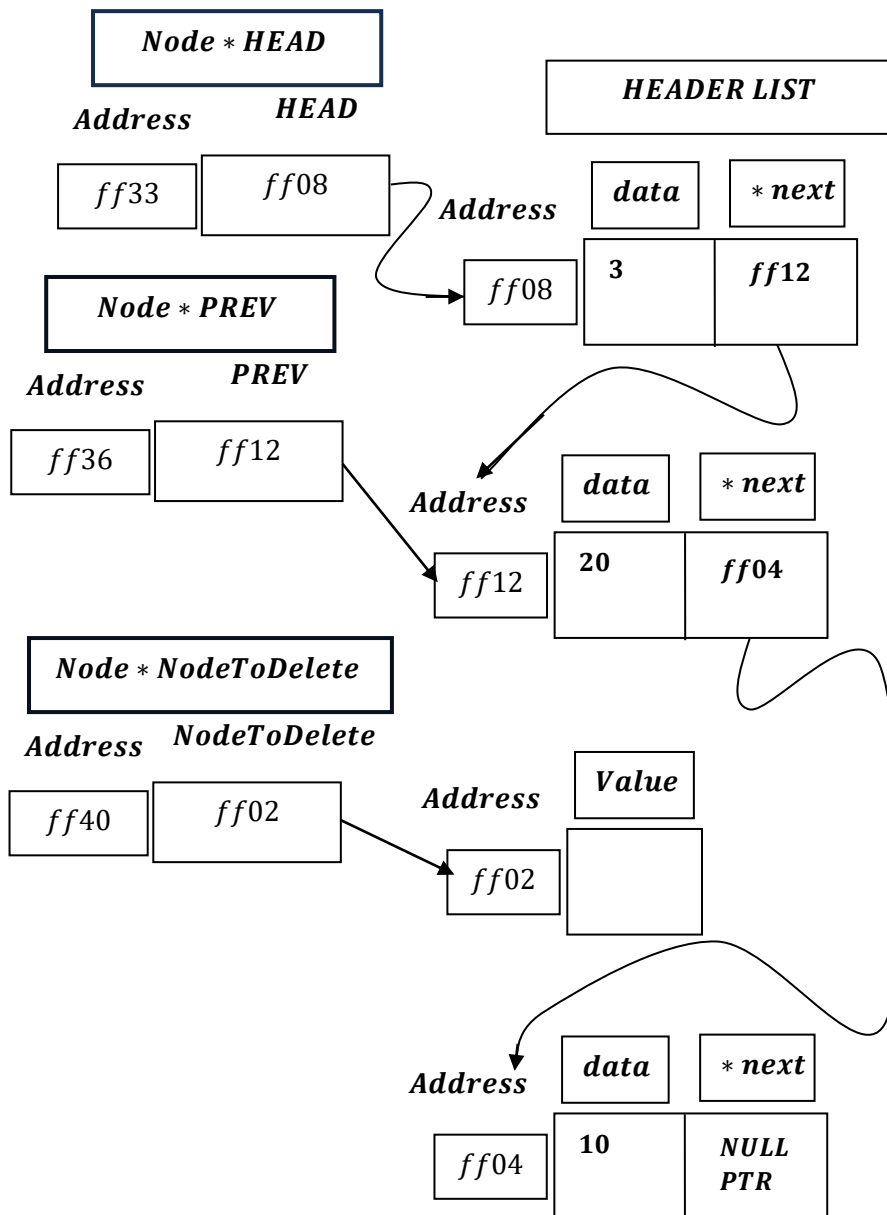
$\text{PREV} \rightarrow \text{NEXT} = \text{NodeToDelete} \rightarrow \text{NEXT} = \text{ff04}$.



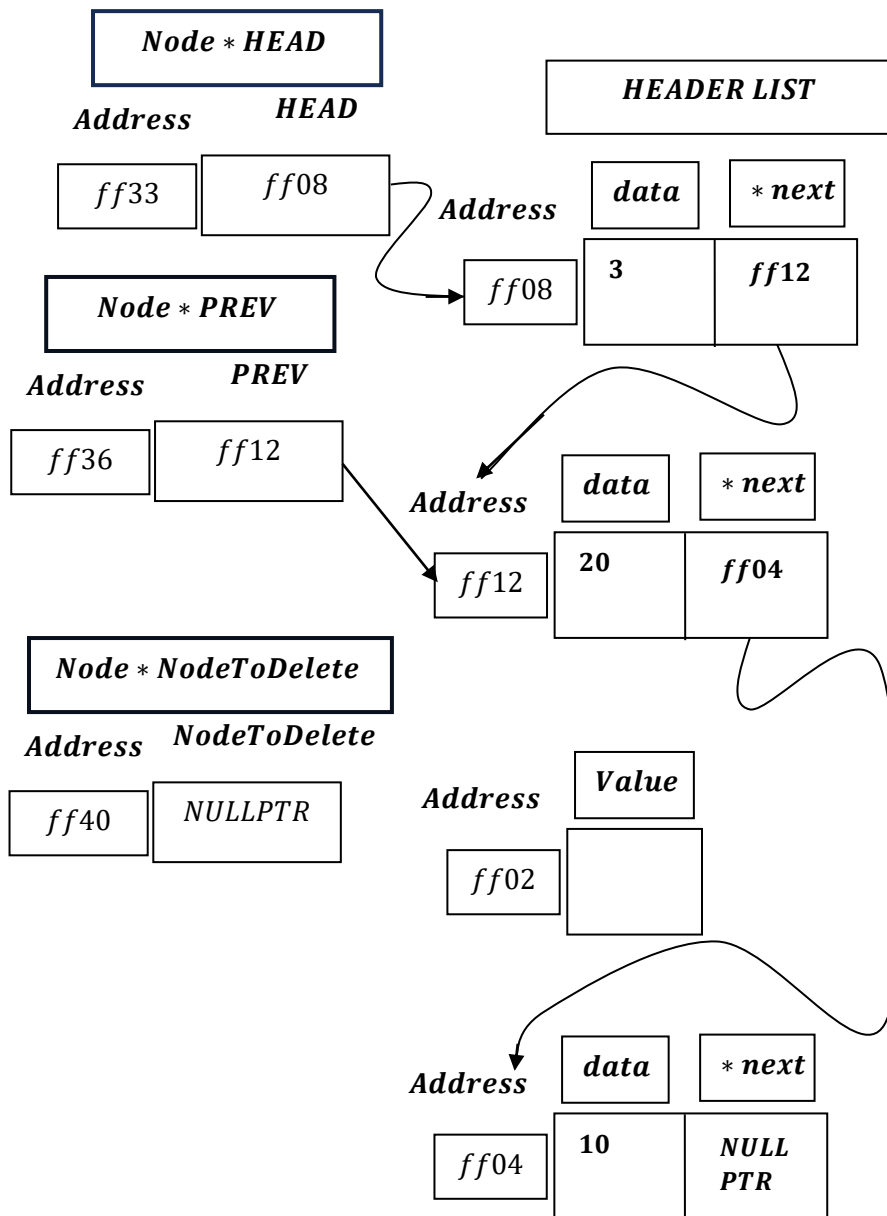
```
cout << "Deleted " << nodeToDelete->data << " from
position " << pos << "." << endl;
```

Output $\Rightarrow$ Print: Deleted `5` from position `2` .

```
free(nodeToDelete);
```

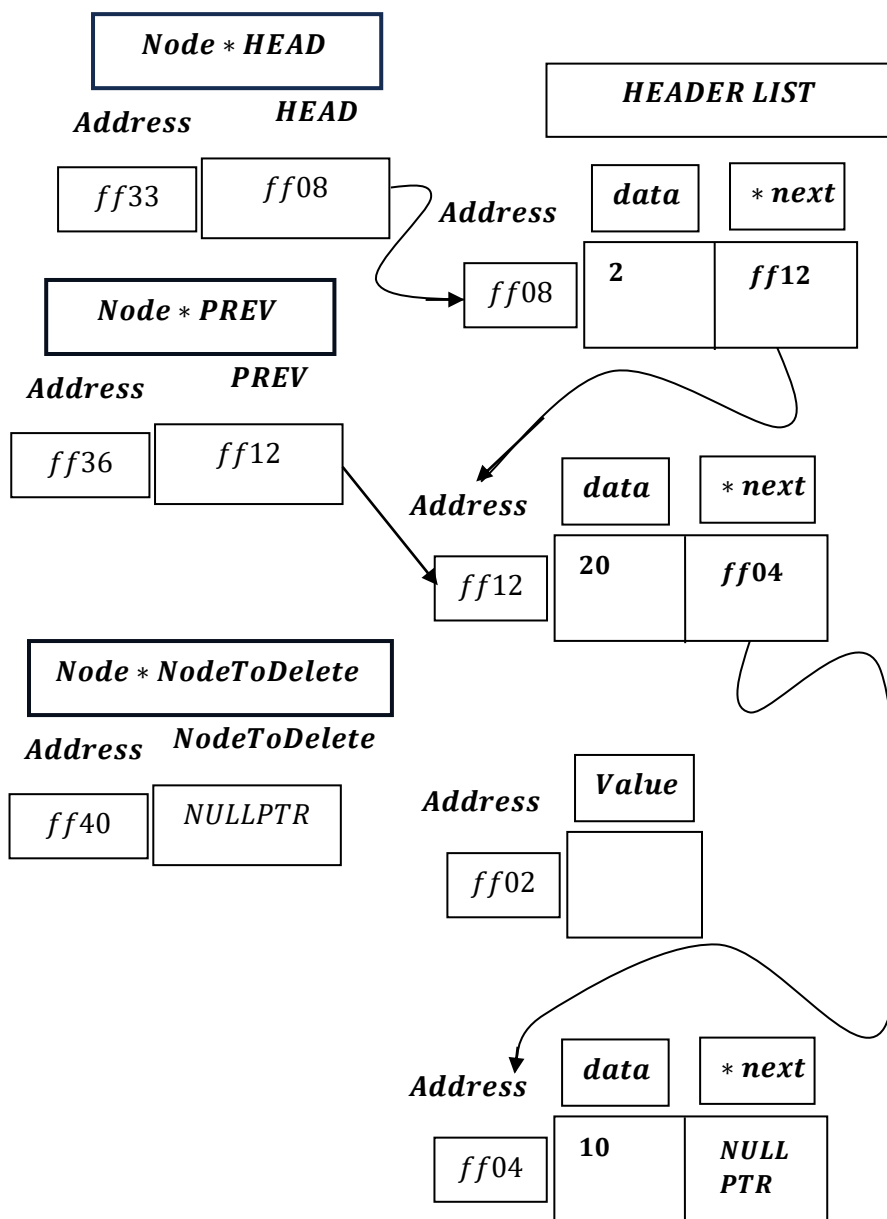


```
nodeToDelete = nullptr;
```

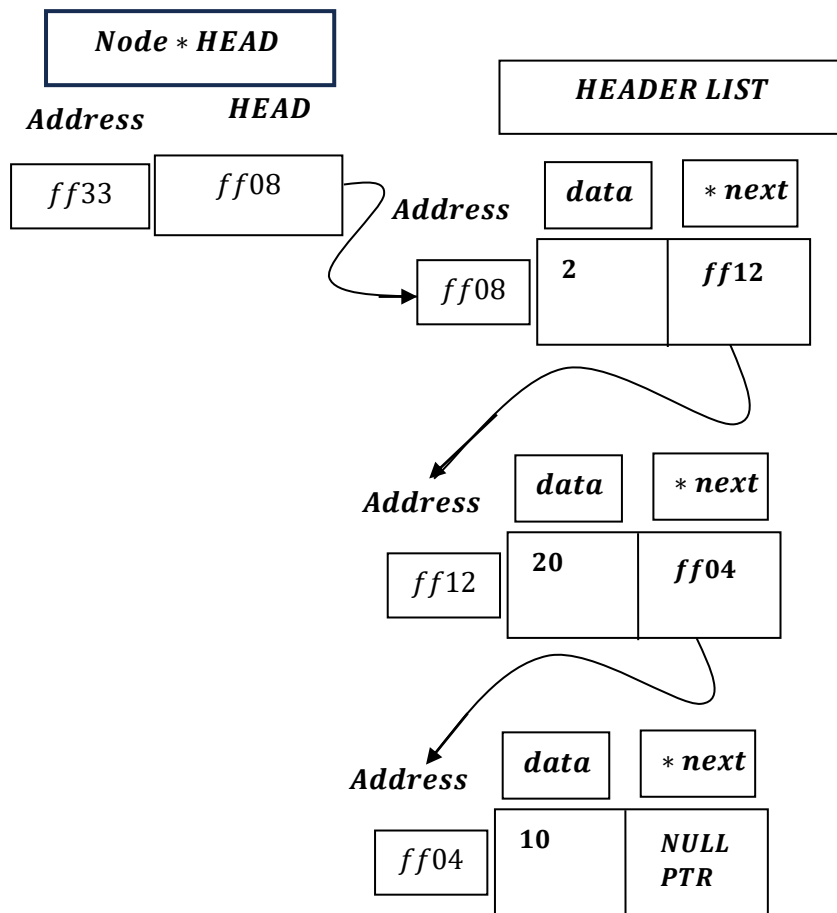


```
head->data--;
```

$Head \rightarrow Data -- \Rightarrow Head \rightarrow Data = Head \rightarrow Data - 1 = 3 - 1 = 2.$



And we get our final list:



10. Header Linked List

–Traverse In Order

Program:

```
void traverseInOrder()
{
    if (head->next == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    Node *temp = head->next; // Skip header
    cout << "Header -> ";

    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

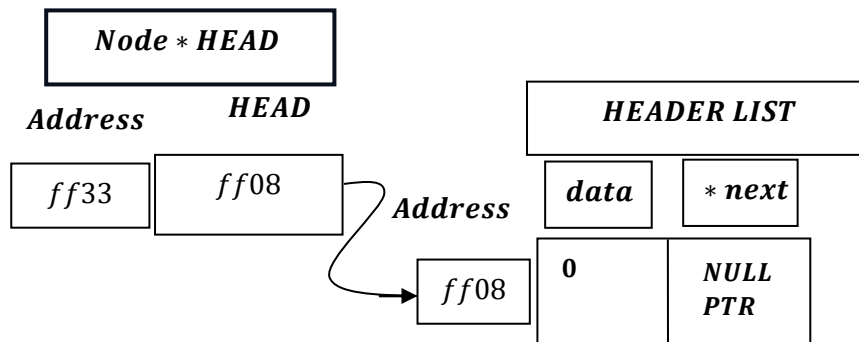
... .

case 8:
    traverseInOrder();
    break;
```

Program Analysis

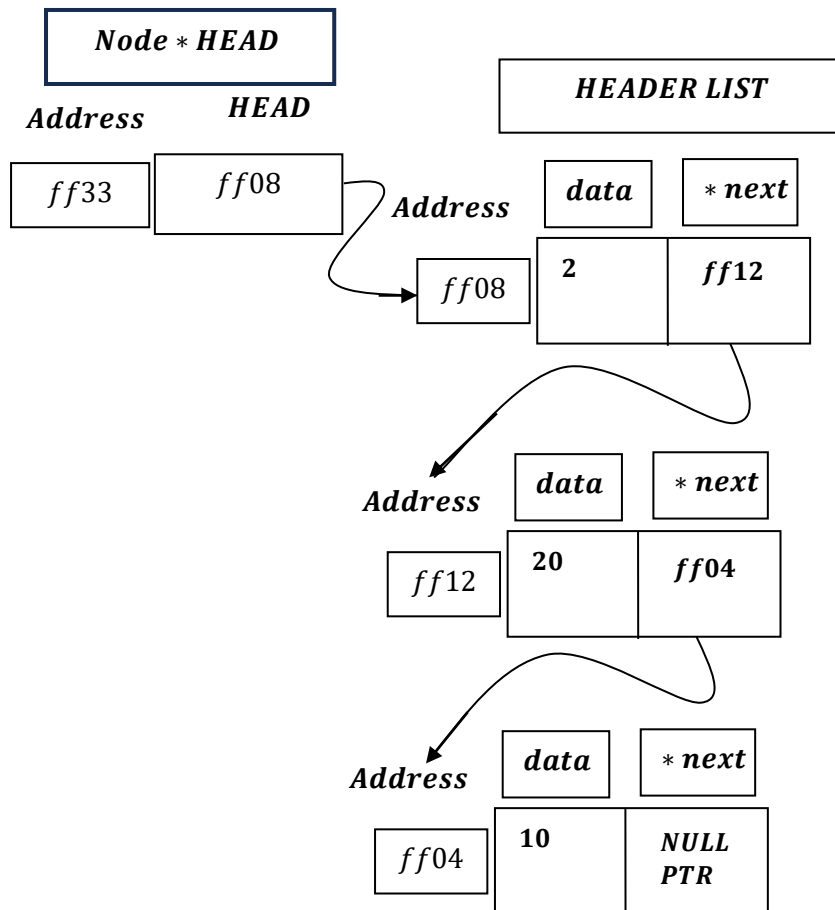
```
if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

i.e.



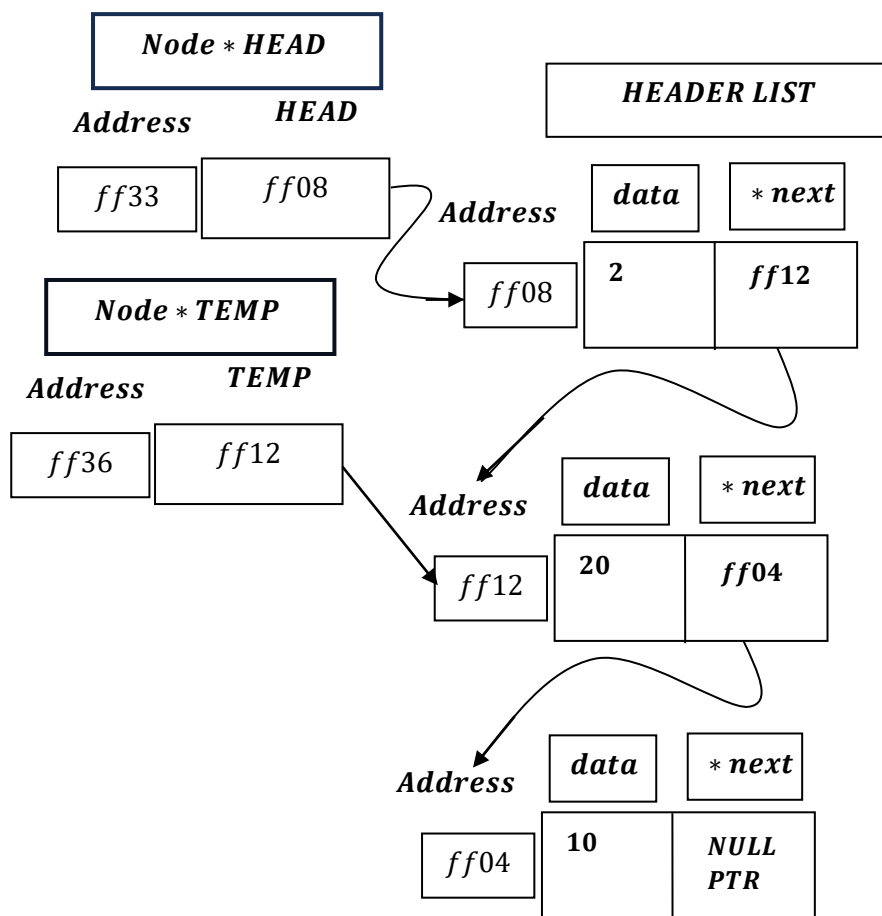
head → *next* = *NULLPTR* , *head* → *Data* = 0 , then list is empty .

If List is not Empty



```
Node *temp = head->next; // Skip header
```

Node * TEMP = HEAD → NEXT = ff12.



```
cout << "Header -> ";

while (temp != nullptr)
{
    cout << temp->data << " -> ";
    temp = temp->next;
}
```

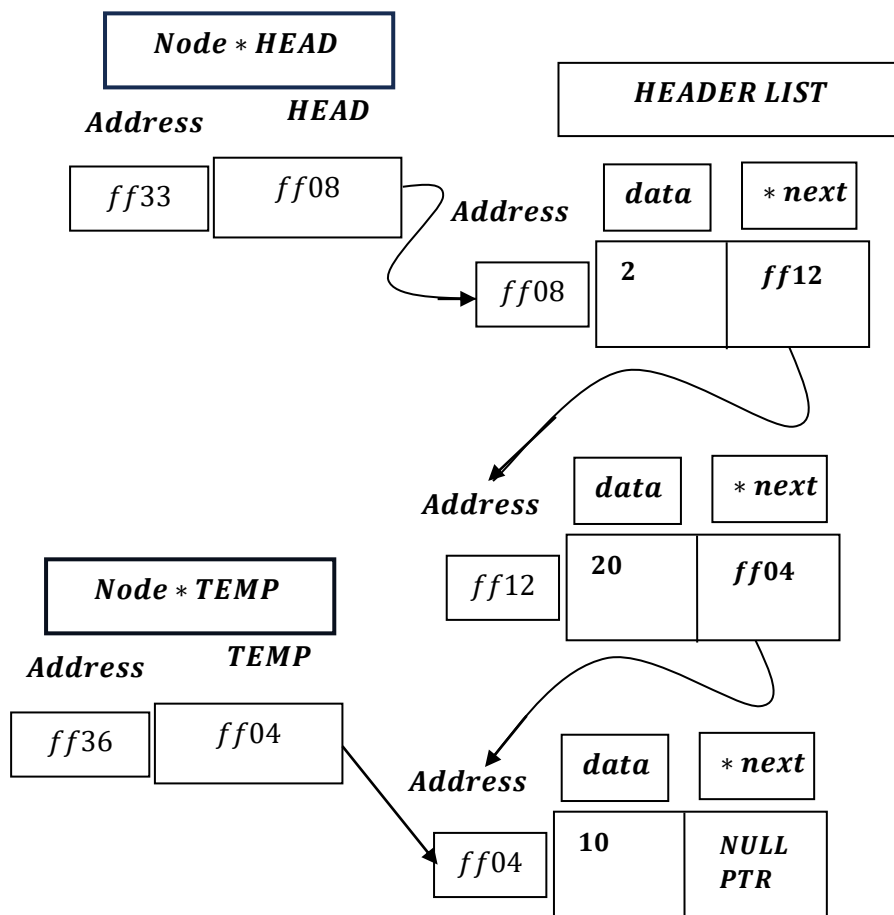
Output : ``Header →

Temp : ff12 ≠ NULLPTR :

Print : TEMP → DATA : 20 →

Temp = TEMP → NEXT = ff04.

i. e., OUTPUT: HEADER → 20 →

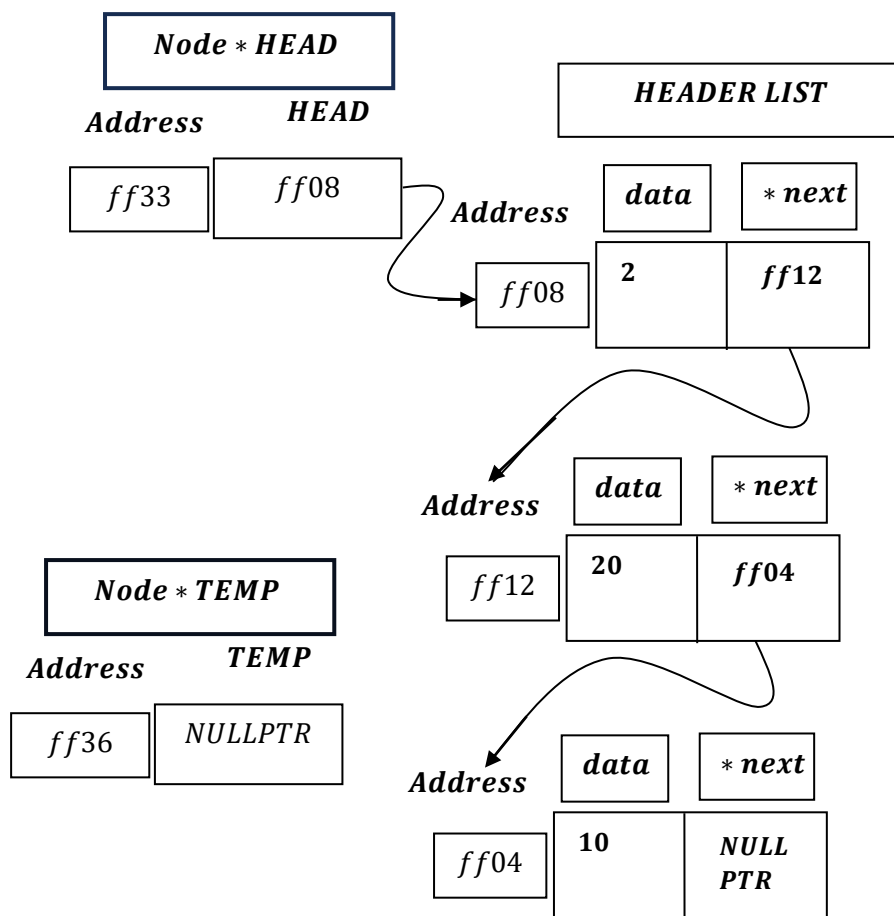


Temp : ff04 ≠ NULLPTR :

Print : TEMP → DATA : 10 →

Temp = TEMP → NEXT = NULLPTR.

i. e., OUTPUT: HEADER → 20 → 10 →



```
cout << "NULL" << endl;
```

i.e., OUTPUT: HEADER → 20 → 10 → NULL.

11. Header Linked List

–Traverse In Order

Program:

```
void destroyList()
{
    Node *current = head->next; // Start after header
    Node *nextNode;

    while (current != nullptr)
    {
        nextNode = current->next;
        free(current);
        current = nextNode;
    }

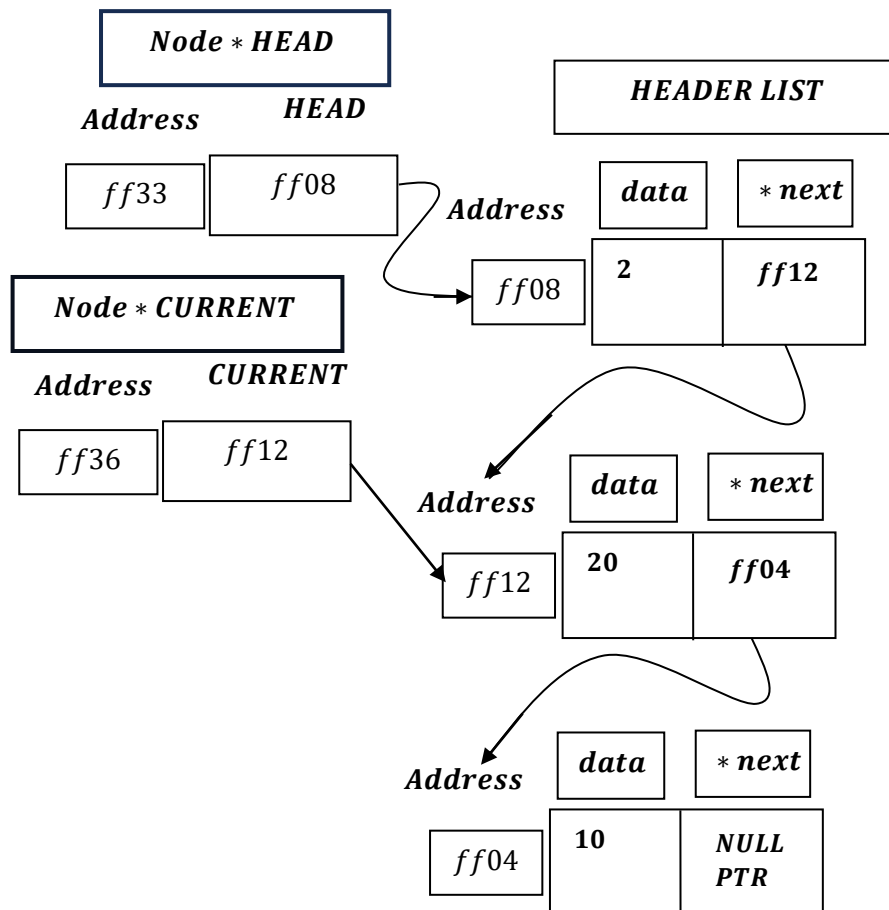
    // Reset header
    head->next = nullptr;
    head->data = 0;

    cout << "List cleared (Header node preserved)." << endl;
}
...

case 9:
    destroyList();
    break;
```

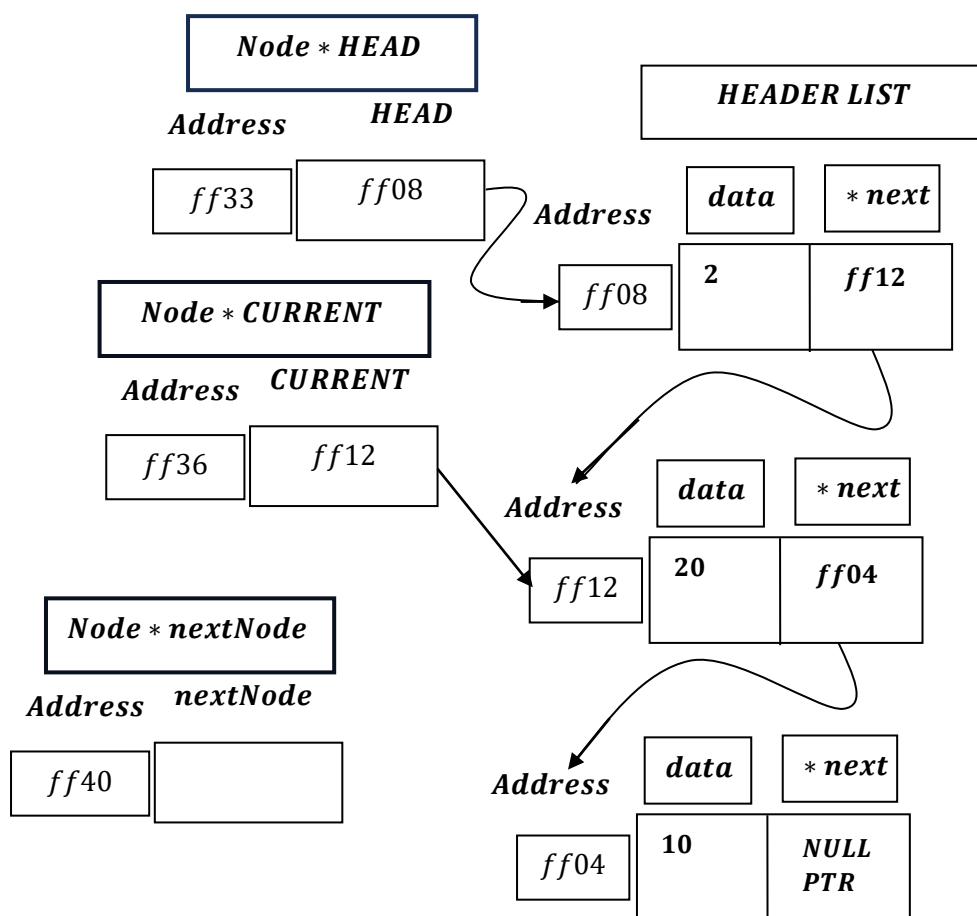
Program Analysis

```
Node *current = head->next; // Start after header  
Node *CURRENT = HEAD → NEXT = ff12.
```



```
Node *nextNode;
```

Declare Node * nextNode;



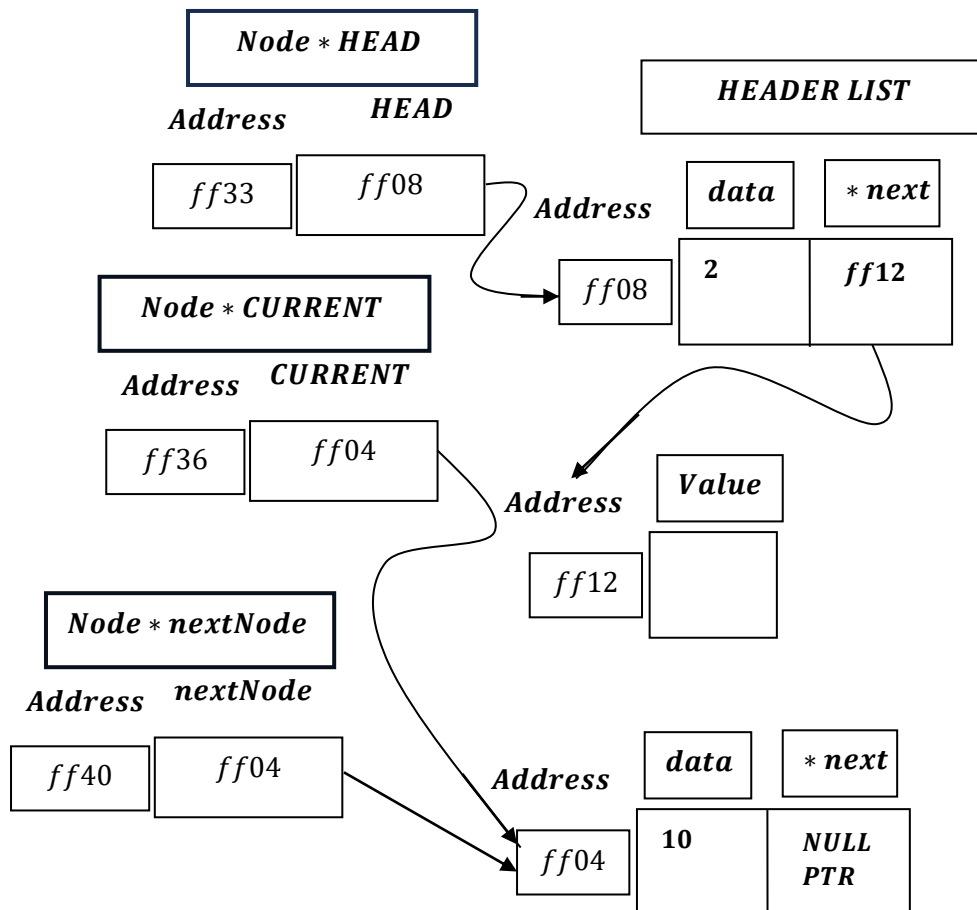
```
while (current != nullptr)
{
    nextNode = current->next;
    free(current);
    current = nextNode;
}
```

CURRENT: ff12 $\neq$ NULLPTR :

nextNode = current $\rightarrow$ next = ff04;

free(current: ff12);

current = nextNode = ff04;

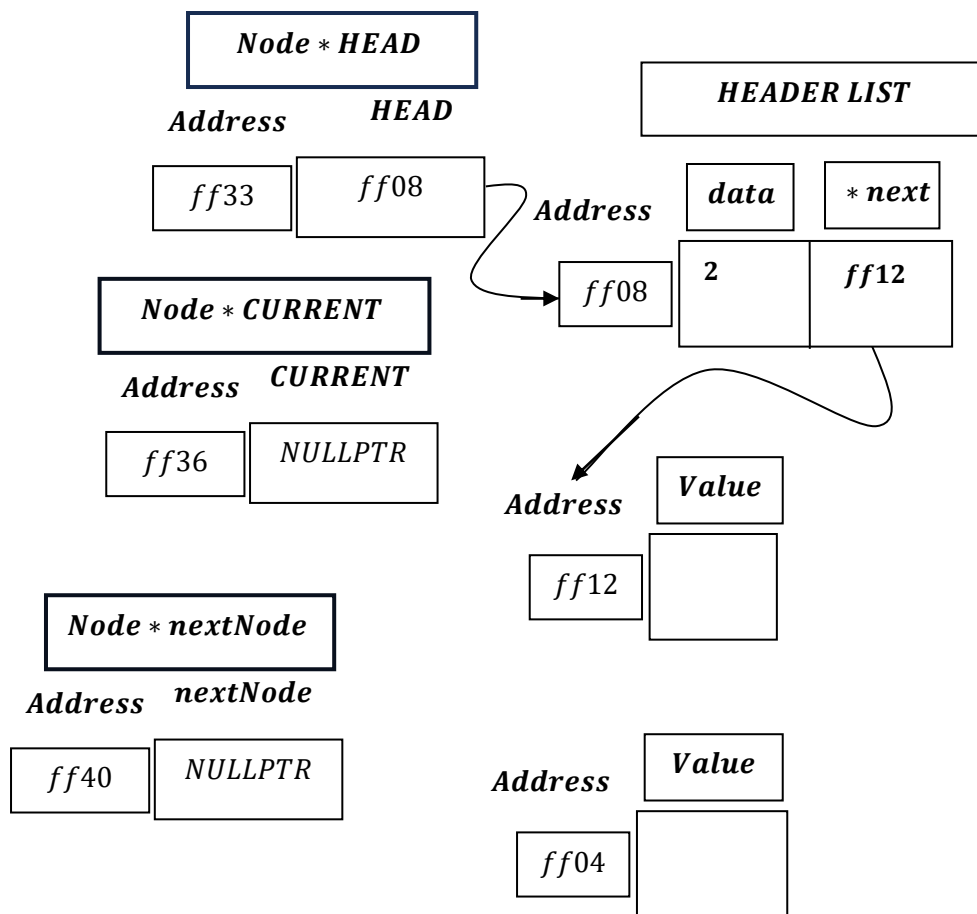


CURRENT: ff04 $\neq$ NULLPTR :

nextNode = current $\rightarrow$ next = NULLPTR;

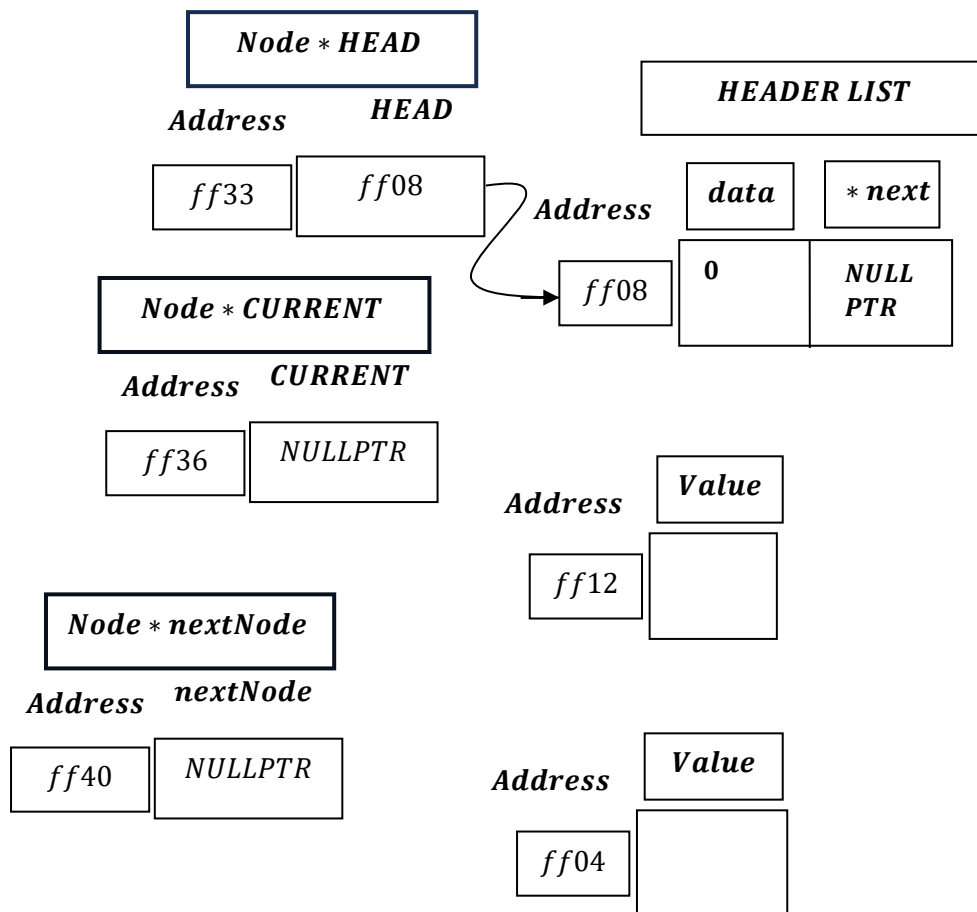
free(current: ff04);

current = nextNode = NULLPTR;



CURRENT: NULLPTR $\neq$ NULLPTR is false and hence exit from the loop.

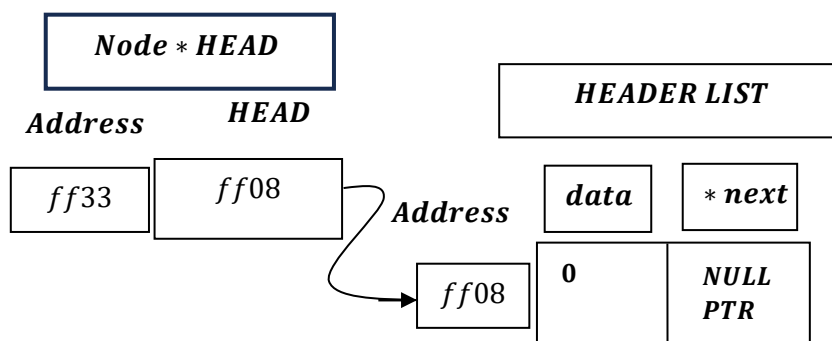
```
head->next = nullptr;
head->data = 0;
```



```
cout << "List cleared (Header node preserved)." << endl;
```

Output: Print → ``List cleared(Header node preserved).``

Hence, after destroying the list we have :



Therefore, Header Node is preserved , and remain undestroyed , even if the list is fully destroyed.

12. Header Linked List

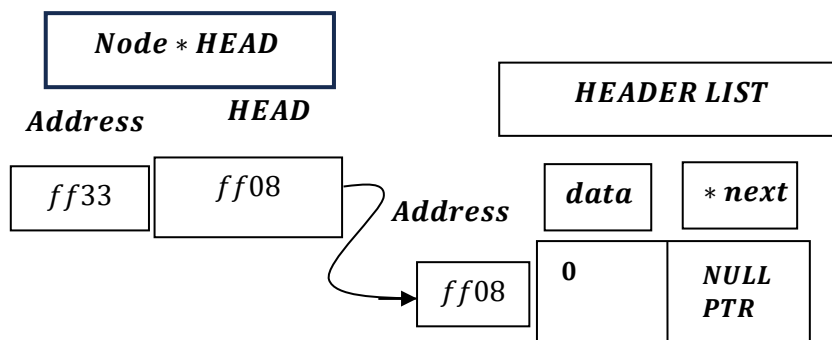
–Free Header

Program:

```
#include <cstdlib>
void freeHeader() {
    if (head != nullptr) {
        free(head);
        head = nullptr;
    }
}
...
case 10:
    cout << "Exiting program. Goodbye!" << endl;
    destroyList();
    freeHeader(); // Free the actual header node
    exit(0);
```

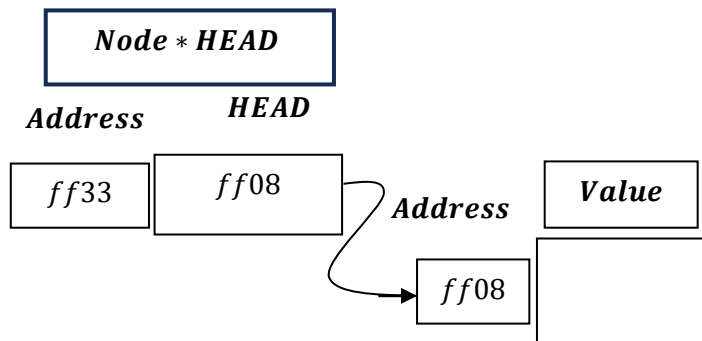
Program Analysis

We know after destroyList() function, we get:

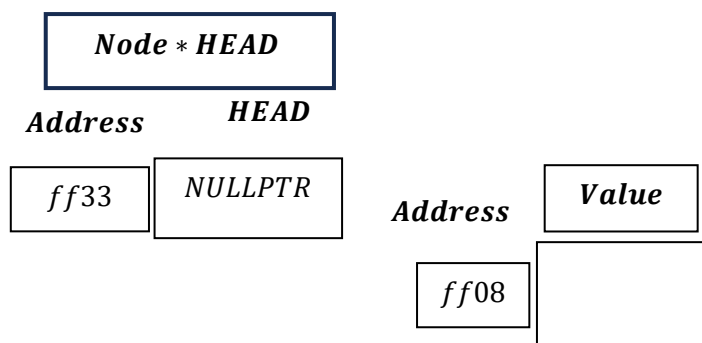


When head $\neq$ NULLPTR :

free (head : ff08);

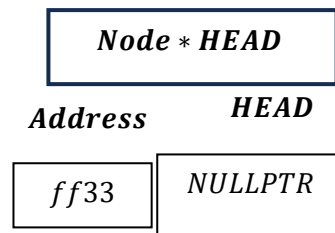


head = nullptr;



And exit(0) of cstdlib header , exits from program successfully.

And now , we are left with:



13. Header Linked List – Time Complexity and Space Complexity

1. Creation of Header List:

- **Time Complexity:**

→ **$Best = Worst = Average = O(1).$**

- **Space Complexity:**

$Input\ Space = O(1).$

$Auxiliary\ Space = O(1).$

$Total\ Space\ Complexity: Input\ Space + Auxiliary = O(1) + O(1) = O(1).$

2. Insert At Beginning:

- **Time Complexity:**

→ **$Best = Worst = Average = O(1).$**

- **Space Complexity:**

→ **$Considering\ only\ int\ data\ as\ input,\ then:$**

$Input\ Space = O(1).$

$Auxiliary\ Space = O(1).$

$Total\ Space\ Complexity: Input\ Space + Auxiliary = O(1) + O(1) = O(1).$

→ **$Considering\ 'n'\ element\ as\ input,\ then:$**

$Input\ Space = O(n).$

$Auxiliary\ Space = O(1).$

Total Space Complexity: $\text{Input Space} + \text{Auxiliary} = O(n) + O(1) = O(n)$.

3. Insert At END:

- ***Time Complexity:***

→ Best = $\Omega(1)$. $\left[\begin{array}{l} \text{When List is Empty} \\ \text{and we enter first data to list.} \end{array} \right]$

***→ Worst Case = $O(n)$ = Average Case = $\Theta(n)$.
 $\left[\begin{array}{l} \text{When List is not empty and there is numerous} \\ \text{Nodes in the list. Note: True Average Case doesn't} \\ \text{exist here.} \end{array} \right]$***

- ***Space Complexity:***

→ Considering only int data as input, then:

***Input Space = $O(1)$.
Auxiliary Space = $O(1)$.***

Total Space Complexity: $\text{Input Space} + \text{Auxiliary} = O(1) + O(1) = O(1)$.

→ Considering `n` element as input , then:

***Input Space = $O(n)$.
Auxiliary Space = $O(1)$.***

Total Space Complexity: $\text{Input Space} + \text{Auxiliary} = O(n) + O(1) = O(n)$.

3. Insert At Position:

- **Time Complexity:**

→ **Best** = $\Omega(1)$. [When $Pos < 1$ or $Pos > head \rightarrow data + 1$]

→ **Worst Case** = $O(n)$. [When Position is not found.]

→ **Average Case**

[Based on Loop]

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 1 time.

→ If the position is 3, the loop's inner statement runs 2 times.

→ If the position is 4, the loop's inner statement runs 3 times.

.

.

→ If the position is $(N + 1)$, the loop's inner statement runs : N times ,

As when $i = N < N$ becomes false , hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^N i = \frac{N(N+1)}{2}$$

Now, as we take all the possibilities then , we take the best case also i.e.

$O(1)$, when position = 1 . Considering ' $T(P)$ ' is time complexity of position ' P ' .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ P - 1 & , \text{when } P = 2, 3, \dots, N + 1. \end{cases}$$

Now, If there is `N` no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N + 1}$.

For position 2, the probability of insertion is $\frac{1}{N + 1}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N + 1}$.

Hence probability of insertion will always remain $\frac{1}{N + 1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N + 1} \times 1 + \frac{1}{N + 1} \times 2 + \dots + \frac{1}{N + 1} \times N \right) + \frac{1}{N + 1} \times 1$$

$$= \frac{1}{N + 1} \times \{(1 + 2 + 3 + \dots + N) + 1\}$$

$$= \frac{1}{N+1} \times \left\{ \left(\sum_{i=1}^N i \right) + 1 \right\}$$

$$= \frac{1}{N+1} \times \left\{ \frac{N(N+1)}{2} + 1 \right\}$$

$$= \left\{ \frac{N(N+1)}{2(N+1)} + \frac{1}{N+1} \right\}$$

$$= \left\{ \frac{N}{2} + \frac{1}{N+1} \right\}$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

→ $\Theta(N)$ grows linearly with N .

→ $\Theta\left(\frac{1}{N}\right)$ decreases to a constant, i. e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n} \right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher , therefore , average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

**Average Case Time Complexity[Most Simplest Alternative Apporach]:
(Simplest approach)**

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

**Hence probability of insertion will always remain $\frac{1}{N+1}$,
for position 1 to $N+1$.**

Average case complexity : Probability of insertion $\times$ $\left(\begin{array}{l} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{1}{2} (N+1)((N+1)+1) \right) \left[\text{As, } \sum_{k=1}^n k = \frac{1}{2} (n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left(\frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N + 2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

Applying Big Theta Θ we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N}{2} + 1\right)$ or $\Theta(N)$.

- **Space Complexity:**

→ Considering only int data, int pos as input, then:

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

→ Considering `n` element as input i.e. `n` nodes , then:

Input Space = $O(n)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(n) + O(1) = O(n)$.

3.Insert After Element:

- **Time Complexity:**

$$\rightarrow \text{Best} = \Omega(1). \left[\begin{array}{l} \text{When List is Empty} \\ \text{and Head} \rightarrow \text{Next} = \text{NULL} \end{array} \right]$$

$$\rightarrow \text{Worst Case} = O(n). \left[\begin{array}{l} \text{When Element is not found} \\ \text{and List is not empty i. e.} \\ \text{Head} \rightarrow \text{Next} \neq \text{NULL} \end{array} \right]$$

$\rightarrow$ **Average Case**

$\rightarrow$ If the Element is inserted after 1st node, the loop's inner statement runs 0 time.

$\rightarrow$ If the Element is inserted after 2nd node, the loop's inner statement runs 1 time.

$\rightarrow$ If the Element is inserted after 3rd node, the loop's inner statement runs 2 time.

$\rightarrow$ If the Element is inserted after 4th node, the loop's inner statement runs 3 time.

.

.

$\rightarrow$ If the Element is inserted at $(n + 1)^{\text{th}}$ node, the loop's inner statement runs $N - 1$ time.

$$\therefore, \sum_{i=0}^{N-1} i = \frac{N(N-1)}{2}$$

Now, If there is 'N' no. of nodes there we can insert data at 2 to N + 1 positions or repositioning 1 to N .

For position 1, the probability of insertion is $\frac{1}{N}$.

For position 2, the probability of insertion is $\frac{1}{N}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N}$.

Hence probability of insertion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations)

$$= \frac{1}{N} \times \left(\sum_{i=0}^{N-1} i \right)$$

$$= \frac{1}{N} \times \frac{N(N-1)}{2}$$

$$= \frac{N-1}{2}$$

Applying , Big O on the above equation:

$$= \frac{1}{2} \times O(N) - O\left(\frac{1}{2}\right)$$

$$= O(N)$$

Hence, Average Time Complexity is : $\Theta(N)$.

***Average Case Time Complexity[Most Simplest Alternative Approach]:
(Simplest approach)***

***(If the elements in the linked list are uniformly distributed
and the key is equally likely to be any element in the list)***

***Note: –The checking that Element or Key is found or not , done in the
condition of while loop, and the condition checks `n` times .***

–if key is found at 1st Node , Key/Element searched for 1 time .

–if key is found at 2nd Node , Key/Element searched for 2 time .

–if key is found at 3rd Node, Key/Element searched for 3 time .

.

.

–if key is found at Nth Node , Key/Element searched for N time .

As , the key is equally likely to be any element in the list,

Probability of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

Probability of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

Probability of finding Nth element (p_n), if key equals to Nth element = $\frac{1}{N}$.

Therefore,

Probability(P) = $p_1 = p_2 = \dots = p_n = \frac{1}{N}$.

Therefore, Average Case = Number of times the Key is searched × probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned}\therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \\ &= \frac{N+1}{2} \\ &= \frac{N}{2} + \frac{1}{2}\end{aligned}$$

Applying , Big Theta Θ we get :

$$\begin{aligned}&= \frac{1}{2} \times (\Theta(N) + \Theta(1)) \\ &= \Theta(N) + \Theta(1) \\ &= \Theta(N).\end{aligned}$$

- ***Space Complexity:***

→ ***Considering only int data, int element as input, then:***

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: $Input\ Space + Auxiliary = O(1) + O(1) = O(1)$.

→ ***Considering `n` element as input i.e. `n` nodes , then:***

Input Space = $O(n)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: $Input\ Space + Auxiliary = O(n) + O(1) = O(n)$.

13. Header Linked List – Time Complexity and Space Complexity –Part 2

1. Delete from Beginning:

- **Time Complexity:**

→ **$Best = Worst = Average = O(1).$**

- **Space Complexity:**

→ **No Parametric Input hence, Parametric Input Space:**

$Input\ Space = O(1).$

$Auxiliary\ Space = O(1).$

Total Space Complexity: $Input\ Space + Auxiliary = O(1) + O(1) = O(1).$

→ **Full Data Definition :**

$Input\ Space[Considering\ the\ whole\ list] = O(n).$

$\left[\begin{array}{l} \text{Deleting one node , total nodes reduces to } n - 1 = \\ O(n - 1) = O(n). \end{array} \right]$

$Auxiliary\ Space = O(1).$

$\left[\begin{array}{l} \text{This is the extra space or temporary memory} \\ \text{used by the algorithm to complete} \\ \text{its task, excluding the space taken by} \\ \text{the input data.} \end{array} \right]$

Total Space Complexity: $Input\ Space + Auxiliary = O(n) + O(1) = O(1).$

2. Delete At End:

- *Time Complexity:*

$$\rightarrow \text{Best} = \Omega(1). \left[\text{When List is Empty} \right]$$

$$\rightarrow \text{Worst Case} = O(n) = \text{Average Case} = \Theta(n). \\ \left[\begin{array}{l} \text{When List is not empty and there is numerous} \\ \text{Nodes in the list. Note: True Average Case doesn't} \\ \text{exist here.} \end{array} \right]$$

- *Space Complexity:*

$\rightarrow$ Parametric Input Space:

There is no parameter present hence:

$$\begin{aligned} \text{Input Space} &= O(1). \\ \text{Auxiliary Space} &= O(1). \end{aligned}$$

$$\text{Total Space Complexity: Input Space} + \text{Auxiliary} = O(1) + O(1) = O(1).$$

$\rightarrow$ Full Data Definition:

Here Input Space includes `n` no. of elements at `n` no. of nodes and last node gets deleted i.e. $n - 1$, then:

$$\begin{aligned} \text{Input Space} &= n - 1 = O(n - 1) = O(n). \\ \text{Auxiliary Space} &= O(1). \end{aligned}$$

$$\text{Total Space Complexity: Input Space} + \text{Auxiliary} = O(n) + O(1) = O(n).$$

3. Delete From Position:

- *Time Complexity:*

$$\rightarrow \text{Best} = \Omega(1). \left[\begin{array}{c} \text{When List is Empty} \\ \text{Or} \\ \text{Pos} < 1 \text{ or } \text{Pos} > \text{Head} \rightarrow \text{Data} \end{array} \right]$$

$$\rightarrow \text{Worst Case} = O(n) [\text{When `pos` is not found.}]$$

$$\rightarrow \underline{\text{Average Case:}}$$

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 1 time.

→ If the position is 3, the loop's inner statement runs 2 time.

.

.

→ If the position is (N), the loop's inner statement runs (N – 1) times.

As when $i = N < N$ becomes false, hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

Now, as we take all the possibilities then, we take the best case also i.e.

$O(1)$, when position = 1. Considering 'T(P)' is time complexity of position 'P'.

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ P - 1 & , \text{when } P = 2, 3, 4, \dots, N. \end{cases}$$

Now, If there is `N` no. of nodes there we can delete data at N positions.

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of deletion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \dots + \frac{1}{N} \times N - 1 \right) + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \dots + N - 1) + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant '\frac{1}{2}', we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms , largest growth rate is the result:

→ $\Theta(N)$ grows linearly with ' N ' .

→ $\Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant}\right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

*Average Case Time Complexity [Most Simplest Alternative Approach]:
(Simplest approach)*

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N , the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$, for position 1 to N .

Average case complexity : Probability of deletion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array}\right)$

$$= \frac{1}{N} \times \left(\sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$\text{or, } \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant '\(\frac{1}{2}\)', we get:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N+1}{2}\right)$ or $\Theta(N)$.

- **Space Complexity:**

→ Parametric Input Space:

There is a parameter `int pos` present hence:

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

→ Full Data Definition:

Here Input Space includes `n` no. of elements at `n` no. of nodes and if a node gets deleted i. e. $n - 1$, then:

Input Space = $n - 1 = O(n - 1) = O(n)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(n) + O(1) = O(n)$.

4. Destroy List:

- *Time Complexity:*

$$\rightarrow \text{Best} = \Omega(1). \left[\text{When List is Empty} \right]$$

$$\rightarrow \text{Worst Case} = O(n) = \text{Average Case} = \Theta(n). \\ \left[\begin{array}{l} \text{When List is not empty and there is numerous} \\ \text{Nodes in the list. Note: True Average Case doesn't} \\ \text{exist here.} \end{array} \right]$$

- *Space Complexity:*

$\rightarrow$ Parametric Input Space:

There is no parameter present hence:

$$\begin{aligned} \text{Input Space} &= O(1). \\ \text{Auxiliary Space} &= O(1). \end{aligned}$$

$$\text{Total Space Complexity: Input Space} + \text{Auxiliary} = O(1) + O(1) = O(1).$$

$\rightarrow$ Full Data Definition:

Before Deletion:

$$\begin{aligned} \text{List `n` nodes} &\rightarrow O(n). \left[\text{Input Space} \right] \\ \text{Auxiliary Space} &\rightarrow O(1). \end{aligned}$$

$$\begin{aligned} \text{Total Space Complexity: Input Space} + \text{Auxiliary} &= O(n) + O(1) \\ &= O(n). \end{aligned}$$

After Deletion:

- ***List becomes empty $\rightarrow O(1)$. [Input Space]***
- ***Auxiliary Space $\rightarrow O(1)$.***

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

5.Free Header:

- ***Time Complexity:***

$\rightarrow \text{Best} = \text{Worst} = \text{Average} = O(1)$.

- ***Space Complexity:***

$\rightarrow$ For both, Parametric Input Space and Full Data Definition:

***Input Space = $O(1)$.
Auxiliary Space = $O(1)$.***

[As, DestroyList() function already executed.]

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

Header Linked List

–Memory Allocation

We are examining Header Singly Linked List:

Lets Recall of Memory Allocation of Singly Linked List :

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ next(a pointer to another Node) is typically 8 bytes.

→ Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for next) .

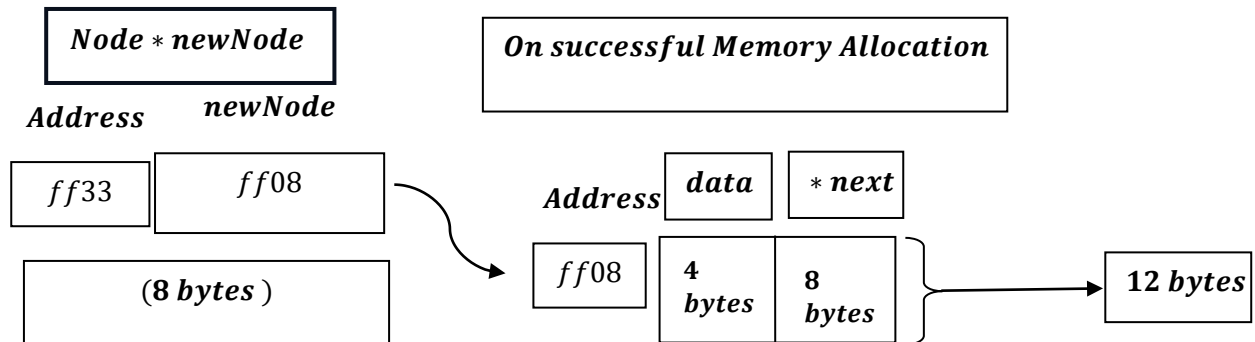
```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ newNode will try to allocate memory for data i.e. 4 bytes , next pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a

pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real-World Allocation

In a real programming environment (like the C + + environments you typically work in)

, the allocation might not be exactly 12 bytes because of

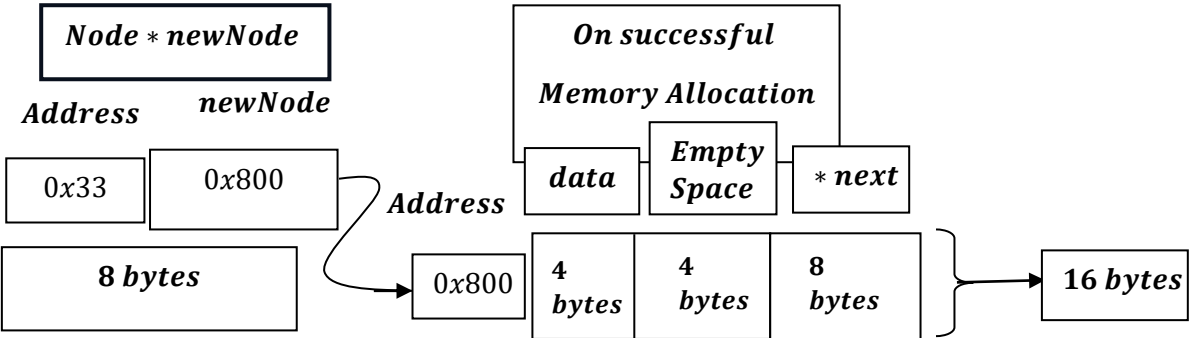
how CPUs access memory:

- **Memory Alignment:** Most 64 – bit compilers align data to 8 – byte boundaries to improve performance.
- **Padding:** To align the 8 – byte next pointer, the compiler will often add 4 bytes of "padding" after the 4 – byte data integer.
- **Actual Size:** Consequently, `sizeof(Node)` would likely return 16 bytes on your system, rather than the 12 bytes.

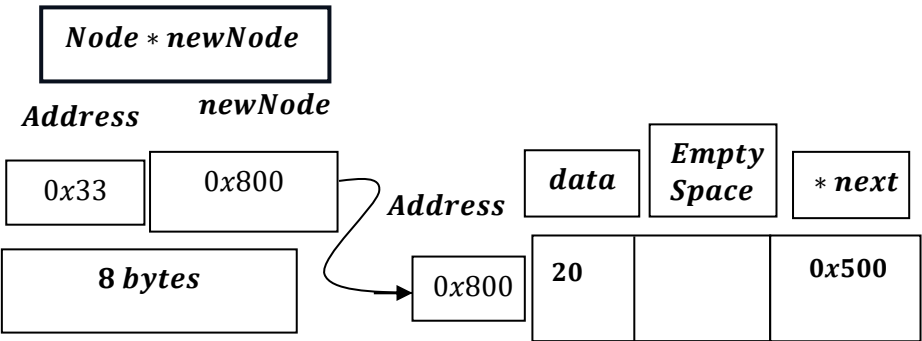
<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0	4 bytes	data	The integer value (e.g., 2).
4	4 bytes	Padding	Empty space added by the compiler.
8	8 bytes	next	The next address of the Node in the linked list (e.g., 0x500).

It will look like:

Say,



Like:



In a 64 – bit environment, the compiler aligns memory to the size of the largest primitive member in a structure – in this case, the 8 – byte next pointer.

Because the data integer only occupies 4 bytes, the compiler inserts empty space (padding) to ensure the pointer starts at an address that is a multiple of 8.

Such as:

$$0x500 = 5 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 5 \times 256 = 1280$$

$$1280 \div 8 = 160 \text{ (no remainder)}$$

Why so ?

As, it depends on 32 – bit system and 64 system.

In 64 – bit system, memory allocation is total : 16 bytes.

*[int variable – 4 bytes
padding – 4 bytes
Next pointer – 8 bytes]*

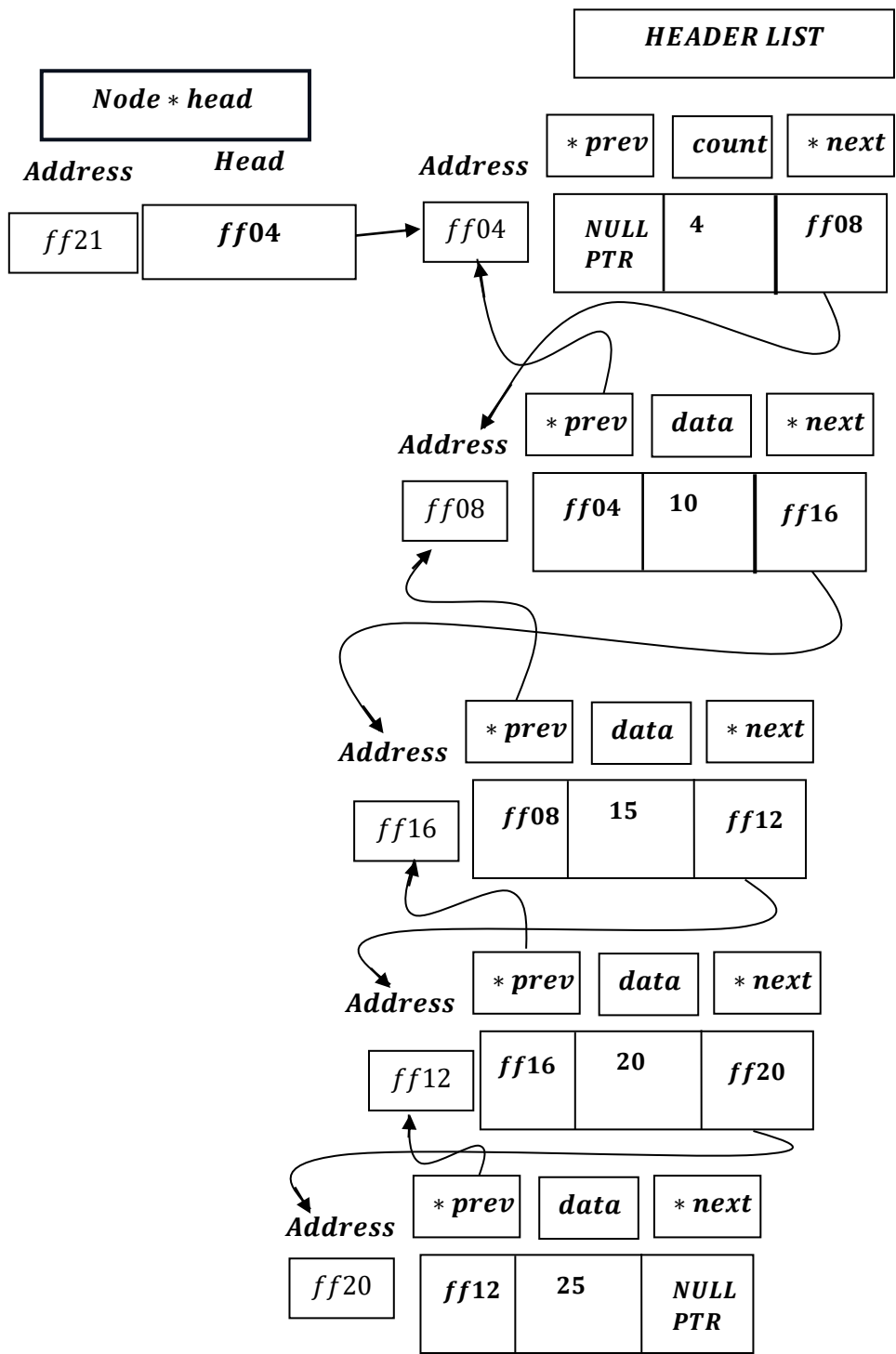
Where, as in 32 – bit system, memory allocation is total 8 bytes.

Where for pointer it is allocated – 4 bytes.

and for integer variable, memory allocated is total – 4 bytes.

Hence for 32 – bit system no, such padding is needed.

If we see Header Doubly Linked List then :



Lets Recall of Memory Allocation of Doubly Linked List :

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ next(a pointer to another Node) is typically 8 bytes.

→ prev(a pointer to previous Node) is typically 8 bytes.

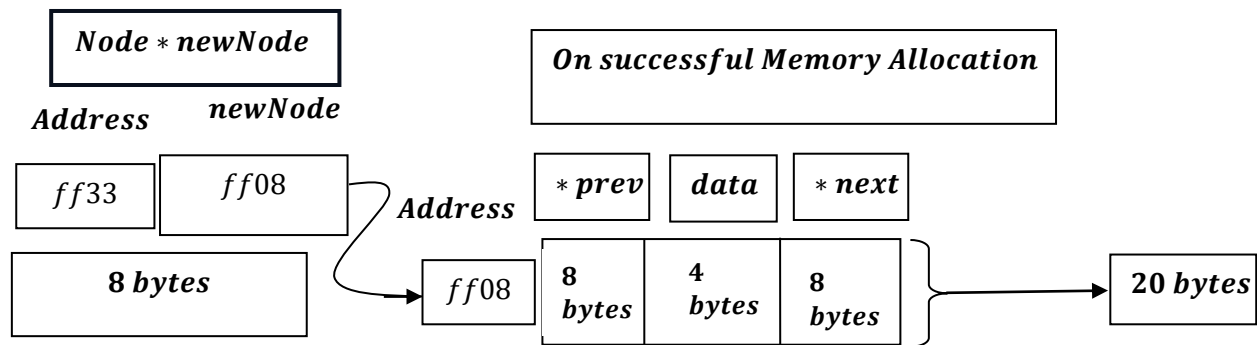
→ Therefore ,the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for prev) .

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ newNode will try to allocate memory for data i.e. 4 bytes ,next pointer variable i.e. typically 8 bytes, and prev pointer variable that is typically 8 bytes i.e. total 20 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real – World Allocation

In a real programming environment (like the C ++ environments),

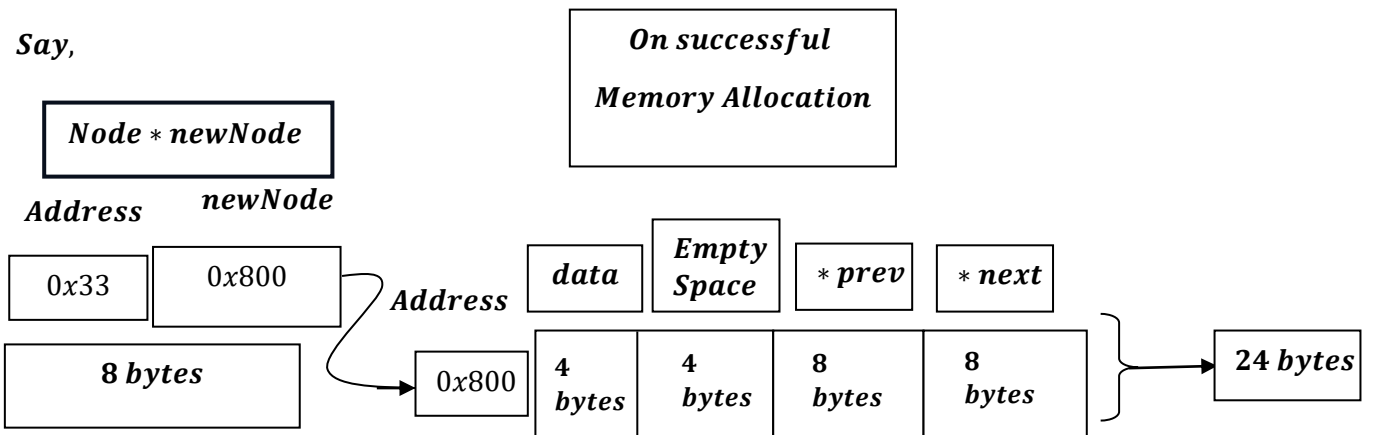
*the allocation might not be exactly **20 bytes** because of how CPUs access memory:*

- **Memory Alignment:** Most 64 – bit compilers align data to **8 – byte boundaries** to improve performance.
- **Padding:** To align the 8 – byte next pointer, the compiler will often add **4 bytes of "padding"** after the 4 – byte data integer.

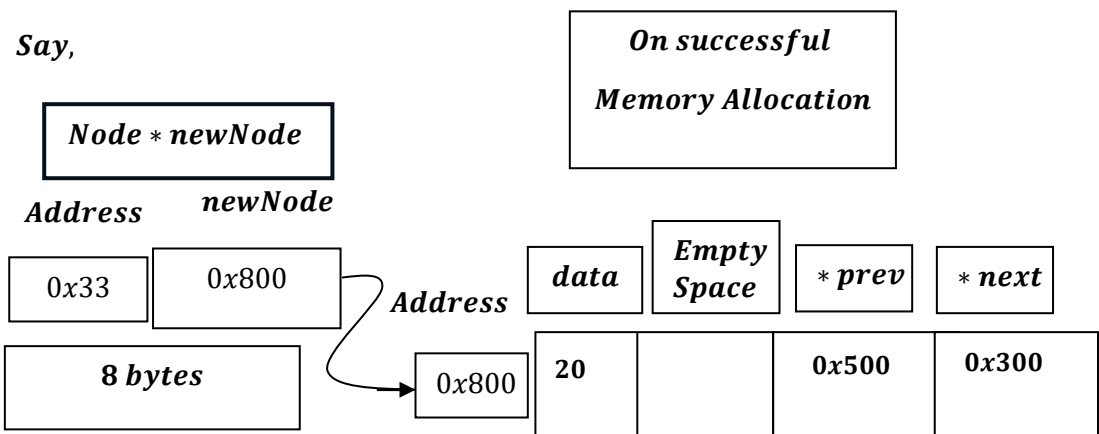
Additionally, since there is another 8 – byte pointer (prev), the structure must maintain proper alignment for both pointers.

- **Actual Size:** Consequently, `sizeof(Node)` would likely return **24 bytes** on your system, rather than the theoretical **20 bytes**.

<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0	4 bytes	<i>data</i>	<i>Integer value</i>
4	4 bytes	<i>padding</i>	<i>Empty space (alignment)</i>
8	8 bytes	<i>next</i>	<i>Address of next node</i>
16	8 bytes	<i>prev</i>	<i>Address of previous node</i>



Like:



In a 64 – bit environment, the compiler aligns memory to the size of the largest primitive member in a structure – in this case, the 8 – byte next pointer.

Because the data integer only occupies 4 bytes, the compiler inserts empty space (padding) to ensure the pointer starts at an address that is a multiple of 8.

Such as:

$$0x500 = 5 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 5 \times 256 = 1280$$

$$1280 \div 8 = 160 \text{ (no remainder)}$$

$$0x300 = 3 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 3 \times 256 = 768$$

$$768 \div 8 = 256 \text{ (no remainder)}$$

Why so ?

As, it depends on 32 – bit system and 64 system.

In 64 – bit system, memory allocation is total : 24 bytes.

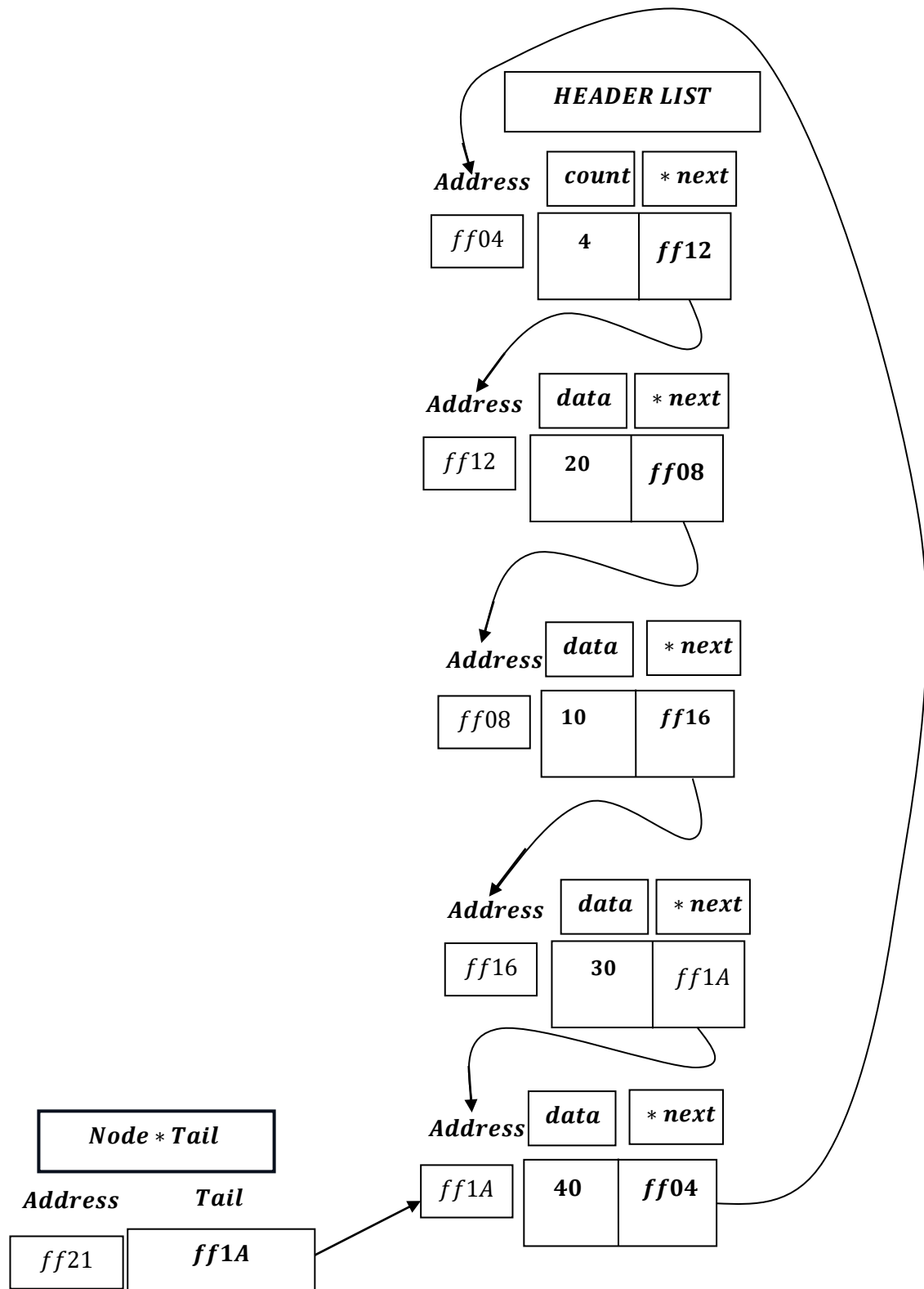
$\left[\begin{array}{l} \text{int variable} - 4 \text{ bytes} \\ \text{padding} - 4 \text{ bytes} \\ \text{Next pointer} - 8 \text{ bytes} \\ \text{prev pointer} - 8 \text{ bytes} \end{array} \right]$

Where, as in 32 – bit system, memory allocation is total 12 bytes.

Where for next pointer it is allocated – 4 bytes, prev pointer it is allocated 4 bytes and for integer variable, memory allocated – 4 bytes.

Hence for 32 – bit system no, such padding is needed.

If we see Header Circular Header Linked List then :



Lets Recall of Memory Allocation of Circular Linked List :

We have seen:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ next(a pointer to another Node) is typically 8 bytes.

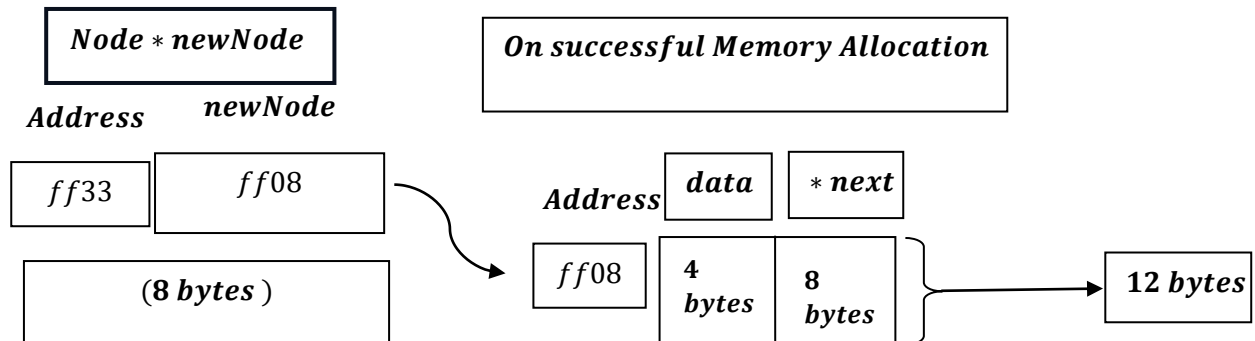
→ Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for next) .

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ newNode will try to allocate memory for data i.e. 4 bytes ,next pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real-World Allocation

In a real programming environment (like the C + + environments) you typically work in

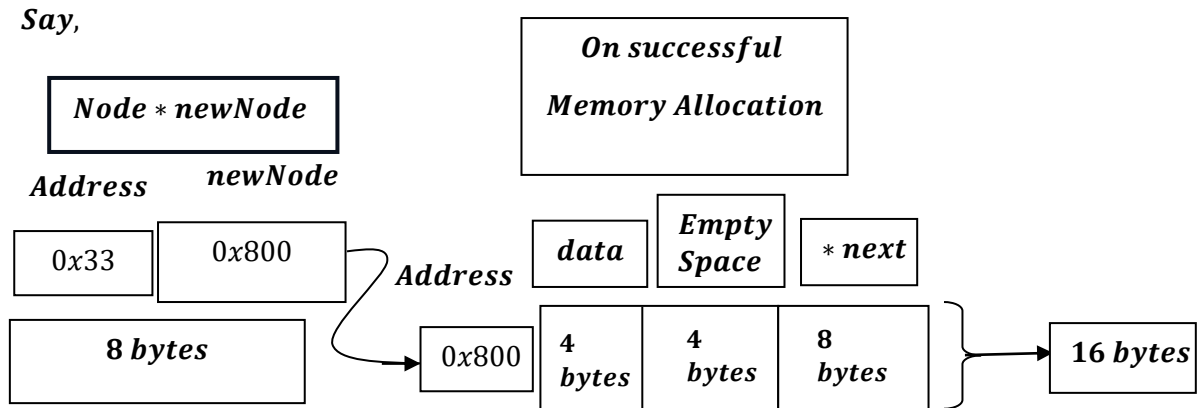
, the allocation might not be exactly 12 bytes because of

how CPUs access memory:

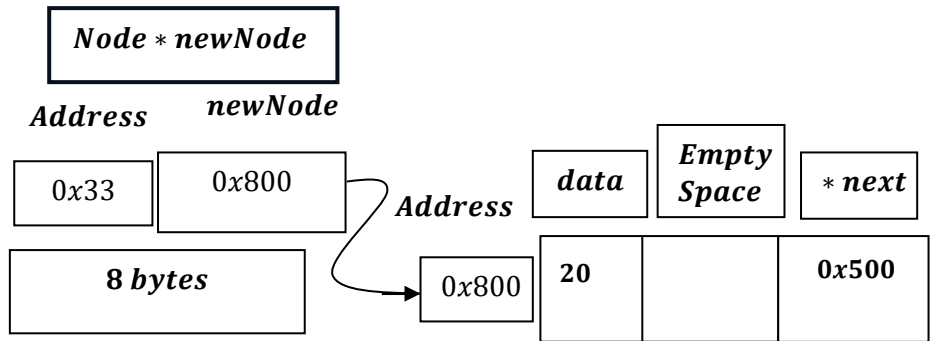
- **Memory Alignment:** Most 64 – bit compilers align data to 8 – byte boundaries to improve performance.
- **Padding:** To align the 8 – byte next pointer, the compiler will often add 4 bytes of "padding" after the 4 – byte data integer.
- **Actual Size:** Consequently, `sizeof(Node)` would likely return 16 bytes on your system, rather than the 12 bytes.

<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0	4 bytes	data	The integer value (e.g., 2).
4	4 bytes	Padding	Empty space added by the compiler.
8	8 bytes	next	The next address of the Node in the linked list (e.g., 0x500).

It will look like:



Like:



In a 64 – bit environment, the compiler aligns memory to the size of the largest primitive member in a structure – in this case, the 8 – byte next pointer.

Because the data integer only occupies 4 bytes, the compiler inserts empty space (padding) to ensure the pointer starts at an address that is a multiple of 8.

Such as:

$$0x500 = 5 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 5 \times 256 = 1280$$

$$1280 \div 8 = 160 \text{ (no remainder)}$$

Why so ?

As, it depends on 32 – bit system and 64 system.

In 64 – bit system, memory allocation is total : 16 bytes.

*[int variable – 4 bytes
padding – 4 bytes
Next pointer – 8 bytes]*

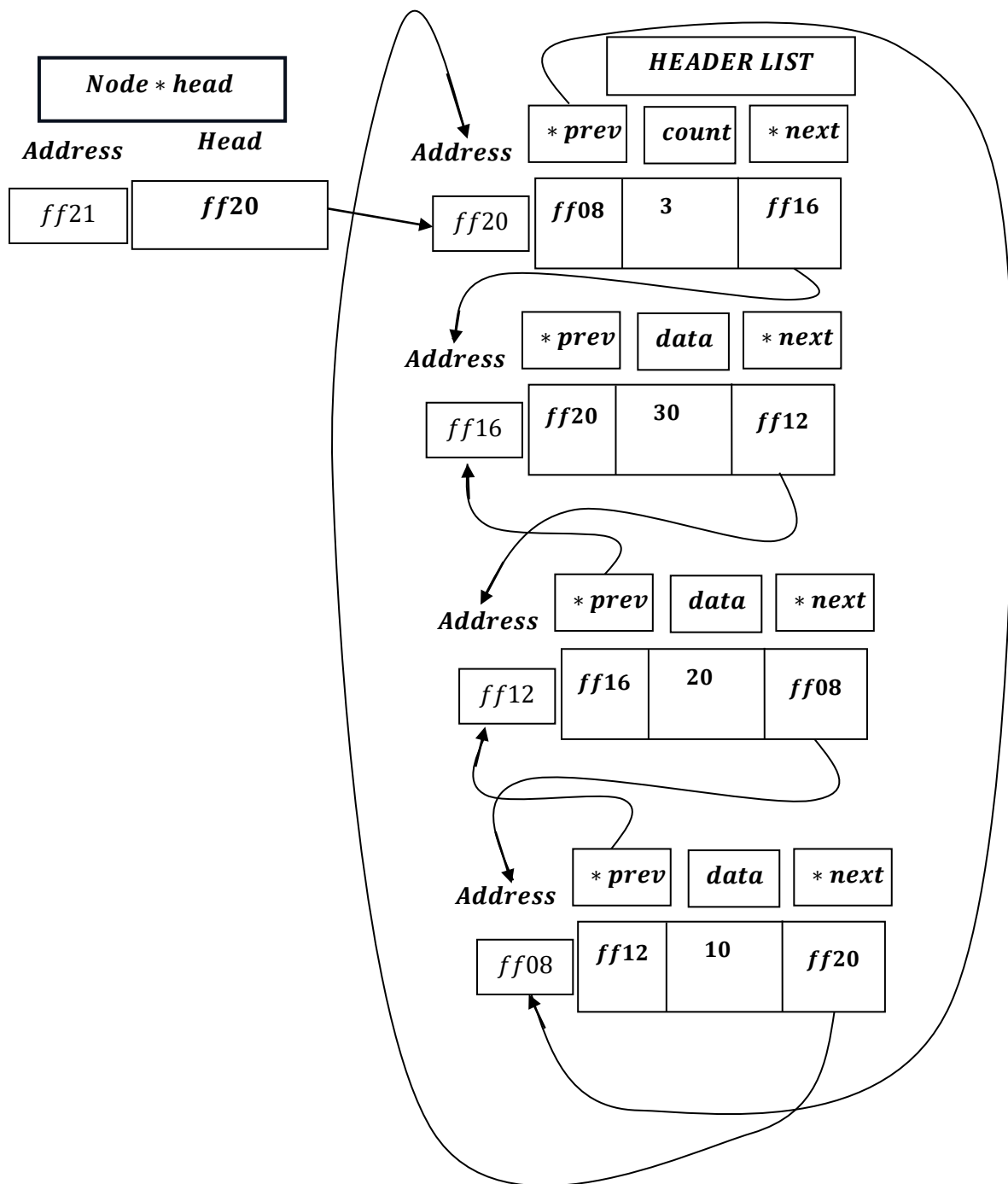
Where, as in 32 – bit system, memory allocation is total 8 bytes.

Where for pointer it is allocated – 4 bytes.

and for integer variable, memory allocated is total – 4 bytes.

Hence for 32 – bit system no, such padding is needed.

If we see Header Circular Header Doubly Linked List then :



Lets Recall of Memory Allocation of Circular Doubly Linked List

We have seen:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ next(a pointer to another Node) is typically 8 bytes.

→ prev(a pointer to previous Node) is typically 8 bytes.

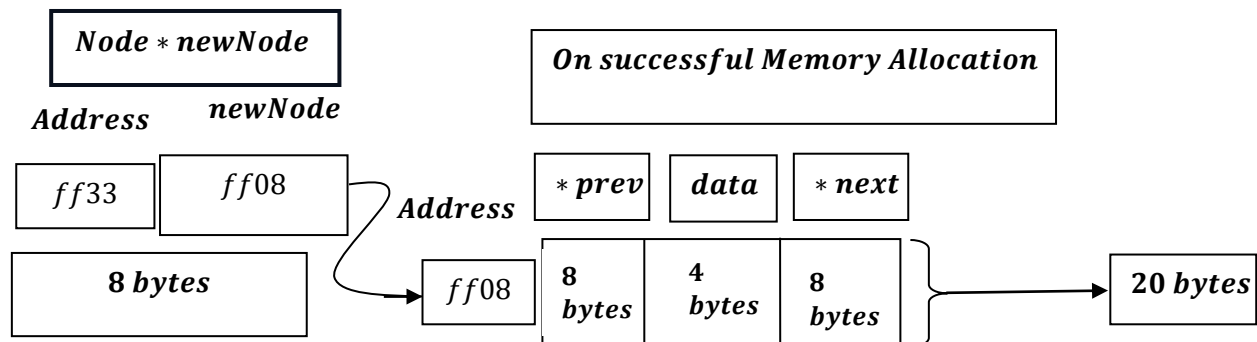
→ Therefore , the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for prev) .

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ newNode will try to allocate memory for data i.e. 4 bytes , next pointer variable i.e. typically 8 bytes, and prev pointer variable that is typically 8 bytes i.e. total 20 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real – World Allocation

In a real programming environment (like the C ++ environments),

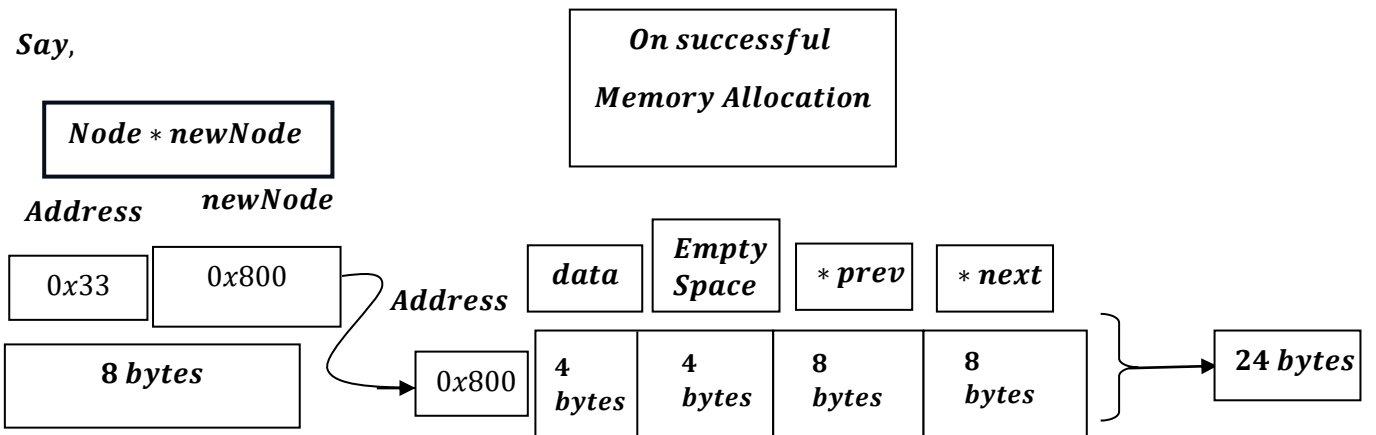
*the allocation might not be exactly **20 bytes** because of how CPUs access memory:*

- **Memory Alignment:** Most 64 – bit compilers align data to **8 – byte boundaries** to improve performance.
- **Padding:** To align the 8 – byte next pointer, the compiler will often add **4 bytes of "padding"** after the 4 – byte data integer.

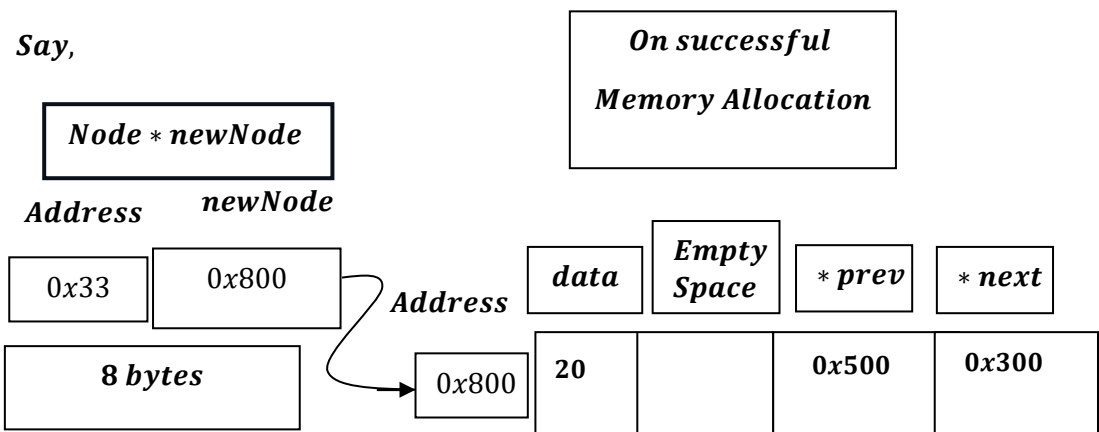
Additionally, since there is another 8 – byte pointer (prev), the structure must maintain proper alignment for both pointers.

- **Actual Size:** Consequently, `sizeof(Node)` would likely return **24 bytes** on your system, rather than the theoretical **20 bytes**.

<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0	4 bytes	<i>data</i>	<i>Integer value</i>
4	4 bytes	<i>padding</i>	<i>Empty space (alignment)</i>
8	8 bytes	<i>next</i>	<i>Address of next node</i>
16	8 bytes	<i>prev</i>	<i>Address of previous node</i>



Like:



In a 64 – bit environment, the compiler aligns memory to the size of the largest primitive member in a structure – in this case, the 8 – byte next pointer.

Because the data integer only occupies 4 bytes, the compiler inserts empty space (padding) to ensure the pointer starts at an address that is a multiple of 8.

Such as:

$$0x500 = 5 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 5 \times 256 = 1280$$

$$1280 \div 8 = 160 \text{ (no remainder)}$$

$$0x300 = 3 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 3 \times 256 = 768$$

$$768 \div 8 = 256 \text{ (no remainder)}$$

Why so ?

As, it depends on 32 – bit system and 64 system.

In 64 – bit system, memory allocation is total : 24 bytes.

$\left[\begin{array}{l} \text{int variable} - 4 \text{ bytes} \\ \text{padding} - 4 \text{ bytes} \\ \text{Next pointer} - 8 \text{ bytes} \\ \text{prev pointer} - 8 \text{ bytes} \end{array} \right]$

Where, as in 32 – bit system, memory allocation is total 12 bytes.

Where for next pointer it is allocated – 4 bytes, prev pointer it is allocated 4 bytes and for integer variable, memory allocated – 4 bytes.

Hence for 32 – bit system no, such padding is needed.
